POLITECNICO DI TORINO

Master's Degree in COURSE



Master's Degree Thesis

TITLE

Supervisors

Prof. Stefano DI CARLO

Dott. Filippo MAROSTICA

Dott. Alessio CAVIGLIA

Candidate

Emanuele BERNACCHI

Novembre 2026

Summary

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Acknowledgements

ACKNOWLEDGMENTS

"HI"
Goofy, Google by Google

Table of Contents

List of Tables VII

List of Figures VIII

Acronyms X

Introduzione 1

1 Fondamenti teorici e stato dell'arte 2
- 1.1 Event camera e rappresentazione degli eventi . . . 2
  - 1.1.1 Dal contrasto luminoso al flusso di eventi . . . 3
  - 1.1.2 Discretizzazione e significato del clock . . . 5
  - 1.1.3 Conteggi e informazione intra-bin . . . 8
  - 1.1.4 Tempo valido, silenzi e padding . . . 12
- 1.2 Dinamica neuronale e memoria nelle Spiking Neural Networks . . . 12
  - 1.2.1 Integrazione, soglia e reset . . . 13
  - 1.2.2 Decadimento discreto e scala temporale fisica . . . 14
  - 1.2.3 Addestramento con surrogate gradient . . . 16
  - 1.2.4 Memoria neuronale e relazioni tra feature . . . 18
- 1.3 Dalle SNN agli Spiking Transformer . . . 19
  - 1.3.1 Self-attention e relazioni tra token . . . 19
  - 1.3.2 La self-attention in forma spiking . . . 22
  - 1.3.3 Attenzione e residui spike-driven . . . 23
  - 1.3.4 La struttura gerarchica . . . 26
- 1.4 QKFormer come riferimento architetturale . . . 28
- 1.5 Strategie per la modellazione temporale esplicita . . . 31
- 1.6 Efficienza neuromorfica e vincoli hardware . . . 34

2 Baseline compatta e metodologia sperimentale 37
- 2.1 Dataset . . . 37
- 2.2 Rappresentazione dell'input . . . 40

2.3 Architettura della baseline compatta 41
2.4 Protocollo sperimentale 46
2.5 Valutazione e incertezza 48
2.6 Caratterizzazione computazionale 49
2.7 Il riferimento nei capitoli successivi 50

3 Riallocazione del budget nella baseline compatta 52

4 Il clock dell'input come variabile sperimentale 53
4.1 I limiti della finestra fissa 54
4.1.1 Code vuote e registrazioni troncate 54
4.1.2 Quando il movimento è più rapido del clock 56
4.2 Tre clock a confronto 58
4.2.1 Due alternative alla finestra fissa 58
4.2.2 Risultati sui quattro dataset 61
4.2.3 Assestamento e costante di tempo 64
4.3 Conseguenze per i confronti successivi 66

5 Memoria temporale relazionale 68
5.1 Dal richiamo del passato al confronto tra passato e presente 68
5.2 Memoria causale a intervalli e disponibilità della storia 72
5.2.1 Dai ritardi puntuali alle bande 72
5.2.2 Quanta storia è davvero disponibile 74
5.2.3 Implementazione e costo 76

A 78

Bibliografia 79

List of Tables

1.1 Rappresentazioni degli eventi a confronto 11
1.2 Evoluzione dei meccanismi di attenzione spiking 28

2.1 Dataset e partizioni di sviluppo 40
2.2 Parametri della rappresentazione a finestra fissa 41
2.3 Struttura della baseline compatta 45
2.4 QKFormer neuromorfico e baseline compatta a confronto 45
2.5 Ricetta di addestramento comune 47
2.6 Grandezze della caratterizzazione computazionale 50
2.7 Il riferimento nei capitoli sperimentali 51

4.1 Calibrazione della rappresentazione a lunghezza variabile 61
4.2 Proprietà dei tre clock dell'input 62
4.3 Accuratezza e Macro-F1 con i tre clock 63

List of Figures

1.1 Generazione degli eventi in un pixel ideale 4
1.2 Quattro suddivisioni di uno stesso flusso di eventi 7
1.3 Event frame, voxel grid e time surface a confronto 10
1.4 Integrazione, soglia e soft reset nei neuroni IF e LIF 15
1.5 Principio del surrogate gradient 18
1.6 L'attenzione come ricerca graduale in un dizionario 21
1.7 Riformulazione della self-attention nel dominio spiking 24
1.8 Connessioni residue SEW e MS 25
1.9 Struttura a scala unica e struttura gerarchica 27
1.10 Tre modalità di accesso al passato 32

2.1 Un campione per ciascun dataset 39
2.2 Architettura della baseline compatta 42

4.1 Coda vuota della rappresentazione a finestra fissa 55
4.2 Dipendenza dalla coda appresa dal modello 55
4.3 Aliasing temporale su una rotazione del braccio 57
4.4 Matrici di confusione sulle classi di rotazione 58
4.5 Una registrazione suddivisa secondo i tre clock 59
4.6 Macro-F1 del modello di riferimento con i tre clock 62
4.7 Accuratezza per quartili di durata della registrazione 64
4.8 Assestamento e costante di tempo su DVS128 Gesture 65

5.1 Dal richiamo al confronto 70
5.2 Ritardi e bande della memoria a intervalli 73
5.3 Storia disponibile nella memoria a intervalli 75

Acronyms

AI

artificial intelligence

Introduzione

L'introduzione sarà redatta al termine della revisione dei capitoli e del consolidamento dei risultati. La traccia seguente ne definisce i contenuti, senza anticipare i contributi definitivi.

- **Contesto e motivazione.** Presentare il problema dell'elaborazione temporale di dati event-based e le ragioni per studiarlo con Spiking Transformer compatti.
- **Problema e domande di ricerca.** Definire quali limiti vengono affrontati e quali relazioni si intendono studiare tra rappresentazione degli eventi, memoria, interazione temporale e capacità del modello.
- **Obiettivi e impostazione metodologica.** Delineare il regime di compattezza, il ruolo dei confronti controllati e la distinzione tra prestazioni predittive e costo computazionale.
- **Contributi ed evidenze.** Enunciare soltanto i contributi sostenuti dai risultati conclusi, precisandone l'ambito di validità e i principali limiti. Distinguere la caratterizzazione software dalle prospettive di implementazione hardware.
- **Organizzazione della tesi.** Presentare la struttura effettiva del documento una volta stabilizzata, evitando di duplicare il background e la metodologia.

Capitolo 1

Fondamenti teorici e stato dell'arte

Questo capitolo raccoglie i fondamenti necessari a comprendere la progettazione di uno Spiking Transformer compatto per dati neuromorfici. L'esposizione segue il cammino dell'informazione, dal sensore alla sua rappresentazione discreta, dalla memoria del singolo neurone alle relazioni tra feature, fino al costo computazionale e di memoria che ogni scelta comporta.

La sezione 1.1 affronta il primo passaggio obbligato, la conversione del flusso asincrono di eventi in tensori regolari, e mostra come questa operazione, apparentemente preliminare, costituisca già una forma di modellazione. La sezione 1.2 descrive la memoria intrinseca dei neuroni spiking e i limiti di uno stato che riassume ricorsivamente il passato. Le sezioni 1.3 e 1.4 ricostruiscono l'incontro tra attenzione e computazione spiking fino a QKFormer, l'architettura da cui deriva la baseline compatta, distinguendo la memoria dei neuroni dalle interazioni esplicite tra istanti diversi. La sezione 1.5 esamina due modi di rendere esplicito l'accesso alla storia, l'attenzione spazio-temporale e il mixing mediante ritardi, mentre la sezione 1.6 introduce le grandezze con cui se ne misura il costo.

Non si tratta di una rassegna esaustiva. Ogni contributo discusso è stato selezionato perché orienta una scelta progettuale compiuta nel seguito del lavoro, ed è presentato in funzione di essa.

1.1 Event camera e rappresentazione degli eventi

Una fotocamera convenzionale scatta fotografie a intervalli regolari, per esempio trenta al secondo, e ogni fotografia registra la luminosità di tutti i pixel, anche di quelli in cui non è cambiato nulla. Una event camera funziona in modo opposto. Ogni pixel resta in silenzio finché la luce che riceve non cambia abbastanza, e in

quel momento segnala il cambiamento con un *evento*. Una scena immobile non produce quasi nulla, mentre una mano che si muove produce una scia di eventi lungo i propri contorni, nell'istante esatto in cui si sposta.

Per elaborare questo segnale con una rete neurale, che lavora su tensori di dimensioni fisse, occorre raccogliere gli eventi in una struttura regolare. Questa sezione descrive prima il segnale prodotto dal sensore e poi le scelte necessarie per trasformarlo in un tensore. Sono scelte tutt'altro che neutre, perché decidono quali informazioni arrivano alla rete e quali vanno perse prima ancora che l'elaborazione cominci.

1.1.1 Dal contrasto luminoso al flusso di eventi

Nel modello ideale di un Dynamic Vision Sensor (DVS) ogni pixel lavora indipendentemente dagli altri e osserva il logaritmo dell'intensità luminosa che lo colpisce, $L(x, y, t) = \log I(x, y, t)$ [1]. Il pixel conserva un valore di riferimento, cioè il livello di L al momento del suo ultimo evento, e lo confronta di continuo con il valore attuale. Quando la differenza raggiunge una soglia $C > 0$, verso l'alto o verso il basso, il pixel emette un evento e aggiorna il riferimento. In formule, l'evento i-esimo viene emesso nell'istante t_i in cui

$$L(x_i, y_i, t_i) - L(x_i, y_i, t_i^{\text{ref}}) = p_i\, C, \tag{1.1}$$

dove t_i^{ref} è l'istante a cui risale il riferimento, cioè l'evento precedente dello stesso pixel o la sua inizializzazione, e la polarità p_i vale $+1$ se la luce è aumentata e -1 se è diminuita [2]. Ogni evento è quindi descritto da quattro numeri,

$$e_i = (x_i, y_i, t_i, p_i), \tag{1.2}$$

la colonna e la riga del pixel, l'istante e la polarità. Un evento con $x_i = 64$, $y_i = 30$, $t_i = 1\,250\,400\,\mu$s e $p_i = +1$, per esempio, dice che il pixel della colonna 64 e della riga 30 è diventato più luminoso 1,25 secondi dopo l'inizio della registrazione.

La figura 1.1 mostra il meccanismo su un singolo pixel, attraversato prima da un oggetto chiaro e poi da uno scuro. Finché la luce cresce, ogni incremento pari a C produce un evento positivo, quando cala ogni decremento pari a C ne produce uno negativo. Due aspetti meritano attenzione. La soglia agisce sulla variazione accumulata dall'ultimo evento, non sulla velocità del cambiamento, sicché nel modello ideale una variazione lenta e una rapida della stessa entità producono lo stesso numero di eventi, soltanto distribuiti in tempi diversi. Le oscillazioni più piccole della soglia, invece, non producono nulla.

Perché proprio il logaritmo? Perché la differenza tra due logaritmi è il logaritmo del rapporto,

$$\Delta L = \log \frac{I(x_i, y_i, t_i)}{I(x_i, y_i, t_i^{\text{ref}})}, \tag{1.3}$$

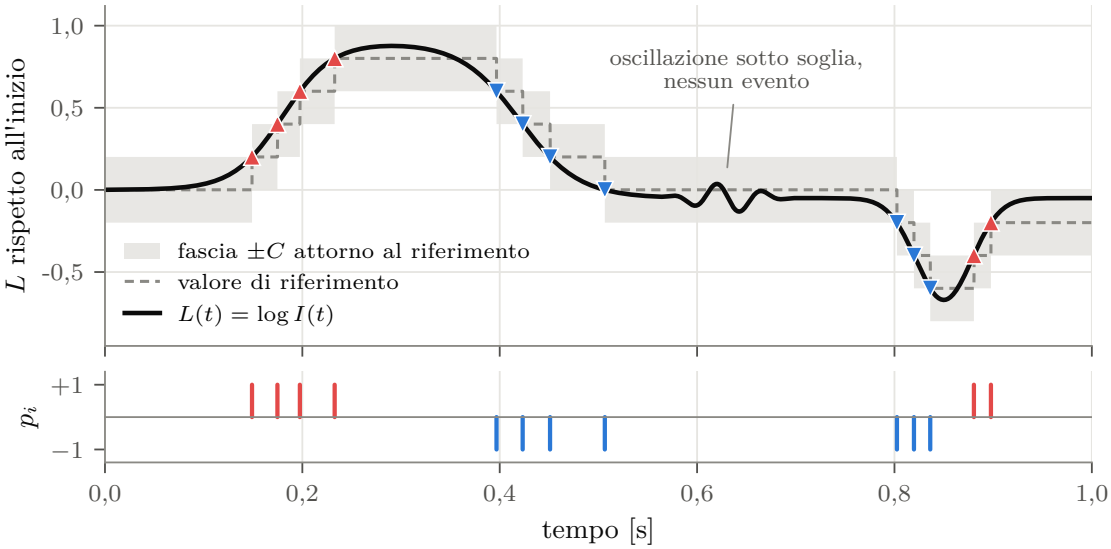


Figura 1.1: Generazione degli eventi in un pixel ideale, su un segnale sintetico con soglia $C = 0{,}2$. In alto, il logaritmo dell'intensità (linea continua), il valore di riferimento del pixel (tratteggio) e la fascia di ampiezza $\pm C$ attorno a esso. Ogni volta che la curva esce dalla fascia il pixel emette un evento, positivo (triangolo rosso) o negativo (triangolo blu), e il riferimento si sposta di C. In basso, la sequenza di eventi che il pixel trasmette. L'oscillazione centrale, più piccola della soglia, non produce alcun evento.

e il pixel reagisce quindi alle variazioni relative della luce, non a quelle assolute. Con una soglia $C = 0{,}2$, scelta qui a titolo illustrativo, un evento positivo richiede un aumento della luce di circa il 22%, poiché $e^{0{,}2} \approx 1{,}22$. Vale allo stesso modo per un pixel in piena luce che passa da 1000 a 1220 unità e per uno in penombra che passa da 10 a 12,2. La stessa scena produce così eventi simili in condizioni di illuminazione molto diverse, ed è una delle ragioni dell'ampia gamma dinamica di questi sensori [2].

La polarità, va precisato, dice soltanto se in quel pixel la luce è aumentata o diminuita, non in quale direzione si muova qualcosa. Si pensi a una barra scura che scorre verso destra su uno sfondo chiaro. I pixel raggiunti dal suo bordo anteriore si scuriscono ed emettono eventi negativi, quelli lasciati dal bordo posteriore tornano chiari ed emettono eventi positivi. Se la barra scorresse verso sinistra, ogni pixel emetterebbe esattamente la stessa sequenza, e la direzione si potrebbe leggere soltanto nell'ordine con cui si attivano pixel vicini. Allo stesso modo, un evento non implica necessariamente un movimento, perché anche una lampada che sfarfalla fa cambiare la luce di molti pixel insieme. Il sensore reale, infine, si allontana dal modello ideale in vari modi. Le soglie positiva e negativa possono differire tra loro e variare da pixel a pixel, e il rumore, la banda limitata del fotorecettore e un breve periodo refrattario dopo ogni evento aggiungono altri scostamenti [2].

Una registrazione è dunque una lunga sequenza di eventi in ordine di tempo,

$$\mathcal{E} = \{e_i\}_{i=1}^{N}, \qquad t_i \leq t_{i+1}. \tag{1.4}$$

Il numero N degli eventi non dipende dalla durata ma da quanto accade nella scena. Nei dataset usati in questo lavoro, per esempio, una parola pronunciata in circa un secondo produce in media poco più di diecimila eventi, mentre un'immagine di CIFAR10-DVS fatta muovere davanti al sensore per un tempo simile ne produce circa duecentomila (sezione 2.1). Anche all'interno della stessa registrazione possono alternarsi lunghi silenzi e raffiche improvvise, e chi elabora il segnale deve saper gestire gli uni e le altre.

1.1.2 Discretizzazione e significato del clock

Una rete neurale spiking elabora l'ingresso a passi discreti. Per presentarle un flusso di eventi bisogna quindi decidere che cosa entri in ciascun passo, cioè come tagliare il flusso in pezzi. La regola che associa ogni passo a un intervallo di tempo, o più in generale a un gruppo di eventi, prende il nome di *clock* dell'input. Le regole possibili sono diverse, e la figura 1.2 le confronta sullo stesso flusso.

La più semplice divide il tempo in intervalli consecutivi della stessa durata Δt, detti bin, a partire da un'origine t_0. Il bin k-esimo raccoglie gli eventi caduti nel suo intervallo,

$$\mathcal{E}_k = \Big\{ e_i \in \mathcal{E} \;\Big|\; t_0 + k\Delta t \leq t_i < t_0 + (k+1)\Delta t \Big\}. \tag{1.5}$$

Con $t_0 = 0$ e $\Delta t = 50\,\text{ms}$, un evento all'istante $t_i = 123\,\text{ms}$ finisce nel bin $k = 2$, quello che va da 100 a 150 ms. Il bordo sinistro è incluso e quello destro escluso, così che un evento che cade esattamente a 150 ms appartenga al bin successivo e mai a due bin insieme. In questo schema ogni passo dura lo stesso tempo, mentre il numero di eventi che contiene cambia liberamente da un passo all'altro.

Fissata la durata dei bin, resta da decidere quanti passi elaborare. La soluzione più comune è una finestra di durata prefissata, $D_{\text{win}} = T\Delta t$, che produce sempre T passi. Con bin da 50 ms e una finestra di 2 s i passi sono 40. Una registrazione di 1,2 s ne riempie i primi 24 e lascia vuoti gli ultimi 16, mentre di una registrazione di 2,5 s vanno persi gli ultimi 0,5 s.

In alternativa si può conservare l'intera registrazione, dal primo evento, che fa da origine t_0, all'ultimo, che cade all'istante t_{last}, mantenendo bin della stessa durata. Il numero di passi diventa allora

$$T = \left\lfloor \frac{t_{\text{last}} - t_0}{\Delta t} \right\rfloor + 1. \tag{1.6}$$

La parte intera conta i bin completamente trascorsi prima dell'ultimo evento, e l'unità aggiunta fa posto al bin che lo contiene. Con bin da 50 ms e un ultimo evento a 1199 ms si ottengono 24 passi. Se l'ultimo evento cadesse esattamente a 1200 ms, sul bordo sinistro di un nuovo bin, i passi sarebbero 25, ed è proprio l'unità aggiunta a garantire che quell'evento trovi posto. In questo schema la durata dei bin è la stessa per tutte le registrazioni, e a cambiare è la lunghezza della sequenza.

Una terza possibilità fa il contrario, fissa il numero di passi T e adatta la durata dei bin a ciascuna registrazione n, di durata D_n,

$$\Delta t_n = \frac{D_n}{T}. \tag{1.7}$$

Con $T = 40$, un gesto di 4 s viene diviso in bin da 100 ms e uno di 8 s in bin da 200 ms. Il clock cambia così da un campione all'altro. Una distanza di cinque passi corrisponde a mezzo secondo nel primo gesto e a un secondo intero nel secondo, e la rete non ha modo di saperlo. Fissare il numero dei bin, in altre parole, significa non fissare la risoluzione temporale.

Si può infine rinunciare del tutto a misurare il tempo e contare gli eventi. Una prima variante chiude un gruppo ogni M eventi, nell'ordine in cui arrivano. Una seconda divide l'intera registrazione in T gruppi che contengono lo stesso numero di eventi, a meno di uno,

$$n_k \in \left\{ \left\lfloor \frac{N}{T} \right\rfloor, \left\lceil \frac{N}{T} \right\rceil \right\}, \qquad \sum_{k=1}^{T} n_k = N, \tag{1.8}$$

dove n_k è il numero di eventi del gruppo k. Con $N = 10\,003$ eventi e $T = 40$ gruppi, per esempio, tre gruppi ricevono 251 eventi e gli altri 37 ne ricevono 250. In questo

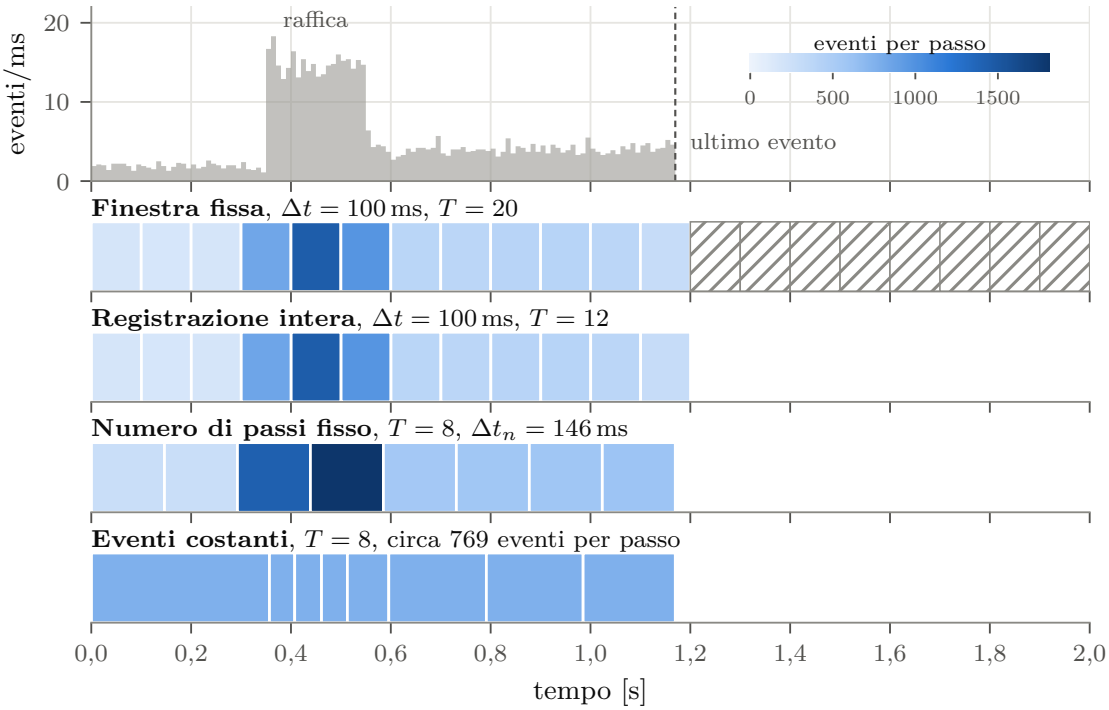


Figura 1.2: Quattro modi di suddividere lo stesso flusso sintetico, che contiene una raffica di attività tra 0,35 e 0,55 s e termina a circa 1,17 s. In alto, la frequenza degli eventi. Sotto, i passi prodotti da ciascuna regola, colorati in base al numero di eventi che contengono. La finestra fissa lascia una coda di passi vuoti (tratteggiati), la registrazione intera termina con l'ultimo evento, il numero di passi fisso allarga i bin per coprire la durata, la partizione a eventi costanti produce passi con lo stesso contenuto ma di durata molto diversa, stretti durante la raffica e larghi altrove.

schema ogni passo contiene la stessa quantità di attività, ma la sua durata dipende da quanto è intensa la scena. Una raffica concentrata in un decimo di secondo può riempire molti gruppi, mentre un secondo di quiete può starne in uno solo. La seconda variante richiede di conoscere in anticipo il numero totale N degli eventi, e si può applicare soltanto a registrazioni già concluse, cioè offline. La prima non ha questo limite, perché ogni gruppo si chiude non appena raggiunge M eventi. In entrambi i casi, però, una distanza misurata in gruppi non corrisponde a un intervallo di tempo fisso.

Quando i passi seguono il numero di eventi, la loro durata non è più fissata in anticipo ma diventa una proprietà di ciascun gruppo, misurabile come intervallo (*span*) tra il suo primo e il suo ultimo evento. Se la rete riceve soltanto i conteggi, però, questa durata le resta invisibile. Un gruppo di 250 eventi raccolti in 5 ms e uno raccolto in mezzo secondo occupano entrambi esattamente un passo.

La scelta del clock non è dunque un dettaglio di preprocessing. Da essa dipende il significato fisico di ogni ritardo espresso in passi e di ogni costante di decadimento della rete. Dimezzare Δt su una stessa registrazione raddoppia il numero dei passi, dimezza in media i conteggi di ciascun bin e fa sì che un neurone che perde una frazione fissa del proprio potenziale a ogni passo dimentichi due volte più in fretta in tempo reale. I protocolli adottati in questo lavoro sono descritti nel capitolo 2 e messi a confronto nel capitolo 4.

1.1.3 Conteggi e informazione intra-bin

Una volta suddiviso il flusso, bisogna decidere che cosa conservare di ciascun bin. La rappresentazione più semplice conta, per ogni pixel e per ogni bin, quanti eventi di ciascuna polarità vi sono caduti,

$$F_k^p(x,y) = \sum_{e_i \in \mathcal{E}_k} \mathbf{1}[x_i = x,\ y_i = y,\ p_i = p], \qquad F_k \in \mathbb{N}_0^{2\times H\times W}. \tag{1.9}$$

L'indicatore $\mathbf{1}[\cdot]$ vale 1 quando la condizione tra parentesi è vera e 0 altrimenti, quindi la somma conta semplicemente gli eventi del bin k caduti nel pixel (x, y) con polarità p. Il risultato è un'immagine a due canali, uno per gli eventi positivi e uno per quelli negativi, spesso chiamata *event frame* (figura 1.3a). Se in un bin un pixel riceve tre eventi positivi e uno negativo, il canale positivo vale 3 e quello negativo 1.

Esistono versioni più povere della stessa idea. Una versione binaria, $\mathbf{1}[F_k^p(x,y) > 0]$, registra soltanto se il pixel è stato attivo, e nell'esempio darebbe 1 in entrambi i canali. Una versione a un solo canale somma gli eventi con il loro segno, e darebbe $3 - 1 = 2$. Quest'ultima ha un difetto evidente, perché due eventi positivi e due negativi nello stesso pixel si annullano e producono zero, come se non fosse accaduto nulla. Tenere separate le polarità evita il problema.

Tutte queste versioni condividono però un limite più sottile. Una somma non dipende dall'ordine degli addendi, e i conteggi di un bin non cambiano se gli eventi al suo interno vengono rimescolati. Si pensi a un oggetto chiaro che attraversa rapidamente un pixel scuro. Il pixel prima si illumina e poi torna scuro, ed emette un evento positivo seguito da uno negativo. Un oggetto scuro che attraversa un pixel chiaro produce la sequenza opposta. Se entrambi gli eventi cadono nello stesso bin, i due casi danno esattamente gli stessi conteggi, uno per canale, e nessuna rete, per quanto sofisticata, potrà distinguerli sulla base di quel solo bin. La rappresentazione fissa un limite all'informazione disponibile, che va tenuto distinto dalla capacità della rete di sfruttare quella che rimane.

La *voxel grid* recupera parte dell'informazione temporale perduta [3]. Si considera un pacchetto di eventi consecutivi, elaborato come un'unica unità, e se ne riscalano

i timestamp in modo che il primo evento valga 0 e l'ultimo $B-1$,

$$t_i^* = (B-1)\frac{t_i - t_{\min}}{t_{\max} - t_{\min}}, \tag{1.10}$$

dove B è il numero di bin temporali desiderati e il pacchetto deve avere durata non nulla, $t_{\max} > t_{\min}$. Ogni evento viene poi distribuito tra i due bin più vicini alla sua posizione t_i^*,

$$\mathcal{V}(b,x,y) = \sum_i p_i \max\bigl(0,\, 1 - |b - t_i^*|\bigr)\mathbf{1}[x_i = x,\ y_i = y], \qquad b \in \{0,\dots,B-1\}. \tag{1.11}$$

Il peso $\max(0, 1-|b-t_i^*|)$ vale 1 quando l'evento cade esattamente sul bin b, decresce in modo lineare con la distanza e si annulla a un bin di distanza. Con $B = 5$, un evento positivo con $t_i^* = 2{,}3$ aggiunge 0,7 al bin 2 e 0,3 al bin 3, e niente agli altri. L'evento non viene attribuito tutto a un bin o tutto all'altro, e la sua posizione temporale sopravvive, sfumata, nella ripartizione dei pesi (figura 1.3b). La formulazione completa interpola anche nello spazio, quando la correzione della distorsione ottica produce coordinate non intere [3].

Due dettagli vanno tenuti presenti. La somma usa la polarità con il suo segno, e può quindi produrre le stesse cancellazioni dell'event frame a un canale, che volumi separati per eventi positivi e negativi eviterebbero. Inoltre la scala temporale è relativa a ciascun pacchetto. Con $B = 5$, un intervallo di 10 ms occupa meno di mezzo bin in un pacchetto lungo 100 ms e ben due bin in uno lungo 20 ms, e non va confuso con un clock fisico comune a tutti i campioni.

La *time surface* adotta un punto di vista diverso. Invece di contare gli eventi, descrive quanto sia recente l'attività di ciascun pixel. Nella forma più semplice,

$$S_p(x,y,t) = \exp\left[-\frac{t - t^p_{\text{last}}(x,y)}{\tau_s}\right], \tag{1.12}$$

dove $t^p_{\text{last}}(x,y)$ è l'istante dell'ultimo evento di polarità p arrivato al pixel prima di t, e il valore è zero dove non è ancora arrivato nulla. Un pixel appena attivato vale circa 1, e il suo valore decade poi in modo esponenziale con costante τ_s. Con $\tau_s = 50$ ms, un pixel attivato 10 ms prima vale circa 0,82, uno attivato 100 ms prima circa 0,14. Il risultato somiglia alla scia di una cometa, intensa dove il movimento è appena passato e sempre più tenue alle sue spalle (figura 1.3c). È la variante elementare di una famiglia più ampia. HATS, per esempio, somma i contributi esponenziali di più eventi in un intorno spazio-temporale e li aggrega in descrittori [4].

Event frame, voxel grid e time surface riassumono dunque aspetti diversi della stessa storia, la quantità dell'attività, la sua collocazione nel tempo e la sua recenza. La tabella 1.1 ne riassume i compromessi. Una rappresentazione più ricca costa di più da costruire e da conservare, e la sua utilità dipende in ultima analisi dalle operazioni che il modello è in grado di applicarvi.

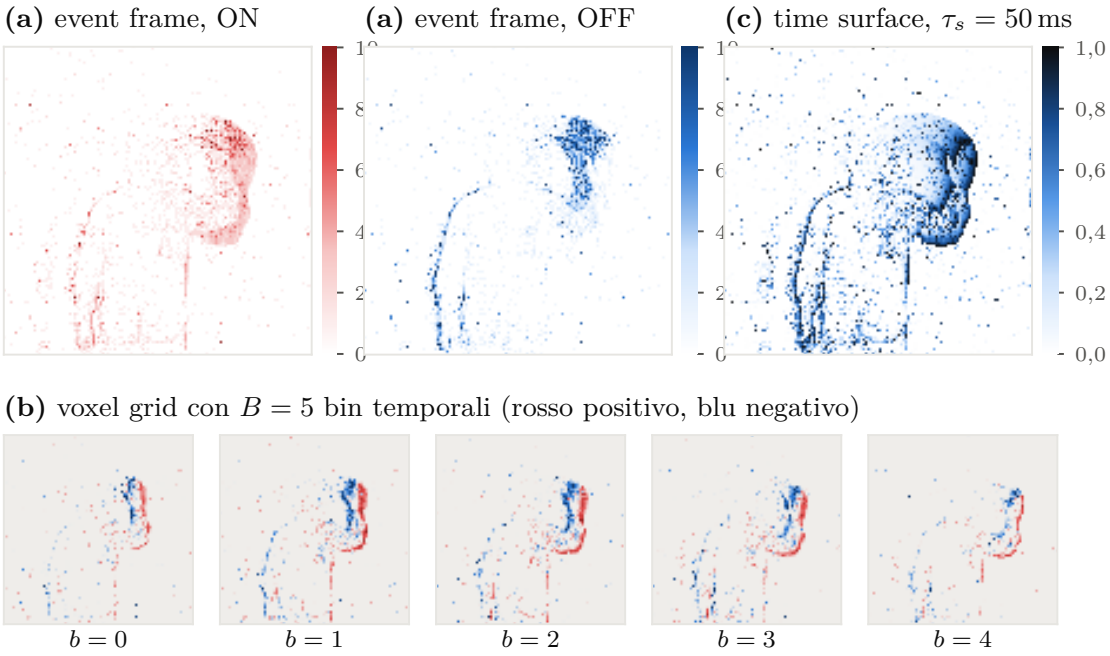


Figura 1.3: Tre rappresentazioni degli stessi 100 ms di una registrazione di DVS128 Gesture, durante una rotazione del braccio sinistro (circa 9 600 eventi). (a) Event frame, con i conteggi degli eventi positivi e negativi in due canali separati. (b) Voxel grid con $B = 5$ bin temporali, in cui la distribuzione dell'attività cambia da un bin all'altro, con i valori positivi in rosso e quelli negativi in blu. (c) Time surface dell'ultimo evento di entrambe le polarità, valutata alla fine dell'intervallo, in cui la posizione più recente del movimento appare come una scia.

Tabella 1.1: Informazione conservata e principali limiti delle rappresentazioni considerate.

Formato	**Informazione conservata**	**Descrizione del tempo**	**Limite principale**
Stream raw	Posizione, polarità e timestamp dei singoli eventi	Istanti originari	Sequenza irregolare e carico variabile
Conteggi ON/OFF	Molteplicità per pixel e polarità	Appartenenza al bin	Ordine intra-bin eliminato
Voxel grid	Accumuli su più bin temporali	Posizione temporale discretizzata o interpolata	Aggregazione e possibile cancellazione nella versione firmata
Time surface dell'ultimo evento	Recenza per pixel e polarità	Tempo trascorso dall'ultima attivazione	Molteplicità e storia anteriore non rappresentate separatamente

1.1.4 Tempo valido, silenzi e padding

Quando le sequenze hanno lunghezze diverse sorge un problema pratico. Le reti vengono addestrate su batch, cioè gruppi di campioni elaborati insieme in un unico tensore, e un tensore richiede che tutte le sequenze abbiano la stessa lunghezza. Le sequenze più brevi vengono quindi allungate con passi fittizi, il cosiddetto *padding*, fino alla lunghezza della più lunga, $T_{\max}$. Per tenere traccia di quali passi siano veri, a ogni campione n di lunghezza T_n si associa una maschera

$$m_{k,n} = \mathbf{1}[k < T_n], \qquad k = 0, \dots, T_{\max} - 1, \tag{1.13}$$

che vale 1 sui passi osservati e 0 su quelli aggiunti. In un batch di tre sequenze lunghe 25, 33 e 40 passi, per esempio, tutte vengono portate a 40, e la maschera della prima vale 1 sui primi 25 passi e 0 sugli ultimi 15.

Un passo di padding e un bin valido senza eventi possono contenere entrambi soltanto zeri, ma significano cose opposte. Il bin vuoto dice che è trascorso del tempo senza attività, ed è a tutti gli effetti un'osservazione. Il padding non osserva nulla. Confonderli altera il campione. Se la rete calcolasse la media delle proprie feature su tutti i 40 passi anziché sui 25 validi, il risultato della prima sequenza verrebbe ridotto di oltre un terzo senza alcuna ragione legata ai dati. Lo stesso accade se il padding entra nelle statistiche della normalizzazione, e un modulo dotato di stato interno deve far avanzare il proprio tempo soltanto sui passi validi.

C'è infine una terza situazione, diversa da entrambe. Quando la registrazione finisce prima della finestra prevista, come nell'esempio della finestra di 2 s, i passi successivi sono validi ma privi di eventi. La rete, però, non resta necessariamente inattiva, perché i potenziali di membrana e le memorie interne continuano a evolvere anche senza ingresso. Questa coda vuota e il padding di un batch vanno quindi dichiarati separatamente.

L'intera catena di elaborazione si può riassumere nello schema

$$\mathcal{E} \xrightarrow{\mathcal{R}} X_{1:T} \xrightarrow{\mathcal{M}} Z_{1:T} \xrightarrow{\mathcal{D}} \hat{y}, \tag{1.14}$$

dove $\mathcal{R}$ costruisce la rappresentazione, $\mathcal{M}$ ne elabora la dinamica e $\mathcal{D}$ prende la decisione. Ognuno dei tre stadi può limitare l'uso dell'informazione temporale. Per questo, quando si confrontano architetture diverse, rappresentazione e clock vanno tenuti fissi, e quando è l'ingresso stesso l'oggetto dell'esperimento ogni sua modifica va dichiarata come tale.

1.2 Dinamica neuronale e memoria nelle Spiking Neural Networks

Un neurone artificiale convenzionale, come quello di un percettrone multistrato con attivazione ReLU, calcola la propria uscita a partire dal solo ingresso corrente. Se

riceve due volte lo stesso ingresso risponde due volte allo stesso modo, e non conserva nulla da un'elaborazione all'altra. Il neurone di una Spiking Neural Network (SNN) si comporta diversamente. Possiede una variabile interna, il potenziale di membrana, che accumula gli ingressi ricevuti e sopravvive da un passo al successivo, e la sua risposta dipende quindi anche da ciò che è accaduto prima. È la struttura di una rete ricorrente, con una differenza importante. In una rete ricorrente classica il passato rientra attraverso connessioni esplicite che riportano l'uscita di uno strato al suo ingresso, mentre in una SNN la ricorrenza è già dentro ogni neurone, e la rete ha memoria anche quando i suoi strati sono collegati soltanto in avanti [5].

Questa sezione descrive come un neurone accumuli e dimentichi gli ingressi, come la velocità con cui dimentica dipenda dal clock dell'input, come si addestri una rete che comunica con impulsi discontinui e che cosa una memoria di questo tipo permetta di fare.

1.2.1 Integrazione, soglia e reset

Il modello più semplice è l'Integrate-and-Fire (IF), che si può immaginare come un accumulatore. A ogni passo il neurone somma al proprio potenziale il contributo dell'ingresso,

$$H_k = V_{k-1} + X_k, \tag{1.15}$$

e quando il potenziale raggiunge una soglia V_{th} emette uno spike,

$$S_k = \Theta(H_k - V_{\text{th}}), \qquad \Theta(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases} \tag{1.16}$$

La funzione a gradino Θ traduce il confronto con la soglia in una risposta tutto o niente. Lo spike vale 1 se il potenziale ha raggiunto la soglia e 0 altrimenti, senza valori intermedi, e un potenziale appena sopra la soglia produce lo stesso spike di uno che la supera di molto.

Le due equazioni usano due simboli diversi per il potenziale, e conviene chiarire subito perché. All'interno di ogni passo avvengono due cose in sequenza. Prima il neurone riceve l'ingresso e si carica, e H_k indica il potenziale in questo momento, prima che il neurone decida se emettere lo spike. Poi, se lo spike è stato emesso, il potenziale viene riportato verso il basso, ed è il valore che resta dopo questa operazione, indicato con V_k, a passare al passo successivo. Se lo spike non viene emesso, i due valori coincidono.

L'operazione che segue lo spike si chiama reset, e ne esistono due varianti. L'*hard reset* riporta il potenziale a un valore fisso, di solito zero, mentre il *soft reset*, adottato in questo lavoro, si limita a sottrarre la soglia,

$$V_k = H_k - S_k V_{\text{th}}. \tag{1.17}$$

La differenza emerge quando il potenziale supera la soglia di un certo margine. Con soglia 1 e $H_k = 1{,}3$, l'hard reset riparte da zero e perde l'eccedenza di 0,3, mentre il soft reset riparte da 0,3 e la conserva per il passo successivo [6]. Qui e nel seguito si adotta la convenzione con potenziale di riposo nullo.

Il neurone IF ha una memoria perfetta. Al di fuori del reset, un contributo ricevuto all'inizio della sequenza pesa esattamente quanto uno appena arrivato, perché il modello non prevede alcuna forma di oblio.
Il Leaky Integrate-and-Fire (LIF) introduce proprio questo elemento, una perdita o *leak*. A ogni passo, prima di sommare il nuovo ingresso, il potenziale perde una frazione fissa del proprio valore,

$$H_k = \lambda V_{k-1} + \eta X_k, \qquad 0 < \lambda < 1. \tag{1.18}$$

Il fattore di decadimento λ indica quale frazione del potenziale sopravvive da un passo al successivo, e il coefficiente η stabilisce quanto pesi l'ingresso. Con $\lambda = 0{,}5$, in assenza di ingresso, il potenziale si dimezza a ogni passo. Il neurone IF si ritrova ponendo $\lambda = 1$ ed $\eta = 1$.

La perdita cambia in modo sostanziale il comportamento del neurone, come mostra la figura 1.4. Un neurone con soglia 1 riceve tre ingressi da 0,6, una volta in tre passi consecutivi e una volta a tre passi di distanza l'uno dall'altro. Il neurone IF emette uno spike in entrambi i casi, non appena il secondo ingresso porta il potenziale a 1,2. Il neurone LIF con $\lambda = 0{,}5$ ed $\eta = 1$, quando gli ingressi sono ravvicinati, sale a 0,6, poi a 0,9 e infine a 1,05, e supera la soglia al terzo passo. Quando invece sono distanziati, tra un ingresso e l'altro il potenziale ha il tempo di scendere, non va mai oltre 0,68 e lo spike non arriva. A parità di ingresso complessivo, il LIF risponde agli ingressi che arrivano vicini e lascia cadere quelli isolati, e si comporta in questo senso come un rilevatore di coincidenze.

1.2.2 Decadimento discreto e scala temporale fisica

Il fattore λ è definito per passo, mentre la memoria di un neurone, dal punto di vista della scena osservata, si misura in millisecondi. Per collegare le due cose bisogna risalire al modello da cui il LIF discende. Nella formulazione originale, a tempo continuo, la membrana si comporta come un condensatore che si scarica lentamente attraverso una resistenza,

$$\tau_m \frac{\mathrm{d}V(t)}{\mathrm{d}t} = -\Big(V(t) - V_{\mathrm{rest}}\Big) + RI(t), \tag{1.19}$$

dove $I(t)$ è la corrente d'ingresso, R la resistenza di membrana, V_{rest} il potenziale di riposo e τ_m la costante di tempo di membrana [7]. Il primo termine a destra riporta il potenziale verso il riposo con una spinta proporzionale alla distanza, il secondo lo sposta in base all'ingresso. Se l'ingresso si annulla, il potenziale torna

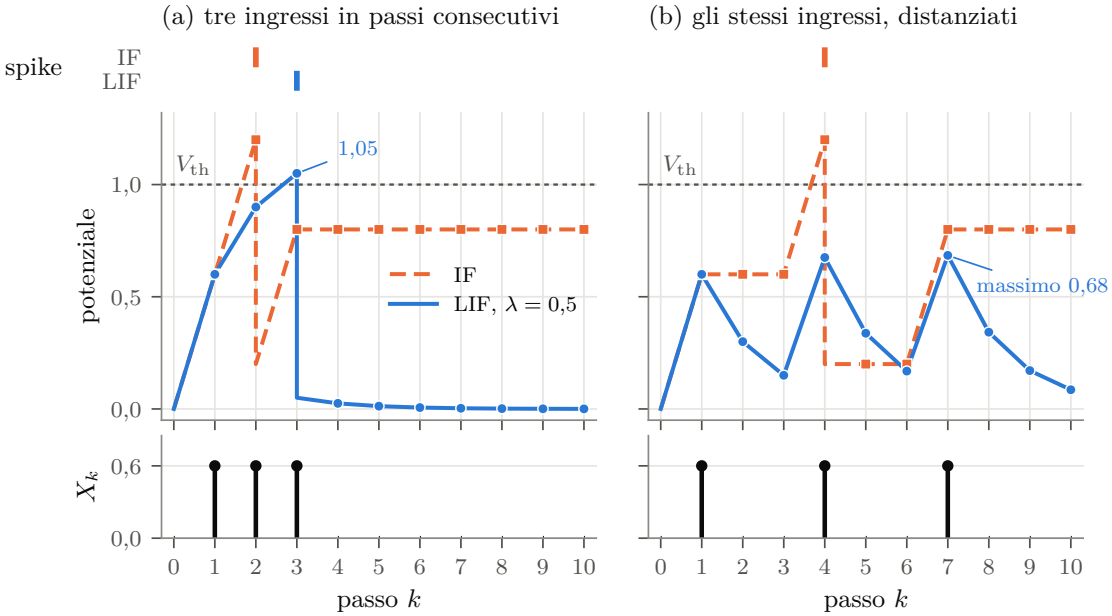


Figura 1.4: Integrazione, soglia e soft reset in un neurone IF (tratteggio arancione) e in un neurone LIF con $\lambda = 0{,}5$ (linea blu), entrambi con soglia $V_{\text{th}} = 1$. I due neuroni ricevono tre ingressi da 0,6 (in basso), consecutivi in (a) e distanziati in (b). I marcatori indicano il potenziale H_k appena caricato, i tratti verticali il soft reset che segue uno spike, la fila in alto gli spike emessi. Il neurone IF, che non dimentica, emette uno spike in entrambi i casi. Il LIF lo emette soltanto quando gli ingressi sono abbastanza ravvicinati da sommarsi prima che il potenziale decada.

verso il riposo lungo un'esponenziale. Trascorso un tempo pari a τ_m la distanza dal riposo si è ridotta a circa il 37%, poiché $e^{-1} \approx 0{,}37$, e la costante τ_m misura dunque, in unità di tempo, per quanto il neurone ricorda un ingresso.

Per simulare questa dinamica a passi di durata Δt occorre discretizzarla. Molte implementazioni, tra cui la libreria SpikingJelly [8], adottano la discretizzazione di Eulero, che sostituisce la derivata con un rapporto incrementale e ritrova proprio la forma dell'equazione (1.18),

$$H_k = V_{k-1} + \frac{X_k - V_{k-1}}{\tau_d}, \qquad \lambda = 1 - \frac{1}{\tau_d}, \quad \eta = \frac{1}{\tau_d}. \tag{1.20}$$

Qui la costante τ_d, che deve superare 1 perché λ resti positivo, non è un tempo ma un numero puro, espresso in passi. Letta come discretizzazione dell'equazione continua vale $\tau_d = \tau_m/\Delta t$, ma Eulero approssima soltanto la soluzione esatta, per la quale varrebbe $\lambda = e^{-\Delta t/\tau_m}$. Con una costante pari a due passi si ottiene $\lambda = 0{,}5$ nel primo caso e circa 0,61 nel secondo. Quando un lavoro riporta una costante

di tempo, quindi, bisogna sapere se si tratti di un tempo fisico o di un numero di passi, e con quale discretizzazione venga usata.

Srotolando la ricorrenza, cioè sostituendo a ritroso ogni potenziale con la sua espressione, si vede che cosa il neurone conservi della propria storia. In assenza di spike e reset, con coefficienti costanti, si ottiene

$$V_k = \lambda^k V_0 + \eta \sum_{j=1}^{k} \lambda^{k-j} X_j. \tag{1.21}$$

Il potenziale è la somma di tutti gli ingressi ricevuti, ciascuno moltiplicato per λ tante volte quanti sono i passi trascorsi dal suo arrivo. Con $\lambda = 0{,}5$, rispetto all'ingresso appena arrivato, quello del passo precedente pesa la metà, quello di tre passi prima un ottavo e quello di dieci passi prima meno di un millesimo. Gli ingressi passati, però, non vi compaiono come osservazioni separate. Sono fusi in un unico numero, e su questa proprietà si tornerà alla fine della sezione.

La stessa espressione dice come tradurre λ in un tempo fisico. Un ingresso arrivato n passi prima, cioè $n\Delta t$ secondi prima, pesa λ^n. Cercando la costante τ_{eq} per cui $\lambda^n = \exp(-n\Delta t/\tau_{\text{eq}})$, come nel rilassamento continuo, si trova

$$\tau_{\text{eq}} = -\frac{\Delta t}{\ln \lambda}. \tag{1.22}$$

Con $\lambda = 0{,}5$ vale circa $1{,}44\,\Delta t$, quindi poco più di 14 ms con bin da 10 ms e poco più di 140 ms con bin da 100 ms.

Il punto essenziale è che τ_{eq} dipende dalla durata del bin, cioè dal clock dell'input. Lo stesso neurone, con lo stesso λ, dimentica due volte più in fretta in tempo reale se la durata dei bin si dimezza. Adeguare λ al clock conserverebbe la memoria fisica, ma non renderebbe equivalenti i due modelli, perché cambiando i bin cambiano anche i conteggi che ciascuno contiene e il numero di occasioni in cui il neurone può emettere spike. Con bin di durata variabile, come nelle partizioni a eventi costanti, un coefficiente fisso per passo non corrisponde infine ad alcun decadimento uniforme nel tempo fisico, perché un passo lungo mezzo secondo e uno lungo cinque millisecondi fanno perdere al potenziale la stessa frazione.

Queste relazioni descrivono un singolo neurone isolato, al di sotto della soglia. La memoria di una rete, che combina molti neuroni, trasformazioni apprese e interazioni tra strati, non ha un orizzonte massimo così rigidamente definito.

1.2.3 Addestramento con surrogate gradient

Le reti neurali si addestrano con la retropropagazione dell'errore, che calcola per ogni peso di quanto cambierebbe la loss modificandolo di poco, e lo corregge di conseguenza. La soglia che genera gli spike rende questo calcolo inutilizzabile. La

funzione Θ è piatta ovunque tranne che nell'origine, dove salta da 0 a 1. Se il potenziale cambia di poco lo spike resta quello che era, e la derivata è quindi nulla quasi ovunque, mentre nell'unico punto in cui lo spike cambia la derivata non è definita. In entrambi i casi nessun segnale di errore riesce ad attraversare la soglia e a raggiungere i pesi degli strati precedenti.

I *surrogate gradient* aggirano l'ostacolo con un compromesso [5]. Nel passaggio in avanti (forward pass), quello che produce le uscite della rete, lo spike resta esattamente quello dell'equazione (1.16). Nel passaggio all'indietro (backward pass), quello che calcola i gradienti, la sua derivata viene sostituita da una funzione regolare $g_\alpha(H_k - V_{\text{th}})$. Una scelta diffusa è la derivata di una sigmoide. Con $\sigma(u) = (1 + e^{-u})^{-1}$ si pone

$$g_\alpha(z) = \alpha\,\sigma(\alpha z)\Big[1 - \sigma(\alpha z)\Big], \qquad \alpha > 0, \tag{1.23}$$

che è proprio la derivata di $\sigma(\alpha z)$. In pratica la rete calcola gli spike con il gradino ma, al momento di calcolare i gradienti, si comporta come se il gradino fosse stato una sigmoide (figura 1.5). Il parametro α regola la ripidità della sigmoide, e quindi l'altezza e la larghezza del gradiente surrogato, che raggiunge il massimo sulla soglia e si annulla rapidamente allontanandosene. Ricevono così un gradiente apprezzabile soltanto i neuroni il cui potenziale era vicino alla soglia, quelli cioè per cui una piccola modifica dei pesi avrebbe potuto cambiare la decisione di emettere o meno lo spike.

La soglia non è l'unico ostacolo. Poiché il potenziale collega ogni passo al successivo, la loss dipende dai pesi anche attraverso il tempo. La Backpropagation Through Time (BPTT) affronta il problema srotolando la ricorrenza [9]. La rete viene vista come una catena di T copie di sé stessa, una per ciascun passo, che condividono gli stessi pesi e sono collegate dagli stati che ciascuna passa alla successiva. Su questa catena, che non contiene più cicli, la retropropagazione ordinaria si applica senza difficoltà, e l'errore si propaga in due direzioni, attraverso gli strati e all'indietro nel tempo. Passando da un passo al precedente attraverso la membrana, il gradiente viene moltiplicato all'incirca per λ, lo stesso fattore con cui il neurone dimentica. Un neurone che dimentica in fretta impara quindi poco dalle conseguenze lontane dei propri stati, una forma della scomparsa del gradiente già nota nelle reti ricorrenti [10].

Il prezzo della BPTT è la memoria. Per percorrere la catena all'indietro servono i valori calcolati in avanti in ogni passo, e il solo stato neuronale da conservare cresce come $\mathcal{O}(T \sum_l N_l)$, dove N_l è il numero di unità dello strato l [5]. In inferenza, invece, a una rete di soli neuroni LIF basta il potenziale attuale di ciascun neurone, cioè $\sum_l N_l$ valori qualunque sia T. Le due grandezze vanno tenute distinte, perché un modello può richiedere pochissimo stato durante l'esecuzione e, al tempo stesso, un grafo molto esteso durante l'ottimizzazione.

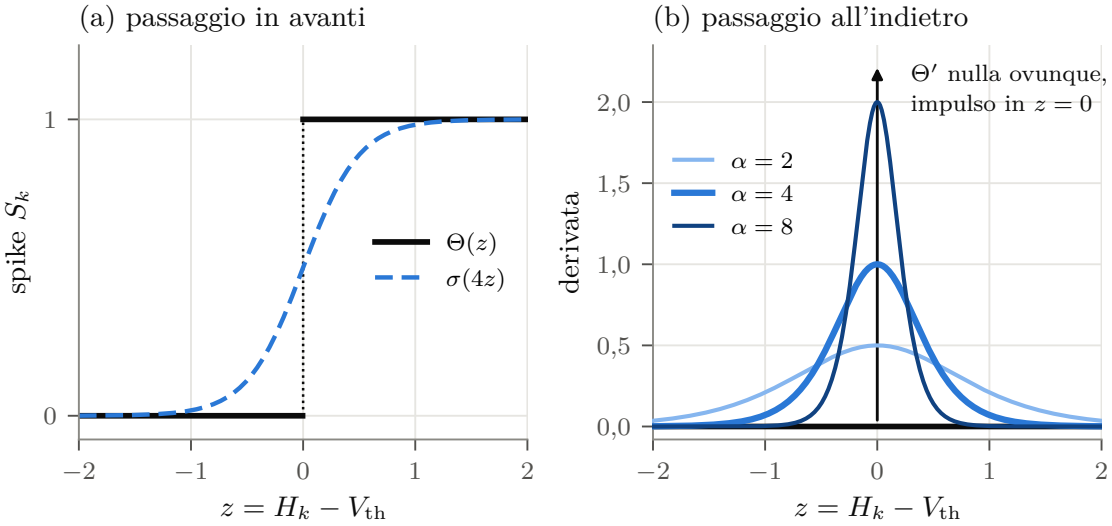


Figura 1.5: Principio del surrogate gradient, in funzione della distanza $z = H_k - V_{\text{th}}$ tra potenziale e soglia. (a) Nel passaggio in avanti lo spike è prodotto dal gradino Θ (linea nera). La sigmoide $\sigma(4z)$ (tratteggio) non viene mai calcolata, ma è la funzione di cui il passaggio all'indietro usa la derivata. (b) La derivata del gradino è nulla ovunque tranne nell'origine, dove non è definita (freccia), e non trasmette alcun gradiente. La surrogata g_α la sostituisce con una campana centrata sulla soglia, tanto più alta e stretta quanto maggiore è α. Il valore evidenziato, $\alpha = 4$, è quello adottato in questo lavoro.

In questo lavoro la rete spiking viene addestrata direttamente, con surrogate gradient e BPTT. È un paradigma diverso dalla conversione ANN-SNN, in cui una rete convenzionale viene prima addestrata e poi tradotta in una rete spiking che ne approssima le attivazioni con la frequenza degli spike. La dinamica discreta, la funzione surrogata e il trattamento del reset effettivamente adottati sono specificati nella sezione 2.3.

1.2.4 Memoria neuronale e relazioni tra feature

Si torni all'equazione (1.21). Per un neurone LIF privo di ulteriori stati interni, tutto ciò che la storia degli ingressi può trasmettere al futuro passa per un unico numero, il potenziale V_k. L'intera sequenza viene compressa in un solo valore, e storie diverse possono lasciare lo stesso potenziale. Con $\lambda = 0{,}5$ ed $\eta = 1$, partendo da un potenziale nullo, un ingresso pari a 0,4 arrivato al passo precedente e un ingresso pari a 0,2 arrivato adesso lasciano entrambi $V_k = 0{,}2$. Da quel momento, se gli ingressi successivi coincidono, il neurone si comporta in modo identico nei due casi, e non ha più modo di sapere se abbia visto qualcosa di intenso poco fa

o qualcosa di debole in questo istante. La rete nel suo complesso può comunque conservare la differenza in altre unità o in altri moduli, sicché la compressione operata da un singolo potenziale non implica di per sé una perdita equivalente a livello dell'intera rete.

Si delineano così due capacità diverse. La prima è la *persistenza*, cioè la capacità di conservare a lungo una traccia del passato. La seconda è l'*accesso*, cioè la possibilità di recuperare separatamente ciò che è accaduto in un istante preciso per confrontarlo con il presente. Si potrebbe pensare di migliorare la memoria di un neurone avvicinando λ a 1, ma la modifica agisce soltanto sulla prima. I contributi passati sopravvivono più a lungo, e proprio per questo un numero maggiore di ingressi si mescola nella stessa somma, che diventa ancora più difficile da scomporre. Si pensi alla barra della sezione 1.1.1, la cui direzione è scritta nell'ordine con cui si attivano pixel vicini. Per riconoscerla non basta sapere che entrambi i pixel sono stati attivi di recente, informazione che un potenziale conserva bene. Bisogna sapere quale dei due si è attivato prima, cioè mettere a confronto due istanti distinti, e un'unica somma pesata può farlo soltanto in modo indiretto.

La distinzione tra persistenza e accesso motiva l'interesse per operatori capaci di combinare o confrontare esplicitamente feature diverse. La sezione successiva descrive come l'attenzione sia stata integrata nelle architetture spiking, e sarà allora necessario distinguere le relazioni costruite all'interno di uno stesso passo da quelle che collegano istanti diversi.

1.3 Dalle SNN agli Spiking Transformer

La dinamica neuronale offre alle SNN un modo naturale di trasportare informazione da un passo temporale, o *timestep*, al successivo. Resta però aperto un problema di altra natura, quello di mettere in relazione le numerose feature che compongono una rappresentazione visiva. Un neurone LIF integra nel tempo il proprio ingresso, ma nulla nella sua dinamica gli permette di stabilire quali regioni della scena siano rilevanti l'una per l'altra, né di collegare feature spazialmente lontane. Questa sezione ripercorre il modo in cui l'attenzione dei Transformer, nata per affrontare questo problema nelle reti convenzionali, è stata adattata al dominio spiking, fino all'architettura da cui muove la tesi.

1.3.1 Self-attention e relazioni tra token

Le architetture convoluzionali affrontano il problema componendo operatori locali. Ogni convoluzione combina l'informazione di un intorno limitato, e soltanto impilando molti strati, che allargano progressivamente il campo recettivo, la rete arriva a costruire relazioni spaziali estese. Su questo principio si fonda l'efficacia delle

CNN nella visione artificiale, ed è su di esso che sono state costruite molte delle prime SNN profonde.

Il Transformer adotta una prospettiva diversa. Anziché affidare la costruzione delle dipendenze a una lunga catena di operatori locali, permette agli elementi di una rappresentazione di interagire direttamente attraverso il meccanismo di self-attention [11]. Trasferita alla visione, l'idea consiste nel suddividere l'immagine in porzioni, le patch, ciascuna delle quali viene proiettata in un vettore di feature detto token [12]. I token possono poi influenzarsi a vicenda in base al contenuto che rappresentano, qualunque sia la distanza che li separa nell'immagine.

Il meccanismo si capisce bene a partire da un dizionario, la struttura dati che associa a ogni chiave un valore. Per recuperare un valore si presenta una richiesta (*query*), la si confronta con le chiavi (*key*) e si ottiene il valore (*value*) associato alla chiave identica alla richiesta. Un dizionario di traduzioni, interrogato con la parola «gatto», trova la chiave «gatto» e restituisce «cat». L'attenzione rende graduale questa ricerca (figura 1.6). Richiesta e chiavi diventano vettori di feature, e il confronto non dà più un esito sì o no ma un punteggio di somiglianza con ciascuna chiave. Il risultato, di conseguenza, non è un singolo valore ma una media di tutti i valori, pesata con quei punteggi. Con pesi pari a 0,7, 0,2 e 0,1 si ottiene $0{,}7\,v_1 + 0{,}2\,v_2 + 0{,}1\,v_3$, dove conta soprattutto il valore associato alla chiave più affine alla richiesta.

Nella self-attention richiesta, chiavi e valori provengono tutti dagli stessi token, ed è da qui che viene il prefisso *self*. Data una matrice di token $X \in \mathbb{R}^{N\times D}$, con N token di dimensione D, tre trasformazioni lineari apprese ricavano da ciascun token tre vettori,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \tag{1.24}$$

la Query, la Key e il Value. Ogni token ricopre quindi tre ruoli. Con la Query esprime che cosa cerca negli altri token, con la Key descrive che cosa offre a chi lo interroga, con il Value fornisce il contenuto che gli altri riceveranno se lo trovano rilevante. Tenere separati i tre vettori permette alla rete di imparare che ciò che un token cerca può essere diverso da ciò che offre. Il token che copre un tratto di contorno, per esempio, può imparare a cercare i token che ne proseguono il profilo, anche se si trovano dall'altra parte dell'immagine.

Le formule traducono direttamente questa ricerca. L'elemento (i, j) del prodotto $QK^\top$ è il prodotto scalare tra la Query del token i e la Key del token j, cioè il punteggio di somiglianza tra ciò che i cerca e ciò che j offre. La softmax trasforma ogni riga di punteggi in pesi positivi che sommano a 1, e questi pesi combinano i Value,

$$Y = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \tag{1.25}$$

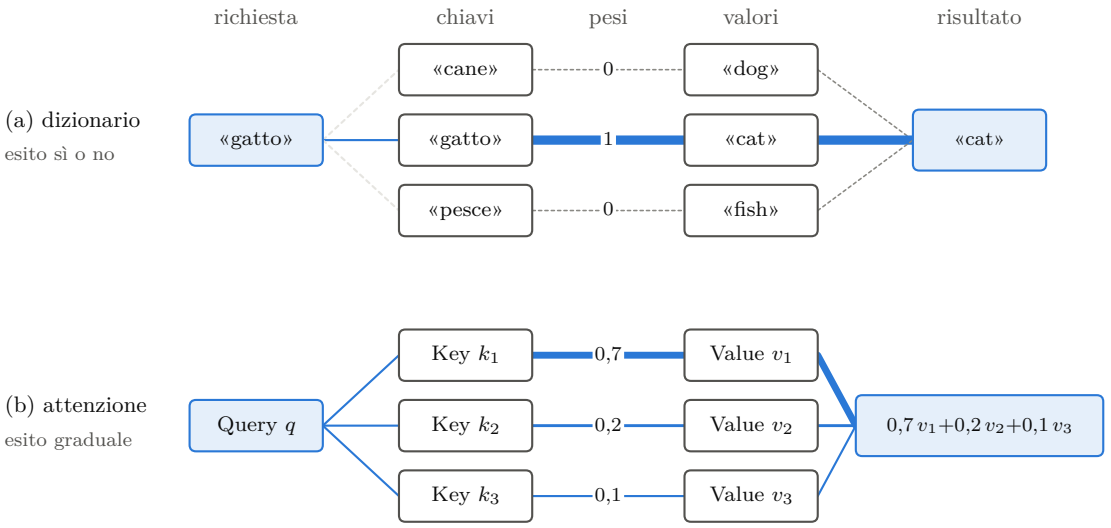


Figura 1.6: L'attenzione come ricerca graduale in un dizionario. (a) Un dizionario confronta la richiesta con le chiavi e restituisce soltanto il valore associato alla chiave identica. (b) L'attenzione confronta la Query con tutte le Key, ricava da ciascun confronto un peso e restituisce la media dei Value pesata in questo modo. Lo spessore dei collegamenti è proporzionale al peso, e i valori numerici sono illustrativi.

dove la divisione per $\sqrt{d}$, con d dimensione di Query e Key, evita che i prodotti scalari crescano troppo con la dimensione. La riga i di Y è il nuovo token i, cioè la media dei Value di tutti i token pesata secondo la loro compatibilità con la sua Query. Nel seguito questa forma viene indicata come VSA (*Vanilla Self-Attention*). Ne deriva una capacità che la convoluzione non possiede nella stessa forma. Due regioni lontane possono interagire direttamente all'interno di un singolo blocco, senza che l'informazione debba attraversare una lunga successione di operatori locali.

Questa libertà ha un prezzo. Poiché $QK^\top$ confronta ogni token con tutti gli altri, la matrice di attenzione ha dimensione $N \times N$, il costo del prodotto cresce come $\mathcal{O}(N^2 D)$ e la memoria necessaria a conservare la matrice come $\mathcal{O}(N^2)$. Con 256 token la matrice contiene già più di 65 000 elementi. In una SNN se ne aggiunge un secondo vincolo, perché le stesse operazioni vanno ripetute a ogni timestep. Per un ingresso $X \in \mathbb{R}^{T \times N \times D}$ l'attenzione viene tipicamente applicata, istante per istante, alle feature di ciascun passo, e la dimensione temporale, che finora era apparsa come la sorgente della memoria neuronale, diventa così anche un moltiplicatore del costo.

1.3.2 La self-attention in forma spiking

L'incontro tra SNN e Transformer nasce da questa tensione. La VSA si basa su attivazioni continue, prodotti tra matrici dense e normalizzazioni, operazioni che non traggono alcun vantaggio dalla natura binaria e sparsa degli spike. Sostituire le attivazioni di un Transformer con neuroni LIF non basta dunque a ottenere uno Spiking Transformer. Occorre ripensare almeno in parte il meccanismo di attenzione, altrimenti la rappresentazione spiking verrebbe subito riconvertita in una computazione del tutto analoga a quella di una rete convenzionale (ANN).

Per capire che cosa si voglia preservare conviene anticipare un'idea che la sezione 1.6 riprenderà. Per applicare un peso a un'attivazione continua serve una moltiplicazione seguita da un accumulo (MAC). Se l'attivazione è uno spike binario la moltiplicazione scompare, perché basta sommare il peso quando lo spike c'è e non fare nulla quando manca, con un semplice accumulo (AC). Una rete si dice *spike-driven* quando funziona così, cioè quando ogni operazione è innescata da uno spike e l'assenza di uno spike coincide con l'assenza di calcolo [13].

Con Spikformer, Zhou et al. propongono nel 2023 la Spiking Self-Attention (SSA), mostrando che Query, Key e Value possono essere espressi in forma spiking e che la softmax può essere eliminata [14]. Dalle feature spiking si ricavano tre rappresentazioni binarie,

$$Q_s,\ K_s,\ V_s \in \{0,1\}^{N\times D}, \tag{1.26}$$

la cui interazione continua a svolgere il ruolo dell'attenzione. In forma semplificata,

$$\mathrm{SSA}(Q_s, K_s, V_s) = \mathrm{SN}\left(Q_s K_s^{\top} V_s\, \beta\right), \tag{1.27}$$

dove $\mathrm{SN}(\cdot)$ indica lo strato di neuroni spiking che riporta il risultato in forma binaria e β è un fattore di scala. La rinuncia alla softmax ha un peso particolare, per due ragioni. La softmax richiede esponenziali, divisioni e una normalizzazione che coinvolge tutti i coefficienti di una riga, operazioni estranee a una computazione guidata dagli eventi. Nel dominio spiking, inoltre, viene meno il suo compito principale, quello di rendere positivi i pesi, perché il prodotto di due matrici binarie non può produrre valori negativi [14].

Spikformer dimostra così che self-attention e SNN sono compatibili, ma porta alla luce due questioni. La prima riguarda la natura effettiva delle operazioni. Che Query, Key e Value siano spike non garantisce che l'intero blocco esegua soltanto operazioni economiche. L'elemento (i, j) di $Q_s K_s^{\top}$ conta in quanti canali la Query del token i e la Key del token j sono attive insieme. Se la prima vale [1,0,1,1] e la seconda [1,1,1,0] il risultato è 2, perché coincidono nel primo e nel terzo canale. Calcolare questo numero è economico, perché basta incrementare un contatore quando entrambi i bit valgono 1. Il risultato, però, non è più uno spike ma un intero, che può arrivare al numero dei canali, e lo strato SN lo riporta in forma

binaria soltanto dopo che è stato combinato con i Value e scalato. Un problema analogo si ripresenta nelle connessioni residue, dove la somma di due spike può già valere 2. Quando un valore intero raggiunge uno strato dotato di pesi, ciascun peso va moltiplicato per quel valore, e ricompaiono le moltiplicazioni che il paradigma spike-driven vorrebbe evitare.

La seconda questione riguarda la scalabilità. Il prodotto $Q_s K_s^\top$ conserva una matrice tra tutte le coppie di token, e anche con dati sparsi la dimensione logica dell'operazione resta legata a N^2, un costo che diventa critico al crescere del numero dei token. Da questi due problemi sono nate linee di ricerca diverse e complementari, che le prossime sottosezioni esaminano nell'ordine.

1.3.3 Attenzione e residui spike-driven

Spike-driven Transformer affronta la prima questione, chiedendosi quanta parte della rete sia davvero spike-driven [13]. Gli autori osservano che diverse architetture precedenti mescolano accumuli e moltiplicazioni, dando luogo a una computazione ibrida, e si pongono l'obiettivo di mantenere binaria la comunicazione tra i blocchi. Il loro meccanismo, la Spike-Driven Self-Attention (SDSA), abbandona il prodotto completo $Q_s K_s^\top$. In una delle sue formulazioni, Query e Key vengono moltiplicate elemento per elemento e sommate lungo la dimensione dei token, e il risultato attraversa uno strato spiking,

$$A_s = \mathrm{SN}\left(\mathrm{SUM}_N\left(Q_s \odot K_s\right)\right), \tag{1.28}$$

dopodiché il vettore binario così ottenuto viene applicato ai Value,

$$Y_s = A_s \odot V_s, \tag{1.29}$$

dove $\odot$ indica il prodotto elemento per elemento. Con operandi binari questo prodotto equivale a una maschera, perché ogni elemento viene semplicemente conservato oppure azzerato. Il procedimento si legge in tre tempi. Il prodotto $Q_s \odot K_s$ segna dove Query e Key sono attive insieme, la somma conta per ciascun canale quante volte ciò accade tra tutti i token, e il neurone trasforma questi conteggi in una decisione binaria, canale per canale. I canali promossi attraversano il blocco, gli altri vengono azzerati.

Rispetto all'equazione (1.25) il cambiamento è sostanziale, come mostra la figura 1.7. Non viene costruita alcuna matrice $N \times N$ di relazioni tra coppie di token, e l'attenzione si riduce a una scelta su quali canali lasciar passare. Per questo SDSA può essere interpretata come una forma particolare di attenzione lineare, la cui complessità cresce linearmente con il numero di token e di canali anziché quadraticamente con il numero di token [13]. Si rinuncia così a parte della ricchezza delle correlazioni tra coppie, in cambio di un'operazione molto più adatta alla computazione sparsa.

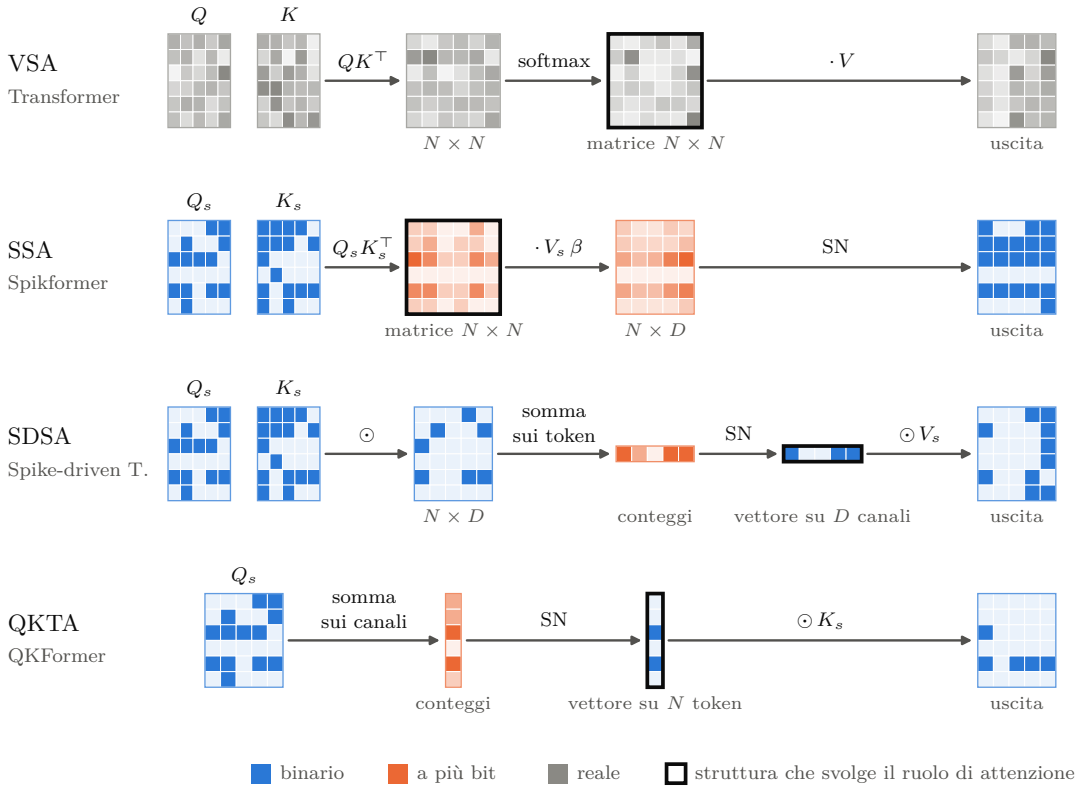


Figura 1.7: Riformulazione della self-attention nel dominio spiking, calcolata su un piccolo esempio casuale con 6 token (righe) e 5 canali (colonne). Il colore indica il tipo di valore, e la cornice spessa la struttura che svolge il ruolo di attenzione. Dalla VSA alla SSA scompare la softmax e i dati in ingresso diventano binari, ma la matrice tra coppie di token resta e contiene conteggi. SDSA e QKTA sostituiscono la matrice con un vettore binario, che in uscita azzera interi canali nel primo caso e interi token nel secondo. La soglia degli strati SN è scelta per rendere leggibile l'esempio.

Spike-driven Transformer interviene anche su un elemento meno appariscente, ma importante per comprendere gli sviluppi successivi, cioè la posizione delle connessioni residue. Una connessione residua somma l'ingresso di un blocco alla trasformazione che il blocco stesso produce, e nelle reti profonde facilita la propagazione dell'informazione e dei gradienti. In una SNN, però, conta dove la somma viene eseguita, se tra spike oppure tra potenziali di membrana (figura 1.8).

Nella connessione residua Spike-Element-Wise (SEW) adottata da Spikformer si sommano direttamente due tensori di spike. Il risultato può valere 0, 1 o 2, e l'intervallo si allarga a ogni blocco attraversato, perché la somma di un blocco

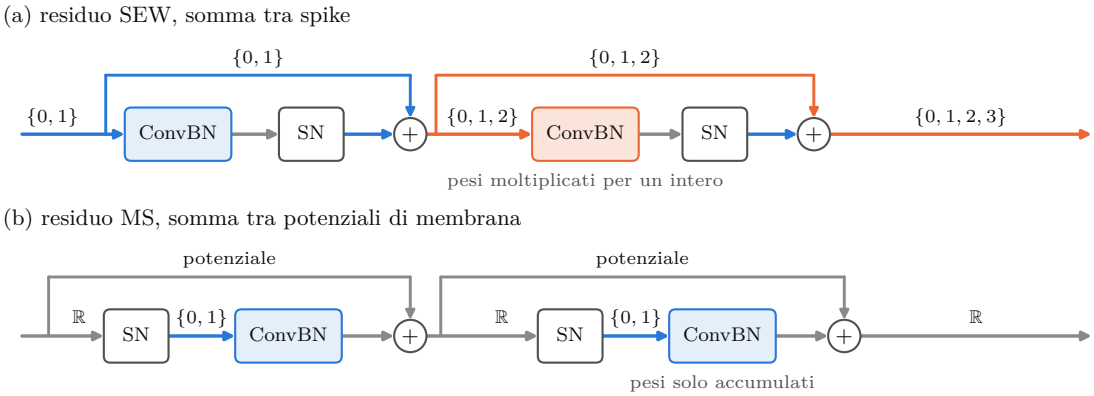


Figura 1.8: Due posizioni della connessione residua, indicate con i valori che ogni tratto può assumere [15]. (a) Nel residuo SEW la somma avviene tra spike. I valori crescono a ogni blocco, e lo strato pesato successivo deve moltiplicare ciascun peso per un intero. (b) Nel residuo MS la somma avviene tra potenziali di membrana, cioè tra valori reali, e un neurone spiking precede ogni strato pesato, che riceve così soltanto ingressi binari.

diventa un addendo di quella del successivo. Il segnale smette così di essere binario. Quando raggiunge uno strato pesato non basta più selezionare i pesi corrispondenti agli spike attivi, perché ciascun peso va moltiplicato per un valore intero, e ricompaiono moltiplicazioni tra interi e numeri in virgola mobile.

Spike-driven Transformer sposta invece la somma residua sui potenziali di membrana, e colloca un neurone spiking subito prima di ogni strato pesato. La somma avviene così tra valori reali, come quella che ogni neurone esegue già al proprio interno, e ciò che entra negli strati pesati resta binario [13]. Il dettaglio contiene una lezione progettuale di portata più generale. Contare i neuroni spiking presenti in un'architettura dice poco sulla sua natura event-driven, che dipende piuttosto dalla posizione di quei neuroni rispetto alle operazioni lineari e ai percorsi residui.

Spikingformer approfondisce lo stesso punto in una linea di ricerca parallela [15]. Gli autori analizzano le connessioni residue di Spikformer e mostrano che la variante SEW genera valori a più bit lungo i collegamenti diretti, mentre una Membrane Shortcut (MS) preserva ingressi binari per gli operatori successivi. Combinando questa soluzione con la SSA, il modello mira a conservare insieme comunicazione spike-driven e capacità di modellazione globale.

Il confronto mette in luce un compromesso che accompagna l'intera evoluzione degli Spiking Transformer. Semplificare l'attenzione in modo aggressivo ne riduce il costo, ma impoverisce le relazioni che il modello può rappresentare. Conservare una self-attention più globale offre maggiore espressività, ma obbliga a intervenire

altrove per preservare l'efficienza spike-driven. La storia di queste architetture non va letta, allora, come una successione di modelli via via migliori. È piuttosto l'esplorazione di uno spazio definito da almeno tre assi, la capacità relazionale, la scalabilità e l'aderenza al paradigma spike-driven, lungo i quali ciascun lavoro sceglie la propria posizione.

1.3.4 La struttura gerarchica

Resta aperta una questione di altro genere, che riguarda il modo in cui scalare l'architettura stessa. Un Vision Transformer tradizionale mantiene spesso la stessa risoluzione lungo gran parte dell'encoder, e molte delle prime architetture Spiking Transformer seguono la stessa strategia. Spikformer, per esempio, usa uno Spiking Patch Splitting (SPS) per convertire l'ingresso in N token di dimensione D, elaborati poi da una sequenza di blocchi che ne conservano il numero e la dimensione (figura 1.9a).

La soluzione è semplice, ma si discosta dalla struttura gerarchica che caratterizza molte CNN e numerosi Vision Transformer moderni. In una rete gerarchica le prime fasi operano su molte posizioni con pochi canali, mentre quelle profonde lavorano su pochi token descritti da feature più ricche. Questa progressione presenta due vantaggi. Preserva il dettaglio spaziale nelle prime fasi mentre costruisce rappresentazioni via via più astratte, e riduce il numero dei token proprio dove la dimensione delle feature, e con essa il costo dei blocchi Transformer, tende a crescere.

Da questa esigenza nasce QKFormer [16]. Gli autori individuano due limiti congiunti delle architetture precedenti, la complessità quadratica della SSA rispetto al numero dei token e l'elevato costo di memoria che deriva dal combinare la dimensione spaziale con la simulazione temporale. Per superarli introducono una struttura gerarchica a tre stadi, collegati da moduli di Spiking Patch Embedding with Deformed Shortcut (SPEDS), e un nuovo meccanismo di attenzione, la Q-K Attention (figura 1.9b). Oltre a chiedersi come rendere più economico il prodotto tra Q, K e V, QKFormer si domanda se tutte e tre le componenti siano davvero necessarie per selezionare l'informazione. La Q-K Attention ne usa soltanto due, Query e Key, e riduce l'attenzione a un vettore binario che decide quali token, o quali canali, lasciar passare, come mostra l'ultima riga della figura 1.7. La sezione 1.4 ne descrive il funzionamento in dettaglio.

La tabella 1.2 riassume il percorso. QKFormer ne è il punto di partenza naturale per questa tesi, perché riunisce tre proprietà preziose nel regime compatto studiato più avanti, un'attenzione di costo lineare, una rappresentazione gerarchica e una trasmissione residua che attraversa i cambi di risoluzione.

Altrettanto importante è osservare quale problema QKFormer non si proponga di risolvere. Le operazioni di attenzione descritte finora costruiscono relazioni tra

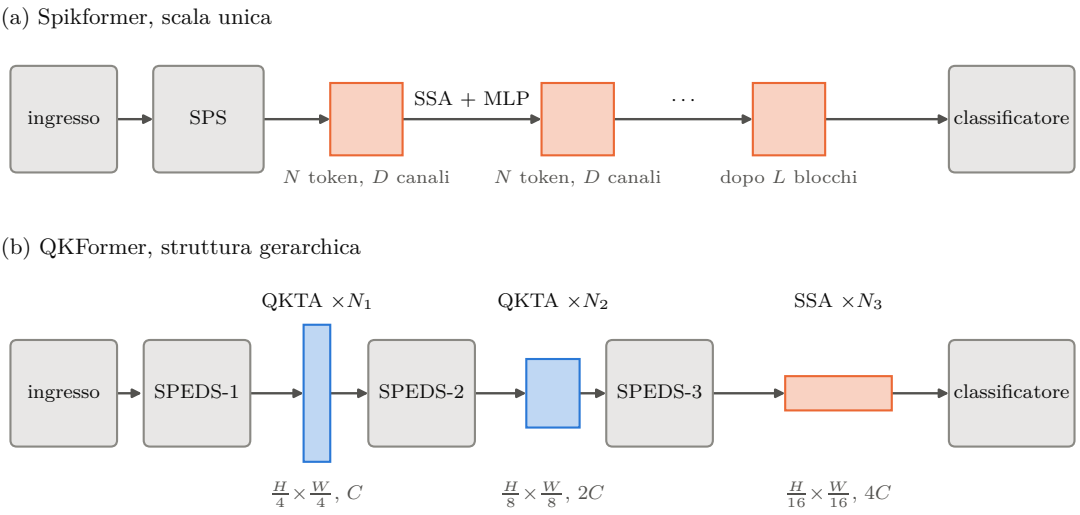


Figura 1.9: Struttura a scala unica e struttura gerarchica, secondo le descrizioni originali. I rettangoli rappresentano le feature, con l'altezza proporzionale al lato della griglia dei token e la larghezza al numero dei canali, mentre il colore distingue i blocchi con SSA da quelli con Q-K Token Attention (QKTA). (a) In Spikformer tutti i blocchi operano sugli stessi N token. (b) In QKFormer ogni modulo SPEDS riduce la griglia e aumenta i canali. La dimensione temporale T resta invariata lungo entrambe le reti.

token o canali all'interno delle feature di un singolo timestep. Data una sequenza $X_1, X_2, \ldots, X_T$, l'attenzione applicata all'istante t riceve tipicamente la sola X_t, e il collegamento con X_{t-1} resta affidato principalmente agli stati dei neuroni che attraversano il tempo. L'espressione *self-attention* suggerisce, in modo intuitivo ma impreciso, un accesso globale all'intera sequenza. La globalità va invece sempre riferita alla dimensione lungo cui l'operatore agisce. Uno Spiking Transformer può disporre di un'attenzione globale sui token e, insieme, di una comunicazione temporale prevalentemente ricorrente e locale.

Resta da capire se la dinamica neuronale descritta nella sezione 1.2 basti a governare anche l'interazione tra X_t e X_{t-k}, ed è da questa domanda che muove la sezione 1.5. Prima conviene però esaminare in dettaglio il punto di arrivo dell'evoluzione descritta, perché QKFormer determina direttamente la baseline su cui si fonda la parte sperimentale della tesi.

Tabella 1.2: Evoluzione dei meccanismi di attenzione spiking rilevanti per il lavoro.

Architettura	**Problema affrontato**	**Operatore**	**Ambito**	**Costo**	**Spike-driven**
Transformer / ViT	Relazioni globali tra token	VSA con softmax	Coppie di token	Quadratica in N	No
Spikformer	Rendere spiking la self-attention	SSA con Q, K, V spiking	Coppie di token	Quadratica in N	Parziale
Spike-driven Transformer	Eliminare le operazioni non spike-driven e ridurre il costo	SDSA, maschere e addizioni sparse	Selezione di canali	Lineare	Sì
Spikingformer	Conservare l'attenzione globale con residui spike-driven	SSA con residui MS	Coppie di token	Quadratica in N	Sì
QKFormer	Scalabilità e struttura gerarchica	QKTA o QKCA con SPEDS	Selezione di token o canali	Lineare nella dimensione	Nell'attenzione

Nota. «Spike-driven» indica qui la proprietà architetturale dichiarata dai rispettivi lavori. Il costo effettivo dipende comunque da firing rate, operatori periferici, rappresentazione degli input e piattaforma hardware, e verrà discusso nella sezione 1.6.

1.4 QKFormer come riferimento architetturale

La sezione 1.3.4 ha presentato QKFormer come risposta ai limiti di scalabilità delle architetture precedenti [16]. Qui se ne riprende ciò che serve al resto del lavoro, perché da questa architettura discende la baseline compatta del capitolo 2. Contano in particolare tre aspetti, la forma esatta del blocco di attenzione, il peso del patch embedding nella gerarchia e il modo in cui il modello tratta il tempo.

Il blocco con Q-K Token Attention La Q-K Attention ricava dal proprio ingresso X due sole proiezioni spiking,

$$Q = \mathrm{SN}_Q\left(\mathrm{BN}(XW_Q)\right), \qquad K = \mathrm{SN}_K\left(\mathrm{BN}(XW_K)\right), \tag{1.30}$$

dove BN indica la batch normalization e $Q, K \in \{0,1\}^{T\times N\times D}$, con T timestep, N token e D canali. Nella Q-K Token Attention (QKTA), la variante adottata dalla baseline, le due proiezioni svolgono compiti distinti. La Query decide quali token lasciar passare, la Key ne trasporta il contenuto. Considerando un solo timestep, si sommano per ciascun token gli spike della Query lungo i canali, e il conteggio

attraversa un neurone che lo traduce in una decisione binaria,

$$A^{\text{tok}} = \text{SN}\left(\sum_{d=1}^{D} Q_{:,d}\right), \qquad A^{\text{tok}} \in \{0,1\}^{N\times 1}, \qquad X' = A^{\text{tok}} \odot K. \tag{1.31}$$

Un token la cui Query è abbastanza attiva conserva in uscita la propria Key, gli altri vengono azzerati su tutti i canali, come mostra l'ultima riga della figura 1.7. La variante Q-K Channel Attention applica lo stesso principio all'altro asse, sommando lungo i token per decidere quali canali conservare, ma non viene impiegata in questo lavoro.

Conviene chiarire anche che cosa questa operazione non faccia. Ogni componente di A^{tok} dipende soltanto dalla Query del token corrispondente, perché le proiezioni XW_Q e XW_K agiscono sui canali di ciascun token separatamente. Diversamente da VSA e SSA, la QKTA non confronta quindi token diversi, ma ne seleziona alcuni in base alla loro attività. Le relazioni tra posizioni diverse restano affidate alle convoluzioni dei patch embedding, su scala locale, e alla SSA dell'ultimo stadio, su scala globale. Il capitolo 3 riesamina questa scelta nel regime compatto.

Il guadagno è nel costo. L'attenzione si riduce a un vettore di N elementi, e gli autori riportano una complessità $\mathcal{O}(D)$ in tempo e $\mathcal{O}(N)$ in spazio, contro $\mathcal{O}(N^2D)$ e $\mathcal{O}(N^2 + ND)$ della SSA [16]. La somma degli spike ha inoltre una varianza molto più contenuta del prodotto $Q_s K_s^\top V_s$, e il fattore di scala β può essere eliminato. Questi valori, però, riguardano la sola operazione di attenzione. Le matrici Q e K vanno comunque calcolate e occupano $\mathcal{O}(ND)$, e attorno alla maschera restano proiezioni, normalizzazioni e neuroni. Un'attenzione di costo trascurabile non implica quindi un blocco di costo trascurabile.

Al mascheramento segue infatti una proiezione lineare, che ricombina le feature dei token selezionati,

$$X'' = \text{SN}\left(\text{BN}\left(\text{Linear}(X')\right)\right), \tag{1.32}$$

e il blocco completo conserva la struttura di un encoder Transformer,

$$X'_l = \text{QKA}(X_{l-1}) + X_{l-1}, \tag{1.33}$$

$$X_l = \text{SMLP}(X'_l) + X'_l, \tag{1.34}$$

dove QKA indica l'intero modulo di attenzione e SMLP uno spiking MLP [16]. Le somme residue avvengono a valle dei neuroni, secondo lo schema SEW della figura 1.8a. L'attenzione opera quindi con maschere e addizioni, ma ciò che circola tra i blocchi contiene piccoli interi, e gli strati pesati che li ricevono eseguono moltiplicazioni. Per questo i conteggi di questo lavoro distinguono MAC e AC in base all'operando e non al tipo di strato (sezione 2.6).

Gerarchia e patch embedding La struttura a tre stadi è quella della figura 1.9b. Poiché a ogni passaggio i token diminuiscono e i canali aumentano, un unico tipo

di attenzione non è ugualmente adatto a tutta la rete. Gli autori adottano perciò un'integrazione mista, con la QKTA nei primi stadi, dove i token sono molti, e la SSA nell'ultimo, dove sono pochi e il costo quadratico pesa meno. Le prove di ablazione sostengono la scelta. Su CIFAR10-DVS la combinazione di QKTA e SSA raggiunge l'84,0%, mentre la sola QKTA si ferma all'80,7% [16].[1] La baseline eredita questa sequenza, e il capitolo 3 verifica se la SSA resti necessaria anche con una capacità molto inferiore.

Un collegamento residuo ordinario presuppone che ingresso e uscita abbiano le stesse dimensioni, ma il patch embedding che separa due stadi cambia sia il numero dei token sia quello dei canali. Il modulo SPEDS risolve il problema inserendo una proiezione leggera W_d sul ramo residuo,

$$Y = \mathcal{F}(X, \{W_i\}) + \mathrm{SN}(W_d X), \tag{1.35}$$

dove $\mathcal{F}$ è il ramo principale, una sequenza di convoluzioni, normalizzazioni, neuroni e max-pooling, e W_d una convoluzione 1×1 con passo maggiore di uno, che adegua in un solo colpo risoluzione e canali [16]. Da qui il nome di *deformed shortcut*, perché il collegamento diretto non può più essere un'identità e va «deformato» per portare l'ingresso nello spazio dell'uscita. Inserito in Spikformer al posto del modulo originale, SPEDS porta l'accuratezza su CIFAR10-DVS dall'80,90% all'82,20% [16].

La gerarchia ha però un rovescio che conta per questa tesi. Rendendo economica l'attenzione e concentrando la riduzione di scala in moduli dedicati, sposta una parte del budget verso il patch embedding, che lavora sulle mappe a risoluzione più alta. Nella baseline compatta il primo patch embedding ospita oltre l'80% dello stato persistente, e i due patch embedding insieme circa due terzi dei parametri (sezione 2.3). Da questa osservazione parte l'analisi del capitolo 3.

Per i dati neuromorfici gli autori impiegano una configurazione ridotta, con 1,50 milioni di parametri, una larghezza massima di 256 canali e blocchi distribuiti sui tre stadi secondo lo schema {0,1,1}, nel quale il primo stadio si riduce al solo patch embedding. Con 16 timestep raggiunge il 98,6% su DVS128 Gesture e l'84,0% su CIFAR10-DVS [16]. La baseline compatta scende a circa un terzo di questi parametri (tabella 2.4), e in un regime simile la riduzione non può essere meccanica. Componenti che in QKFormer occupano una frazione modesta del budget possono diventare dominanti, e il peso di ciascuno va rivalutato quando larghezza e profondità diminuiscono.

Il ruolo del tempo Nelle equazioni precedenti Q e K hanno una dimensione T, e si potrebbe pensare che l'attenzione operi congiuntamente sui timestep. Non è così.

[1] Gli autori segnalano nel repository ufficiale un errore nell'ordine di reshape e LIF della proiezione SSA, con possibile sovrastima dei risultati ImageNet. I punteggi dell'articolo vanno quindi distinti da una riproduzione verificata dell'implementazione corretta [17].

Gli autori precisano che il tempo resta separato soltanto negli strati di neuroni, mentre negli altri operatori viene fuso con la dimensione del batch [16]. Per questi operatori ogni timestep è un campione a sé, e a ciascun istante t l'attenzione lavora sulla sola rappresentazione di quel passo,

$$A_t^{\text{tok}} = \mathcal{A}(Q_t). \tag{1.36}$$

Il passato può influenzare questa decisione soltanto attraverso lo stato dei neuroni, compreso quello che produce il vettore, e non compare alcun termine in cui Q_t venga messa a confronto con Q_{t-1} o con istanti precedenti. QKFormer possiede dunque una struttura fortemente gerarchica nello spazio, ma nessuna gerarchia corrispondente lungo il tempo. Ciò non significa che il modello ignori l'informazione temporale, ma che l'interazione temporale non è l'oggetto del suo meccanismo di attenzione.

Per questa tesi la distinzione è essenziale. QKFormer fornisce un backbone in cui l'elaborazione spaziale e di canale è già stata progettata con cura per essere gerarchica ed efficiente, e consente così di porre una domanda più specifica di quella affrontata dal lavoro originale.

Una volta portata l'architettura in un regime fortemente compatto, la sola memoria LIF è il modo più efficace di spendere il budget destinato alla dimensione temporale?

Il lavoro originale non offre elementi per rispondere, e la risposta va cercata sperimentalmente. La baseline compatta nasce da una scelta conservativa. Mantiene i principi di QKFormer con fedeltà sufficiente a costituire un riferimento riconoscibile, ma ne riduce drasticamente la scala. Soltanto dopo averne caratterizzato il comportamento diventa possibile decidere quali componenti preservare, quali semplificare e quali sostituire. La sezione successiva esamina intanto le strategie proposte in letteratura per rendere esplicito il legame tra istanti diversi.

1.5 Strategie per la modellazione temporale esplicita

La sezione 1.2.4 ha distinto la persistenza di una traccia del passato dall'accesso a ciò che è accaduto in un istante preciso, e la sezione 1.4 ha mostrato che in QKFormer il passato raggiunge il presente soltanto attraverso lo stato dei neuroni. Il limite non è esclusivo di questa architettura, e Shi et al. hanno cercato di quantificarlo con il Temporal Interaction Coefficient (TIC) [18]. Il coefficiente guarda a quanto l'uscita dell'attenzione al passo q sia sensibile agli ingressi dei passi precedenti, e misura con un'entropia quanto questa sensibilità sia distribuita

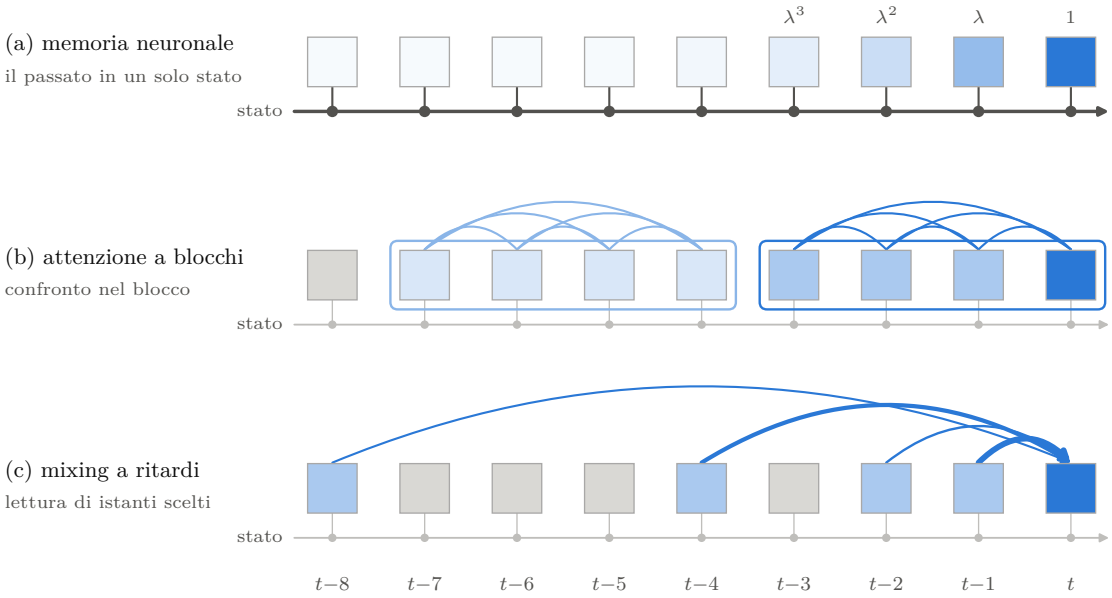


Figura 1.10: Tre modalità di accesso al passato, con il tempo in orizzontale e il passo corrente t a destra. In tutti i casi lo stato dei neuroni collega ogni passo al successivo. (a) Con la sola memoria neuronale il passato arriva al presente compresso in un unico stato, e il peso di ciascun istante dipende soltanto dalla distanza, come indica l'intensità del colore. (b) L'attenzione a blocchi confronta direttamente le feature degli istanti che appartengono allo stesso blocco, qui con $B = 4$. (c) Il mixing a ritardi legge alcuni istanti scelti, ciascuno con il proprio peso, qui con i ritardi di 1, 2, 4 e 8 passi del mixer usato in questo lavoro. Lo spessore dei collegamenti è illustrativo.

su più istanti. Su CIFAR10-DVS, per SSA, SDSA e QKTA, la dipendenza risulta fortemente concentrata sul passo corrente, mentre i passi lontani contribuiscono pochissimo. Il TIC descrive la forma della dipendenza e non la sua utilità, ma rende misurabile un'osservazione altrimenti qualitativa.

Le due strategie esaminate di seguito affiancano alla memoria dei neuroni un accesso esplicito al passato, e la figura 1.10 le mette a confronto. Entrambe contano il tempo in passi, quindi il significato fisico delle loro distanze dipende dal clock dell'input (sezione 1.1.2).

Attenzione spazio-temporale a blocchi STAtten estende la SSA a più timestep [19]. Anziché calcolare l'attenzione separatamente per ciascun istante, raggruppa B timestep consecutivi e dispone token e istanti lungo un unico asse,

come se ogni coppia formata da una posizione e da un istante fosse un token a sé. Con $T = 4$ e $B = 2$, per esempio, vengono elaborati insieme i timestep (0,1) e (2,3). L'informazione del primo elemento di un blocco non deve più sopravvivere nello stato neuronale fino al secondo, perché le feature di entrambi partecipano alla stessa operazione. Di conseguenza l'operazione non è causale all'interno del blocco, dato che ogni istante vede anche quelli che lo seguono.

Procedere a blocchi ha due motivazioni. Una correlazione sull'intera sequenza richiederebbe di conservare tutte le feature temporali, con un costo di memoria elevato. Inoltre il numero di neuroni attivi dopo il prodotto tra Query, Key e Value diminuisce con la distanza temporale, sicché correlare istanti molto lontani diventa poco produttivo [19]. La dimensione B definisce così un orizzonte di interazione esplicita, oltre il quale la relazione torna a essere affidata ai neuroni. Sui dati neuromorfici gli autori usano $B = 4$ con 16 timestep, e integrata in una configurazione di QKFormer a quattro blocchi STAtten guadagna un punto di accuratezza su CIFAR10-DVS. Il prezzo è che le feature di più timestep devono essere disponibili insieme, un costo che non compare nel conteggio dei parametri e che in un modello compatto pesa in modo particolare.

Mixing mediante ritardi Il Multi-Delay Mixer (MD-Mixer) segue una strada diversa. Non apre l'attenzione a più timestep, ma modifica le feature che la raggiungono [18]. Per ciascun canale i considera K ritardi $d_i^{(1)}, \ldots, d_i^{(K)}$ e altrettanti coefficienti, tutti appresi durante l'addestramento, e calcola

$$\widetilde{X}_{t,i} = \sum_{k=1}^{K} \alpha_i^{(k)} X_{t-d_i^{(k)},i}. \tag{1.37}$$

Ogni canale attinge così a più istanti della propria storia, e canali diversi possono imparare ritardi diversi. Il cambiamento rispetto al LIF è qualitativo. Nell'equazione (1.21) è la distanza temporale a fissare il peso di ogni contributo, attraverso il decadimento, mentre qui è la rete a stabilire quanta importanza attribuire a ciascun ritardo, e un contributo lontano raggiunge il presente senza dover sopravvivere a tutti gli aggiornamenti intermedi del potenziale.

Nell'architettura completa il mixer prende il posto della proiezione lineare di Key e Value, mentre la Query resta allineata all'istante corrente. L'attenzione continua a lavorare un passo alla volta, ma su feature che contengono già una parte del passato. Integrato in una configurazione di QKFormer a due blocchi, MD-Mixer guadagna poco più di un punto di accuratezza su CIFAR10-DVS, contro meno di mezzo punto di STAtten nello stesso confronto. Sembra contare soprattutto la possibilità di scegliere quale passato leggere. In un esperimento di ablazione su Spike-driven Transformer, sullo stesso dataset, ritardi casuali e fissi portano circa mezzo punto, mentre ritardi appresi quasi due punti e mezzo [18]. Poiché uno

spostamento discreto non è derivabile rispetto al ritardo, durante l'addestramento ogni ritardo è una combinazione morbida di più spostamenti, che una temperatura decrescente rende via via più netta fino a una scelta discreta.

È l'approccio più vicino al mixer causale a più ritardi del modello di riferimento di questa tesi (capitolo 3), che ne condivide l'idea di fondo ma se ne discosta in tre punti. Agisce nei patch embedding anziché su Key e Value, combina i canali tra loro con una matrice per ciascun ritardo anziché trattarli separatamente, e usa ritardi fissi di 1, 2, 4 e 8 passi, uguali per tutti i canali.

Attenzione a blocchi e mixing a ritardi affiancano dunque la memoria neuronale con esigenze diverse di conservazione delle feature. Per confrontarli occorre guardare, oltre che alle prestazioni, a parametri, operazioni e stato persistente, le grandezze introdotte nella sezione successiva.

1.6 Efficienza neuromorfica e vincoli hardware

Un modello spiking viene spesso definito efficiente perché comunica mediante spike binari e presenta attivazioni sparse. Queste proprietà sono il presupposto di una computazione a basso consumo, ma non la garantiscono. Il vantaggio si materializza soltanto quando l'assenza di attività si traduce in lavoro effettivamente evitato, e dipende quindi anche dall'hardware su cui il modello viene eseguito. Questo lavoro non realizza i modelli su un dispositivo, ma caratterizza ogni architettura con grandezze che ne anticipano il costo. La sezione le introduce e ne chiarisce i limiti.

Dalle operazioni alla stima energetica La sezione 1.3.2 ha già introdotto la differenza tra una moltiplicazione seguita da un accumulo (MAC), necessaria quando l'attivazione è un valore a più bit, e un semplice accumulo (AC), che basta quando è uno spike binario. Il risparmio dipende però da quanti spike vengono emessi. Il firing rate ρ misura la frazione di neuroni attivi, mediata sui passi. Una rete con $\rho = 0{,}05$ attiva in media il 5% dei neuroni a ogni timestep, mentre una con $\rho = 0{,}60$ resta formalmente spiking, ma offre molte meno occasioni di saltare le operazioni associate agli zeri.

Il legame tra attività e costo si esprime con il numero di operazioni sinaptiche (*synaptic operations*, SOP). Se uno strato l richiederebbe F_l operazioni in forma densa, in forma spiking ne esegue circa

$$\mathrm{SOP}_l \approx \rho_l \, T \, F_l, \tag{1.38}$$

perché le operazioni vengono ripetute a ogni timestep, ma soltanto la frazione corrispondente agli spike emessi dà luogo a calcolo. Il profilo di questo lavoro

riporta separatamente il conteggio potenziale, in cui ogni possibile spike è presente, e quello pesato per l'attività osservata (sezione 2.6).

QKFormer adotta questa impostazione e traduce i conteggi in una stima dell'energia, assegnando a ciascuna operazione un costo di riferimento [16],

$$E_{\mathrm{SNN}} \approx E_{\mathrm{MAC}} N_{\mathrm{MAC}} + E_{\mathrm{AC}} N_{\mathrm{AC}}. \tag{1.39}$$

Nei valori di riferimento una MAC costa circa cinque volte un AC, e il risultato dipende quindi molto da come si attribuiscono le operazioni alle due voci. Nel calcolo di QKFormer le MAC provengono dal solo primo strato, che codifica l'ingresso in spike, e tutto il resto conta come accumulo. Il conteggio di questo lavoro è più severo, perché considera MAC ogni operazione con un operando non binario, comprese quelle che ricevono le somme residue descritte nella sezione 1.4. Le stime dei due lavori non vanno perciò confrontate direttamente.

La formula va letta per quello che è, un modello aritmetico e non la misura del consumo di un sistema fisico. Restano fuori, tra l'altro, gli accessi in memoria, il trasferimento dei dati e la logica di controllo, e due modelli con la stessa stima possono comportarsi in modo molto diverso una volta realizzati. In questa tesi la stima serve a confrontare architetture a parità di input, e non viene mai presentata come energia misurata.

Stato persistente e buffer temporali I pesi non sono l'unica memoria che un modello spiking occupa. Ogni neurone LIF deve conservare il proprio potenziale di membrana da un passo al successivo, e un'esecuzione in streaming deve leggerlo, aggiornarlo e riscriverlo a ogni timestep. Questo stato persistente segue il numero dei neuroni, e non quello dei pesi. Una convoluzione con pochi parametri, applicata a una mappa ad alta risoluzione, richiede un potenziale per ogni posizione e per ogni canale. Per questa ragione, nella baseline compatta, il primo patch embedding concentra la maggior parte dello stato (sezione 1.4) pur contenendo meno di un quinto dei parametri.

Una capacità temporale esplicita può accrescere ulteriormente questo costo. Un mixer che legge fino a $d_{\max}$ passi indietro deve conservare in un buffer le feature di quei passi, e occupa circa $d_{\max}$ volte lo spazio di una singola feature, mentre un'attenzione a blocchi deve avere disponibili insieme le feature di B timestep. Per questo gli elementi di stato e i buffer temporali vengono riportati sempre accanto ai parametri. Un modulo temporale con pochi pesi può rivelarsi poco conveniente, se costringe a conservare molte feature storiche.

Ne discende la regola progettuale che guida il resto del lavoro. La capacità di un modello va giudicata per il luogo in cui viene spesa, non soltanto per la sua quantità. Ridurre i canali delle prime feature map può liberare pochi parametri, ma ridurre in misura molto maggiore i potenziali di membrana e le operazioni ripetute a ogni timestep. Allo stesso modo, un piccolo mixer temporale su feature a bassa

risoluzione aggiunge pochi parametri e richiede un buffer ridotto, e può risultare più conveniente di un'attenzione temporale sulle feature ad alta risoluzione. In questo consiste un'allocazione della capacità consapevole dell'hardware (*hardware-aware capacity allocation*), che il capitolo 3 applica alla baseline.

Dalla stima all'hardware I contatori descrivono il modello, non la macchina che lo esegue. Una GPU non è ottimizzata per la sparsità, perché le ampie regioni inattive dei tensori spiking non vengono sfruttate, e il continuo trasferimento dei dati tra memoria e unità di calcolo aggiunge un costo considerevole [20]. Il tempo di inferenza su GPU dice quindi poco sull'efficienza neuromorfica di un modello. L'hardware dedicato cerca invece di rendere visibile la sparsità. L'acceleratore di Li et al. per Spike-driven Transformer opera sugli indirizzi degli spike attivi, con addizioni e confronti anziché moltiplicazioni [21], e quello di Du et al. su FPGA non impiega alcun blocco DSP [22]. Il vantaggio dura però finché i circuiti che gestiscono la sparsità costano meno delle operazioni che permettono di evitare [20].

Su una FPGA il costo si esprime infine in risorse del dispositivo, cioè logica (LUT), registri (FF), memoria (BRAM) e blocchi aritmetici (DSP), oltre che in latenza e potenza. Queste grandezze richiedono una sintesi che esula da questo lavoro. I conteggi qui descritti ne sono al più un'anticipazione, e le prospettive di una futura realizzazione vengono riprese nelle conclusioni.

Si chiude così il percorso del capitolo, che ha seguito l'informazione dal segnale prodotto dalla event camera alla sua discretizzazione, dalla memoria neuronale all'attenzione spiking, fino alla modellazione temporale esplicita e al suo costo. L'obiettivo non è costruire il modello con il minor numero di operazioni, né quello più accurato a qualunque costo, ma trovare una soluzione in cui l'informazione temporale aggiuntiva valga le risorse necessarie a rappresentarla. Il capitolo successivo applica questi criteri ai modelli effettivamente implementati, tenendo distinte le scelte relative all'input, i meccanismi architetturali e le condizioni di valutazione.

Capitolo 2

Baseline compatta e metodologia sperimentale

Ogni modifica studiata nei capitoli successivi viene misurata rispetto a un riferimento, e quella misura vale quanto la precisione con cui il riferimento è stato fissato. Questo capitolo lo descrive. Presenta i quattro dataset, il modo in cui il flusso di eventi diventa l'ingresso della rete, l'architettura della baseline compatta, il protocollo con cui i modelli vengono addestrati e confrontati e le grandezze con cui se ne misurano prestazioni e costo.

La parte sperimentale procede per passi successivi, ciascuno dei quali cambia un solo fattore. Il capitolo 3 ridistribuisce la capacità all'interno della baseline, a parità di ingresso, e ne ricava il modello di riferimento. Il capitolo 4 tiene ferma questa architettura e cambia il clock dell'input. Il capitolo 5, con il clock così scelto, arricchisce la memoria temporale. Ciò che resta costante lungo questo percorso è descritto qui una volta per tutte.

2.1 Dataset

La sperimentazione si svolge su quattro benchmark neuromorfici di classificazione, scelti perché affidano all'informazione temporale ruoli molto diversi. In DVS-Lip le classi sono parole pronunciate, distinte da movimenti labiali brevi e spesso simili tra loro, e l'ordine in cui si susseguono le configurazioni della bocca è parte essenziale del segnale. DVS128 Gesture raccoglie gesti ampi e in larga parte ripetitivi, alcuni dei quali si distinguono soltanto per il verso del movimento. In CIFAR10-DVS la dinamica nasce dal modo in cui immagini statiche vengono mostrate al sensore, più che dal loro contenuto. THU-EACT-50-CHL, infine, contiene azioni umane complete, riprese a una risoluzione più alta e con durate molto variabili. Un meccanismo temporale utile in uno di questi scenari non lo è necessariamente

negli altri, ed è proprio questa eterogeneità a rendere il confronto informativo. La figura 2.1 ne mostra un campione per ciascuno.

DVS-Lip È un dataset di lettura labiale a eventi, introdotto insieme alla rete MSTP [23]. Raccoglie 100 parole inglesi pronunciate da 40 parlanti e rilasciate come regioni di 128×128 pixel attorno alla bocca, con una partizione ufficiale che assegna 30 parlanti al training e 10 al test [23]. Le registrazioni durano circa un secondo e contengono pochi eventi, perché il segnale utile si concentra in un'area ristretta e in variazioni di forma molto fini. Il vocabolario include di proposito 25 coppie di parole visivamente simili. Le 50 parole che le compongono e le restanti 50 formano due gruppi, sui quali si calcolano due accuratezze parziali, Acc1 e Acc2.

DVS128 Gesture È stato acquisito con un sensore DVS128 e comprende 11 classi, dieci gesti di mani e braccia e una classe residua che raccoglie movimenti diversi [24]. Le registrazioni coinvolgono 29 soggetti in diverse condizioni di illuminazione, e la partizione ufficiale riserva al test gli ultimi sei. Ogni registrazione contiene più gesti consecutivi, delimitati da annotazioni temporali, e in questo lavoro un campione coincide con un segmento annotato. I segmenti durano in mediana più di sei secondi, ma alcuni superano i diciotto. Diverse classi si distinguono soltanto per il braccio impiegato o per il verso di rotazione, e per questo trasformazioni come il ribaltamento orizzontale, che le scambierebbero tra loro, sono escluse.

CIFAR10-DVS Converte in eventi 10 000 immagini di CIFAR-10, mille per classe [25]. Le immagini, mostrate su un monitor, vengono fatte muovere con un movimento regolare e ripetuto davanti a un sensore DVS128, così che i loro contorni generino eventi [25]. Ne risultano registrazioni di durata quasi uniforme, poco più di un secondo, e le più dense del gruppo. Poiché il contenuto è statico, la struttura temporale riflette soprattutto il movimento imposto durante l'acquisizione, e il dataset fa da termine di confronto in cui è lecito attendersi che la memoria temporale pesi meno. Non esiste una partizione ufficiale, e in questo lavoro i primi cento campioni di ciascuna classe formano un insieme riservato, escluso dallo sviluppo.

THU-EACT-50-CHL THU-EACT-50 è un benchmark per il riconoscimento di 50 azioni umane con event camera [26]. Questo lavoro ne adotta la versione THU-EACT-50-CHL, acquisita con un sensore DAVIS346 a 346×260 pixel e composta da 2 330 registrazioni di 18 soggetti, che gli autori ripartiscono tra training e test senza soggetti in comune. Le durate hanno una coda lunga, da una mediana inferiore ai tre secondi fino a quasi venti, e il numero di eventi varia da circa diecimila a oltre quattro milioni per registrazione. La risoluzione maggiore moltiplica inoltre il numero di posizioni spaziali, e quindi di token, che la rete elabora a ogni passo.

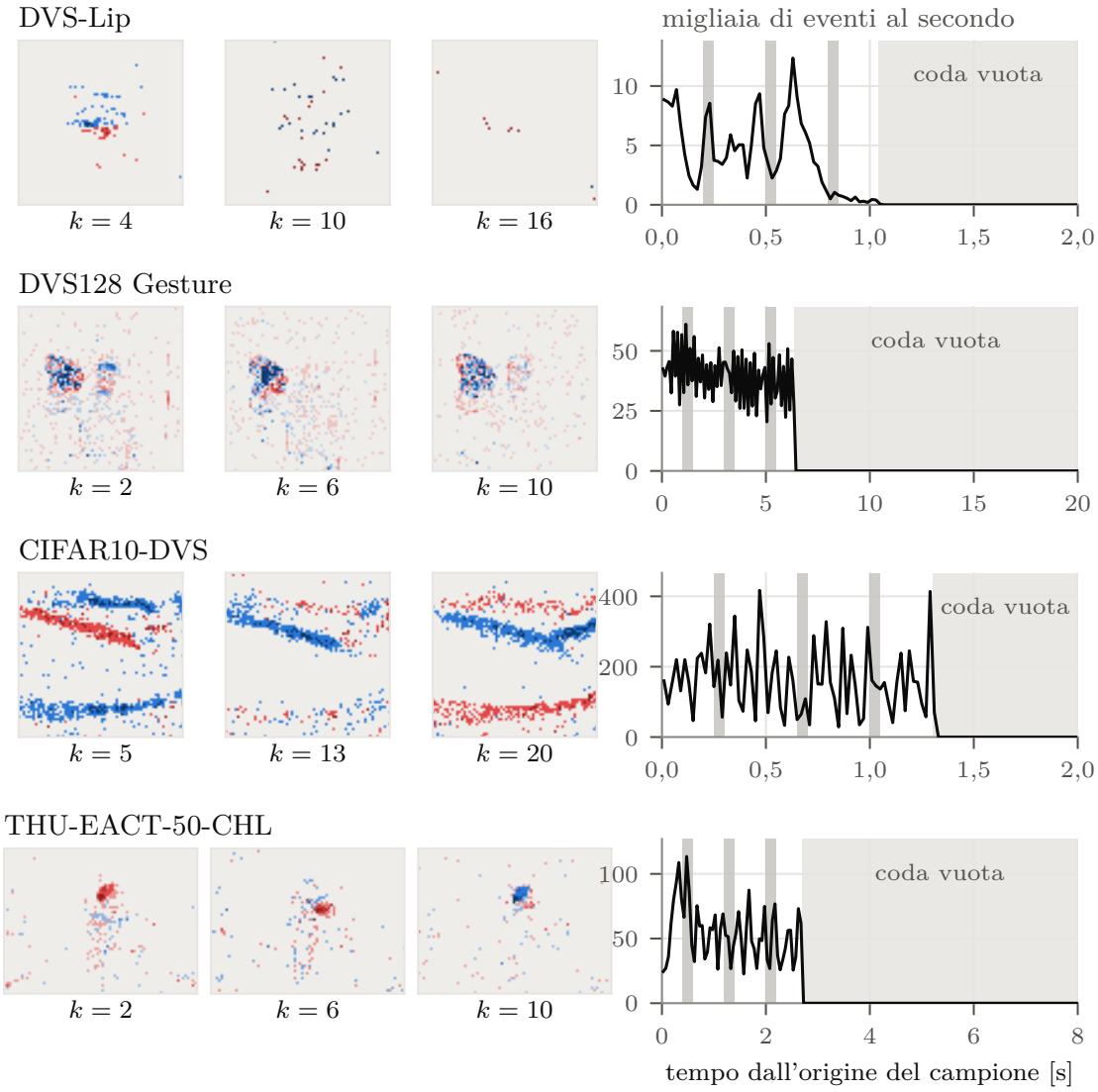


Figura 2.1: Un campione di sviluppo per ciascun dataset, scelto con una durata vicina alla mediana. A sinistra, tre bin della rappresentazione a finestra fissa della sezione 2.2, in rosso dove prevalgono gli eventi ON e in blu dove prevalgono gli OFF. A destra, la frequenza degli eventi sull'intera finestra, con le bande scure in corrispondenza dei bin mostrati e in grigio chiaro la parte successiva all'ultimo evento. I campioni sono la parola *warning* per DVS-Lip, un saluto con la mano destra per DVS128 Gesture e un aereo per CIFAR10-DVS.

Partizioni di sviluppo I test ufficiali non partecipano allo sviluppo. Ogni decisione, dalla configurazione dell'input alla scelta del checkpoint, si basa su una validazione ricavata dal solo training ufficiale, chiamata nel seguito validazione di

Tabella 2.1: Dataset e partizioni di sviluppo. La partizione riservata è il test ufficiale, tranne che per CIFAR10-DVS, per il quale è l'insieme escluso dallo sviluppo. Le durate si riferiscono alle partizioni di sviluppo e riportano il valore mediano e, tra parentesi, il massimo.

Dataset	**Classi**	**Risoluz.**	**Addestr.**	**Valid.**	**Riservati**	**Durata [s]**
DVS-Lip	100	128×128	11 901	2 995	4 975	1,08 (1,89)
DVS128 Gesture	11	128×128	912	264	288	6,36 (18,46)
CIFAR10-DVS	10	128×128	8 100	900	1 000	1,30 (1,46)
THU-EACT-50-CHL	50	346×260	1 387	399	544	2,71 (19,91)

sviluppo (*development validation*). Le partizioni sono fissate prima di qualsiasi esperimento e sono le stesse per tutti i modelli, e ogni confronto avviene quindi sugli stessi campioni. La tabella 2.1 ne riporta le dimensioni insieme alle durate delle registrazioni.

2.2 Rappresentazione dell'input

La baseline riceve il flusso di eventi come frame di conteggio a finestra fissa (*fixed-window count frames*), la più diretta tra le rappresentazioni introdotte nella sezione 1.1. A partire dall'origine del campione si considera una finestra di durata D_{win}, la si divide in T bin della stessa durata secondo l'equazione (1.5), e in ciascun bin si contano, pixel per pixel, gli eventi ON e quelli OFF, come nell'equazione (1.9). Il campione diventa così un tensore

$$X = \big(F_0, \ldots, F_{T-1}\big) \in \mathbb{N}_0^{T \times 2 \times H \times W}, \qquad \Delta t = \frac{D_{\text{win}}}{T}, \tag{2.1}$$

con un canale per ciascuna polarità. Gli istogrammi di eventi a due canali sono tra le rappresentazioni più diffuse nell'apprendimento profondo su dati neuromorfici [27, 2], e la rete li riceve così come sono, senza alcuna normalizzazione. Nel seguito questa scelta verrà indicata come rappresentazione a finestra fissa.

In tutti i dataset la sequenza conta $T = 40$ passi. È un budget comune di passi, non una risoluzione temporale comune, perché la finestra, e con essa la durata dei bin, viene scelta per ciascun dataset in base alle durate di sviluppo (tabella 2.2). Su tre dataset la finestra contiene per intero anche le registrazioni più lunghe. Su THU-EACT-50-CHL, dove la coda delle durate è molto lunga, una finestra estesa fino al campione più lungo avrebbe richiesto bin di mezzo secondo. Quella di 8 s conserva invece bin da 200 ms e il 99% degli eventi, al prezzo di troncare due registrazioni della validazione.

Tabella 2.2: Parametri della rappresentazione a finestra fissa, con $T = 40$ passi in tutti i dataset. Le ultime due colonne si riferiscono alle partizioni di sviluppo. La frazione di finestra occupata è il rapporto tra la durata mediana delle registrazioni e quella della finestra.

Dataset	**Finestra**	**Bin**	**Eventi conservati**	**Finestra occupata**
DVS-Lip	2 s	50 ms	100%	54%
DVS128 Gesture	20 s	500 ms	100%	32%
CIFAR10-DVS	2 s	50 ms	100%	65%
THU-EACT-50-CHL	8 s	200 ms	99,06%	34%

Una finestra prefissata ha però un effetto collaterale, ben visibile nella figura 2.1. Le registrazioni più brevi della finestra ne lasciano vuoti gli ultimi bin, che non sono padding ma osservazioni valide di un intervallo senza attività (sezione 1.1.4). La rete li elabora comunque, e il readout ne include i contributi nella propria media. Su DVS128 Gesture e THU-EACT-50-CHL un campione tipico trascorre così circa due terzi dei passi senza ricevere alcun evento. Il capitolo 4 analizza le conseguenze di questa scelta e la confronta con altri clock. Fino ad allora la rappresentazione a finestra fissa resta invariata, perché ogni differenza osservata nel capitolo 3 possa essere attribuita all'architettura.

2.3 Architettura della baseline compatta

La baseline compatta è un'implementazione scritta per questo lavoro, che conserva i principi di QKFormer (sezione 1.4) riducendone drasticamente la scala. Il backbone ha due stadi gerarchici (figura 2.2a). Un primo patch embedding porta l'ingresso a un ottavo della risoluzione con 64 canali, un blocco con Q-K Token Attention elabora questa rappresentazione, un secondo patch embedding dimezza ancora la risoluzione portando i canali a 128, e un blocco con spiking self-attention precede il readout. Le differenze rispetto all'architettura originale sono riassunte alla fine della sezione.

Neurone e convenzioni di calcolo Tutti i neuroni della rete sono LIF, discretizzati con lo schema di Eulero dell'equazione (1.20) con $\tau_d = 2$ e dotati del soft reset dell'equazione (1.17) con soglia $V_{\text{th}} = 1$. Il potenziale viene azzerato all'inizio di ogni campione. Fanno eccezione i neuroni che prendono le decisioni dell'attenzione, descritti più avanti, la cui soglia è dimezzata. In addestramento la derivata dello spike è sostituita dalla surrogata sigmoidale dell'equazione (1.23) con $\alpha = 4$, e il termine di reset è escluso dal calcolo del gradiente.

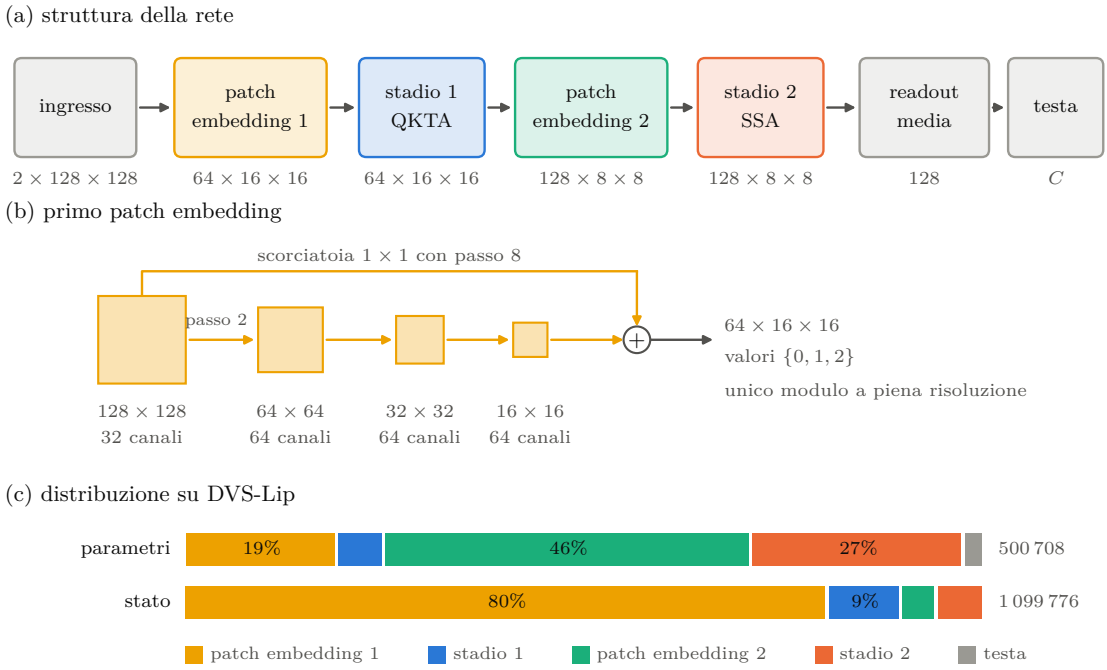


Figura 2.2: Architettura della baseline compatta per un ingresso di 128×128 pixel. (a) Sequenza dei blocchi, con le dimensioni di uscita di ciascuno per ogni passo, nella forma canali $\times$ altezza $\times$ larghezza. (b) Il primo patch embedding, l'unico modulo che lavora a piena risoluzione, con il ramo principale di quattro convoluzioni e la scorciatoia che parte dalla prima mappa di feature. (c) Ripartizione dei parametri e dello stato persistente tra i blocchi su DVS-Lip, secondo la tabella 2.3. I colori dei blocchi corrispondono a quelli del pannello (a).

Con $\tau_d = 2$ il potenziale si dimezza a ogni passo, e la costante di tempo equivalente dell'equazione (1.22) vale circa $1{,}44\,\Delta t$. Lo stesso neurone dimentica quindi in tempi fisici molto diversi da un dataset all'altro, in circa 72 ms su DVS-Lip e CIFAR10-DVS, 290 ms su THU-EACT-50-CHL e 720 ms su DVS128 Gesture. È un primo segno di quanto il clock dell'input fissi anche l'orizzonte della memoria neuronale, un tema che il capitolo 4 riprende.

Convoluzioni, normalizzazioni e proiezioni lineari vengono applicate a ogni passo in modo indipendente, come se i passi fossero campioni distinti. L'unico canale attraverso cui l'informazione passa da un istante al successivo è quindi lo stato dei neuroni, come in QKFormer (sezione 1.4). Anche i collegamenti residui seguono QKFormer (sezione 1.4). Sommano spike a valle dei neuroni, e i tensori scambiati tra i blocchi contengono perciò piccoli interi anziché valori binari, con conseguenze dirette sul conteggio delle operazioni (sezione 2.6).

Primo patch embedding Il primo patch embedding traduce i conteggi in spike e riduce la risoluzione di un fattore otto (figura 2.2b). Il ramo principale è una cascata di quattro convoluzioni 3×3, ciascuna seguita da batch normalization e da un neurone LIF. La prima porta i canali da 2 a 32 a piena risoluzione, le altre tre dimezzano a turno la risoluzione e portano i canali a 64. In parallelo, una scorciatoia preleva le feature del primo strato e le porta direttamente alla risoluzione finale con una convoluzione 1×1 di passo 8. Le due uscite, entrambe spiking, vengono sommate,

$$E_k^{(1)} = M_k^{(1)} + R_k^{(1)} \in \{0{,}1{,}2\}^{64 \times \frac{H}{8} \times \frac{W}{8}}, \tag{2.2}$$

dove $M_k^{(1)}$ e $R_k^{(1)}$ sono gli spike emessi al passo k dal ramo principale e dalla scorciatoia. È il principio della scorciatoia deformata di SPEDS (equazione (1.35)), con due differenze. Il sottocampionamento avviene con convoluzioni a passo 2 anziché con max-pooling, e la scorciatoia attraversa l'intero cambio di scala in un solo salto. È anche l'unico modulo della rete che opera a piena risoluzione, e il suo peso sul bilancio complessivo emerge più avanti.

Stadio con Q-K Token Attention Il primo stadio contiene un unico blocco Transformer con Q-K Token Attention. Per un ingresso di 128×128 pixel lavora su $16 \times 16 = 256$ token con 64 canali, divisi in 8 teste da 8 canali. Query e Key si ottengono come nell'equazione (1.30), e per ogni testa h e ogni token n la somma degli spike della Query sui canali della testa attraversa un neurone con soglia 0,5,

$$a_{h,n} = \mathrm{SN}_{0,5}\Big(\sum_{d \in h} Q_{n,d}\Big), \qquad X'_{n,d} = a_{h,n}\, K_{n,d} \quad \text{per } d \in h. \tag{2.3}$$

La decisione binaria lascia passare o azzera i canali della Key di quella testa per quel token. Rispetto all'equazione (1.31) ogni token dispone così di otto decisioni indipendenti, una per testa. Il neurone che le produce possiede inoltre uno stato, e la selezione di un token dipende anche dalla storia recente della sua Query. Seguono la proiezione di uscita dell'equazione (1.32), il collegamento residuo e uno spiking MLP che espande i canali da 64 a 128 e li riporta a 64, secondo lo schema delle equazioni (1.33) e (1.34).

Secondo patch embedding e stadio con SSA Il secondo patch embedding segue lo stesso schema a due rami. Il ramo principale applica una convoluzione 3×3 da 64 a 128 canali e una seconda con passo 2, la scorciatoia una convoluzione 1×1 con passo 2. Ne risultano 128 canali a un sedicesimo della risoluzione d'ingresso, cioè 64 token. Il secondo stadio impiega la SSA, con Query, Key e Value spiking (1.26) divise in 8 teste da 16 canali. Per ciascuna testa

$$Y = \mathrm{SN}_{0,5}\Big(\beta\, Q(K^\top V)\Big), \qquad \beta = \frac{1}{\sqrt{16}}, \tag{2.4}$$

cui seguono la proiezione di uscita, il collegamento residuo e uno spiking MLP che porta i canali da 128 a 256 e di nuovo a 128. In assenza di softmax il prodotto è associativo, e l'implementazione calcola prima $K^\top V$, una matrice 16×16 per testa. Il risultato coincide con quello dell'equazione (1.27), ma la matrice $N \times N$ tra token non viene mai costruita e il costo cresce linearmente con il numero dei token.

Readout Il readout media la rappresentazione finale prima sulle posizioni e poi sui passi,

$$z = \frac{1}{T\,N_2} \sum_{k=0}^{T-1} \sum_{n=1}^{N_2} Z^{(2)}_{k,n}, \qquad \hat{y} = W_c\, z + b_c, \tag{2.5}$$

dove $Z^{(2)}_{k,n} \in \mathbb{R}^{128}$ è l'uscita del secondo stadio al passo k e nella posizione n, N_2 il numero dei suoi token, W_c e b_c i parametri della testa lineare. Le feature mediate appartengono al flusso residuo e non sono binarie. La media comprende tutti i T passi, anche quelli successivi all'ultimo evento, e la decisione riassume così l'intera sequenza, con lo stesso peso per ogni passo.

Bilancio di parametri e stato La tabella 2.3 riporta, blocco per blocco, le dimensioni, i parametri e lo stato persistente, cioè il numero di potenziali di membrana che devono sopravvivere da un passo al successivo. Il backbone conta 487 808 parametri, ai quali si aggiunge la testa, che dipende dal numero delle classi, per un totale che va da 489 098 su CIFAR10-DVS a 500 708 su DVS-Lip. Su THU-EACT-50-CHL, con un ingresso di 346×260 pixel, i due stadi lavorano su 1 452 e 374 token, e lo stato cresce in proporzione al numero delle posizioni.

La figura 2.2c mostra che parametri e stato si distribuiscono in modo quasi speculare. I due patch embedding contengono circa due terzi dei parametri, e il secondo da solo quasi la metà. Lo stato è invece dominato dal primo, che lavora sulle mappe più risolute e ne ospita oltre l'80%, e nel complesso supera di più del doppio il numero dei parametri. Nel regime compatto capacità e costo si concentrano dunque lontano dall'attenzione, ed è da questa osservazione che parte il capitolo 3.

Differenze rispetto a QKFormer Rispetto alla configurazione neuromorfica di QKFormer (sezione 1.4), la baseline conserva la sequenza di un blocco QKTA e di un blocco SSA, il patch embedding con scorciatoia e la separazione tra memoria neuronale e interazioni all'interno del passo. Cambiano la scala e alcuni dettagli costruttivi, riassunti nella tabella 2.4. La differenza nel numero degli stadi è meno marcata di quanto sembri, perché nella configurazione originale il primo stadio non contiene blocchi e si riduce al patch embedding. Più sostanziali sono la larghezza massima, dimezzata da 256 a 128 canali, il numero dei parametri, sceso a circa un

Tabella 2.3: Struttura della baseline compatta per un ingresso di 128×128 pixel. Le uscite sono indicate come canali $\times$ altezza $\times$ larghezza per ciascun passo. Lo stato conta i potenziali di membrana di tutti i neuroni del blocco. I parametri della testa dipendono dal numero C delle classi, e il totale si riferisce a DVS-Lip.

Blocco	**Operazioni principali**	**Uscita**	**Parametri**	**Stato**
Primo patch embedding	4 conv. 3×3 ($2 \to 32 \to 64 \to 64 \to 64$), scorciatoia 1×1 a passo 8	$64 \times 16 \times 16$	95 360	884 736
Stadio 1	QKTA a 8 teste, MLP $64 \to 128 \to 64$	$64 \times 16 \times 16$	29 440	100 352
Secondo patch embedding	conv. 3×3 ($64 \to 128$), conv. 3×3 a passo 2, scorciatoia 1×1 a passo 2	$128 \times 8 \times 8$	230 144	49 152
Stadio 2	SSA a 8 teste, MLP $128 \to 256 \to 128$	$128 \times 8 \times 8$	132 864	65 536
Readout e testa	media su spazio e tempo, lineare $128 \to C$	C	$129\,C$	0
Totale	DVS-Lip, $C = 100$		500 708	1 099 776

Tabella 2.4: Confronto tra la configurazione neuromorfica di QKFormer, secondo la descrizione della sezione 1.4, e la baseline compatta.

Aspetto	**QKFormer, configurazione neuromorfica**	**Baseline compatta**
Stadi e blocchi	tre stadi, blocchi {0,1,1}	due stadi, un blocco ciascuno
Attenzione	QKTA, poi SSA	QKTA, poi SSA
Larghezza massima	256 canali	128 canali
Parametri	circa 1,50 milioni	da 489 098 a 500 708
Sottocampionamento	convoluzione, BN, neurone e max-pooling	convoluzioni a passo 2
Passi temporali	16	40

terzo, e la lunghezza delle sequenze, di 40 passi anziché 16. L'implementazione, infine, mantiene il tempo come asse separato prima del neurone di uscita della SSA, il punto in cui gli autori hanno segnalato un errore del codice ufficiale [17], e nella baseline quel problema non si presenta.

2.4 Protocollo sperimentale

Tutti i modelli vengono addestrati da zero con la stessa ricetta, e due modelli messi a confronto differiscono per un solo elemento. Questa sezione descrive la ricetta, i vincoli che ne derivano per i confronti e i modi in cui un componente della rete può essere interrogato.

Ricetta di addestramento La ricetta, riassunta nella tabella 2.5, è comune a tutti i modelli e a tutti i dataset. L'ottimizzatore è AdamW [28], con un tasso di apprendimento che cresce nelle prime epoche e decresce poi con un profilo cosinusoidale [29] lungo 128 epoche. La funzione di perdita è l'entropia incrociata con label smoothing [30], la norma del gradiente viene limitata [10] e i calcoli avvengono in precisione mista [31]. L'unica augmentation è un ribaltamento orizzontale su DVS-Lip. Sugli altri dataset non si applica alcuna trasformazione, e su DVS128 Gesture il ribaltamento sarebbe anche scorretto, perché scambierebbe tra loro classi direzionali. La ricetta non viene adattata ai singoli modelli né ai singoli dataset, e quando una campagna la modifica, la modifica vale per tutti i modelli del confronto.

Microbatch e batch normalization Il batch effettivo di 32 campioni si ottiene accumulando i gradienti di più microbatch, la cui dimensione dipende dalla memoria richiesta da ciascun dataset. L'accumulo rende equivalente il gradiente delle componenti lineari, ma non il comportamento della batch normalization [32], che in addestramento normalizza con le statistiche del solo microbatch corrente. Poiché il tempo è fuso con la dimensione del batch, quelle statistiche aggregano tutti i passi e tutti i campioni del microbatch, come avviene comunemente nelle SNN addestrate direttamente [33]. Ne discendono due conseguenze. Il frazionamento in microbatch fa parte del protocollo, perché quattro microbatch da 8 campioni non equivalgono a un batch da 32, e ogni eccezione viene dichiarata dove si presenta. Inoltre, in addestramento ogni passo viene normalizzato con statistiche che dipendono anche dai passi successivi, e la rete non è strettamente causale, pur essendolo ciascuno dei suoi operatori. In inferenza le statistiche sono fisse e ogni passo torna a dipendere soltanto da quelli precedenti.

Test di sovradattamento Prima di un addestramento completo, ogni nuova configurazione deve superare un test di sovradattamento controllato. Il modello viene addestrato su pochi campioni, quattro per ciascuna di al più 16 classi, e deve imparare a classificarli quasi perfettamente, mantenendo per cinque epoche consecutive un'accuratezza superiore al 95%. Il test non dice nulla sulla capacità di generalizzare, ma mostra che il modello è in grado di apprendere e intercetta a un costo trascurabile errori che altrimenti emergerebbero soltanto alla fine di

Tabella 2.5: Ricetta di addestramento comune a tutti i modelli e a tutti i dataset.

Componente	**Scelta**
Ottimizzatore	AdamW, tasso di apprendimento 3×10^{-4}, weight decay 5×10^{-4}
Schedule	warmup lineare di 4 epoche da 0,01 volte il valore nominale, poi decadimento cosinusoidale fino a 10^{-6}
Durata	128 epoche
Funzione di perdita	entropia incrociata con label smoothing 0,1
Stabilizzazione	limite di 1 sulla norma globale del gradiente
Precisione	precisione mista automatica su GPU
Batch effettivo	32 campioni, ottenuti per accumulo su più microbatch
Augmentation	ribaltamento orizzontale con probabilità 0,5, solo su DVS-Lip
Selezione del checkpoint	epoca con la massima Macro-F1 sulla validazione di sviluppo

un addestramento completo. Una configurazione che non lo supera non viene addestrata.

Confronti tra modelli Due modelli messi a confronto differiscono per un solo elemento, mentre dati, partizioni, ingresso, ricetta e regola di selezione restano identici. Quando l'oggetto dell'esperimento è l'ingresso, è l'architettura a restare ferma. Lo stesso componente, però, può essere interrogato in tre modi, che rispondono a domande diverse.

Un'ablazione riaddestrata rimuove o sostituisce il componente e addestra il modello da zero. Misura che cosa riesca a ottenere l'architettura senza di esso, ed è la sola prova che il componente serva davvero.

Un intervento sul checkpoint modifica invece un modello già addestrato, soltanto in inferenza. Spegnere un ramo, eliminare un ritardo o rimescolare l'ordine dei passi misura quanto la soluzione appresa dipenda da quel componente, non se il componente sia necessario. Una rete addestrata con un modulo può essersi adattata alla sua presenza e degradare quando lo si rimuove, anche se un'architettura che ne è priva avrebbe raggiunto da zero un risultato analogo. Il ruolo dell'attenzione nel secondo stadio viene esaminato combinando i due punti di vista, con un test di necessità che la disattiva sul checkpoint addestrato e un test di sostituibilità che la rimpiazza con un mixer spaziale locale e riaddestra il modello.

Un controllo di capacità, infine, separa l'effetto di un operatore da quello delle risorse che aggiunge. Ogni nuovo modulo porta con sé parametri e operazioni, e un miglioramento potrebbe dipendere soltanto da questi. Il confronto con un'alternativa di costo comparabile ma priva della struttura in esame, come un

allargamento uniforme della rete o un percettrone multistrato con lo stesso numero di parametri, permette di attribuire il guadagno alla struttura e non alla quantità.

2.5 Valutazione e incertezza

Metriche La metrica principale è la Macro-F1, cioè la media non pesata, sulle C classi, degli F1 delle singole classi [34],

$$\text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C}\frac{2P_cR_c}{P_c + R_c}, \tag{2.6}$$

dove P_c e R_c sono la precisione e il richiamo della classe c. A differenza dell'accuratezza, la Macro-F1 attribuisce a tutte le classi lo stesso peso, indipendentemente dalla loro numerosità, e penalizza un modello che ottenga un buon risultato complessivo trascurando alcune classi.[1] Accanto a essa si riportano l'accuratezza e, su DVS-Lip, le accuratezze Acc1 e Acc2 dei due gruppi di parole descritti nella sezione 2.1.

Selezione del checkpoint Ogni modello viene valutato sulla validazione di sviluppo al termine di ciascuna epoca, e il checkpoint conservato è quello con la Macro-F1 più alta. Selezione e misura avvengono quindi sugli stessi dati, e il valore riportato è una stima ottimistica della prestazione su dati nuovi, tanto più quanto più la curva di validazione oscilla. Per questo si registrano anche la media e la deviazione standard della Macro-F1 nelle ultime 16 epoche, un indicatore di stabilità meno esposto alla scelta del massimo.

Incertezza dei confronti Quando due modelli vengono valutati sulla stessa validazione, le loro predizioni sono appaiate campione per campione. Il test esatto di McNemar considera soltanto i campioni su cui i due modelli discordano, e verifica se gli errori corretti dall'uno e quelli introdotti dall'altro siano sbilanciati [35, 36]. Il bootstrap appaiato ricampiona la validazione e ricava un intervallo di confidenza al 95% per la differenza di accuratezza [37]. I valori di p non sono corretti per i confronti multipli, e vanno letti come indicazioni più che come verdetti.

[1] Su THU-EACT-50-CHL la media comprende tutte le 50 classi della testa, anche se quattro di esse compaiono soltanto in soggetti del training e mancano sia nella validazione di sviluppo sia nel test ufficiale. Il loro F1 è nullo per costruzione, e il valore massimo raggiungibile della Macro-F1 è del 92%. Il vincolo è identico per tutti i modelli e non altera i confronti, ma va tenuto presente nella lettura dei valori assoluti.

Entrambi gli strumenti misurano l'incertezza dovuta alla dimensione finita della validazione, a modelli fissati. Nessuno dei due misura la variabilità dell'addestramento, cioè quanto cambierebbe il risultato ripetendolo con un'inizializzazione diversa, che può essere dello stesso ordine delle differenze tra i modelli confrontati [38]. Le esplorazioni usano perciò un solo addestramento per configurazione, e i loro risultati sono trattati come indicazioni. Le configurazioni destinate a diventare un riferimento vengono ripetute tre volte, e solo per queste si riportano media e deviazione standard.

Valutazione sui prefissi Su DVS-Lip e DVS128 Gesture si misura anche come la decisione si formi nel tempo. Lo stesso checkpoint viene valutato su porzioni iniziali della sequenza, con il readout calcolato sui soli passi osservati, e un indice sintetico, indicato come PrefixAUC, riassume l'andamento dell'accuratezza al crescere della porzione osservata. È una misura diagnostica. Non interviene nella selezione dei modelli e non autorizza a ridurre la finestra di ingresso, perché un modello addestrato su sequenze complete non è stato ottimizzato per decidere su sequenze troncate.

Partizioni riservate Tutte le metriche riportate nei capitoli successivi sono misurate sulla validazione di sviluppo, salvo indicazione esplicita. Le partizioni riservate restano escluse dallo sviluppo, dalla selezione dei checkpoint e dalla caratterizzazione computazionale. L'unica eccezione riguarda alcune analisi diagnostiche, presentate separatamente, in cui checkpoint già selezionati su DVS-Lip e THU-EACT-50-CHL sono stati valutati in sola inferenza sul test ufficiale. Quei risultati descrivono il comportamento di modelli già scelti e non vengono usati per sceglierne altri.

2.6 Caratterizzazione computazionale

Il costo di ciascun modello viene misurato in inferenza sul checkpoint selezionato, usando un sottoinsieme fisso di 64 campioni della validazione di sviluppo, lo stesso per tutti i modelli di un dataset. Le grandezze raccolte sono quelle introdotte nella sezione 1.6, e la tabella 2.6 ne precisa il significato.

La distinzione tra MAC e accumuli segue il tipo di operando, non il tipo di strato. Uno strato che legge spike esegue accumuli, mentre lo stesso strato applicato a conteggi o a somme residue esegue moltiplicazioni vere e proprie. Per questo i collegamenti residui della baseline, che producono piccoli interi (sezione 2.3), pesano sul bilancio, perché una parte delle proiezioni della rete riceve valori non binari.

Tabella 2.6: Grandezze raccolte nella caratterizzazione computazionale di ciascun checkpoint.

Grandezza	**Che cosa conta**
Parametri	pesi addestrabili, testa inclusa
Stato persistente	potenziali di membrana e buffer temporali che devono sopravvivere da un passo al successivo
MAC	moltiplicazioni e accumuli con un operando non binario, come i conteggi in ingresso, le somme residue e le correnti dei moduli temporali
AC potenziali	accumuli con ingresso binario, contati come se ogni possibile spike fosse presente
SOP	accumuli effettivamente innescati dagli spike osservati, comprese le operazioni dell'attenzione, secondo l'equazione (1.38)
Firing rate	frazione di spike emessi da ciascuno strato

Per riassumere i conteggi in un'unica grandezza si adotta la stima dell'equazione (1.39), con i costi di riferimento per operazioni a 32 bit in tecnologia a 45 nm riportati da Horowitz [39], cioè 4,6 pJ per una MAC, 0,9 pJ per un'addizione e 3,7 pJ per una moltiplicazione. La stima viene calcolata in due varianti, una densa, basata sugli accumuli potenziali, e una pesata per l'attività osservata. Come discusso nella sezione 1.6, è una proxy parziale del solo costo aritmetico, utile per confrontare architetture a parità di input e in nessun caso una misura dell'energia consumata. Lo stato persistente, infine, viene riportato sempre accanto ai parametri, perché nella baseline le due grandezze differiscono di più di un fattore due e un modulo con pochi pesi può richiedere un buffer molto più grande.

2.7 Il riferimento nei capitoli successivi

Dataset e partizioni, ricetta di addestramento, regola di selezione del checkpoint, metriche e grandezze di costo restano gli stessi in tutti i capitoli sperimentali. Cambiano invece, da un capitolo all'altro, la rete di partenza e l'ingresso, secondo il percorso descritto all'inizio del capitolo e riassunto nella tabella 2.7. In ogni capitolo varia un solo fattore, e i confronti avvengono sempre con il riferimento di quel capitolo. I nomi con cui modelli e varianti compaiono nel testo sono descrittivi, e la loro corrispondenza con le esecuzioni effettive è riportata in appendice.

Tabella 2.7: Rete di partenza, ingresso e fattore studiato nei capitoli sperimentali. Tutti gli altri elementi del protocollo restano quelli descritti in questo capitolo.

Capitolo	**Rete di partenza**	**Ingresso**	**Fattore studiato**
3, riallocazione del budget	baseline compatta	finestra fissa	distribuzione della capacità nella rete
4, clock dell'input	modello di riferimento	finestra fissa, eventi costanti, lunghezza variabile	rappresentazione dell'ingresso
5, memoria relazionale	modello di riferimento	lunghezza variabile	memoria temporale

Il capitolo successivo parte dal bilancio della sezione 2.3, che concentra stato e operazioni lontano dall'attenzione, e si chiede come spendere meglio un budget di capacità così ridotto.

Capitolo 3

Riallocazione del budget nella baseline compatta

Questo capitolo sarà redatto al termine delle riesecuzioni che isolano i singoli interventi architetturali. La traccia seguente ne definisce i contenuti.

- **Diagnosi della baseline.** Distribuzione di operazioni e stato tra i blocchi, controlli di capacità a larghezza 192 e 256, rimozione della memoria tra passi e momento del readout.
- **Pyramidal spiking patch embedding.** Ridistribuzione del budget nel primo patch embedding.
- **Causal multi-lag channel mixer.** Formulazione, punto di inserimento e rimozione dei ritardi in inferenza come diagnostica di co-adattamento, con quattro ritardi fin dall'inizio.
- **Depthwise spatial mixer.** Sostituzione della Q-K Token Attention a parità di ritardi, controllo con ritardi apprendibili.
- **Stabilità e trasferimento.** Repliche con tre seed per la baseline e per la configurazione finale, confronto sui quattro dataset, controlli di larghezza.
- **Ruolo dell'attenzione.** Test di necessità e di sostituibilità della SSA.
- **Uso condizionale dei ritardi.** Variante con gate di contenuto, controllo statico riaddestrato e trasferimento tra dataset.

Capitolo 4

Il clock dell'input come variabile sperimentale

Nei capitoli precedenti l'ingresso della rete è rimasto fermo. La rappresentazione a finestra fissa, introdotta insieme alla baseline, ha fatto da sfondo a ogni modifica architetturale, e proprio per questo nessuna di quelle modifiche poteva dire qualcosa sull'ingresso stesso. Questo capitolo inverte i ruoli. L'architettura viene congelata, e a cambiare è soltanto il modo in cui il flusso di eventi viene suddiviso in passi.

La questione può sembrare un dettaglio di preprocessing, ma tocca il cuore del problema. La rete conta il tempo in passi. I neuroni perdono una frazione fissa del proprio potenziale a ogni passo, il mixer temporale guarda indietro di un numero fisso di passi, il readout media sui passi. Che cosa sia un passo, però, lo decide il clock dell'input, e con esso si decide che cosa la rete possa davvero ricordare e confrontare. Le pagine che seguono mostrano che questa scelta pesa sulle prestazioni in misura paragonabile a una modifica dell'architettura, e che i suoi effetti cambiano da un dataset all'altro in modi che la sola accuratezza finale non lascerebbe intuire.

Per tutto il capitolo la rete è il modello di riferimento con cui si chiude il capitolo 3. Rispetto alla baseline compatta, il primo patch embedding ha una struttura piramidale, la Q-K Token Attention del primo stadio è sostituita da un mixer spaziale depthwise 3×3, e in entrambi i patch embedding un mixer causale a più ritardi (*causal multi-lag channel mixer*, CMLCM) somma alla corrente z_t di uno strato combinazioni lineari tra canali delle correnti di alcuni passi precedenti,

$$u_t = z_t + \sum_{k=1}^{4} W_k \, z_{t-d_k}, \qquad (d_1, d_2, d_3, d_4) = (1,2,4,8). \tag{4.1}$$

Il secondo stadio conserva la SSA, il readout resta la media su spazio e tempo, e il numero dei parametri rimane attorno al mezzo milione. Di questa architettura

interessa qui un solo aspetto. I ritardi d_k, come la costante τ_d dei neuroni, sono espressi in passi, e il loro significato fisico dipende interamente dal clock.

4.1 I limiti della finestra fissa

La rappresentazione a finestra fissa ha due pregi evidenti, la semplicità e la confrontabilità. Ogni campione produce esattamente 40 passi, ogni passo corrisponde alla stessa durata fisica, e il modello può essere addestrato a batch senza alcun mascheramento. La stessa rigidità che rende comodo questo schema produce però tre effetti collaterali di natura molto diversa. Il primo riguarda ciò che accade dopo la fine della registrazione, il secondo le registrazioni più lunghe della finestra, il terzo ciò che si perde dentro un singolo bin quando il movimento è più rapido del clock.

4.1.1 Code vuote e registrazioni troncate

Ogni registrazione più breve della finestra lascia dopo l'ultimo evento una sequenza di bin vuoti, già incontrata nella sezione 2.2. Misurata sul modello di riferimento, questa coda occupa in mediana il 45% dei passi su DVS-Lip, circa un terzo su CIFAR10-DVS e circa due terzi su DVS128 Gesture e THU-EACT-50-CHL (figura 4.1a). In un campione tipico di DVS128 Gesture la rete trascorre più tempo ad aspettare che a osservare.

Aspettare, però, non significa restare inerti. I potenziali di membrana non si azzerano quando gli eventi finiscono, il mixer temporale continua a restituire le correnti di alcuni passi prima e i collegamenti residui propagano tutto ciò che resta. La figura 4.1b dà la misura del fenomeno. Le operazioni dense, che si ripetono a ogni passo indipendentemente dall'attività, seguono esattamente la quota dei passi vuoti. Ma anche le operazioni sinaptiche innescate dagli spike restano consistenti, tra il 10% e il 28% del totale a seconda del dataset. Una parte non trascurabile del calcolo viene quindi spesa elaborando il silenzio.

Resta da capire se quel calcolo serva a qualcosa, e un intervento sul checkpoint permette di rispondere almeno in parte. Lo stesso modello, addestrato con la finestra fissa, viene valutato interrompendo l'elaborazione al bin che contiene l'ultimo evento e calcolando il readout sui soli passi osservati. Se la coda fosse un riempitivo innocuo, l'accuratezza non dovrebbe cambiare. Accade invece l'opposto (figura 4.2). Su DVS-Lip l'accuratezza crolla di 17,2 punti e su THU-EACT-50-CHL di 14,8, mentre su DVS128 Gesture e CIFAR10-DVS la perdita si ferma a 2,3 e 3,4 punti.

Il modello ha dunque imparato a usare la coda. Le sue decisioni continuano a maturare dopo la fine dell'ingresso, mentre la dinamica interna si assesta e il readout accumula evidenza. Va però ricordato che si tratta di un intervento sul checkpoint,

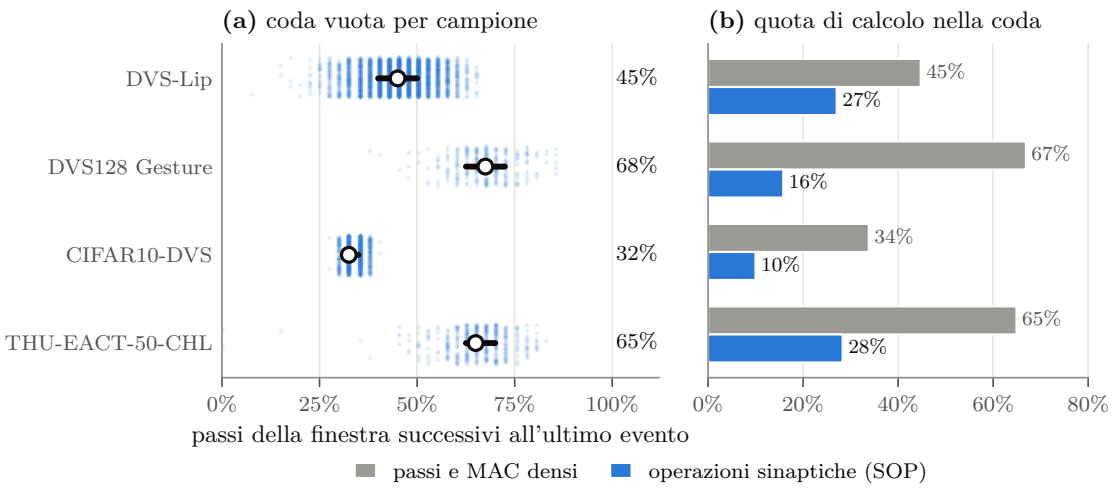


Figura 4.1: Coda vuota della rappresentazione a finestra fissa per il modello di riferimento, sulla validazione di sviluppo. (a) Frazione dei 40 passi successivi al bin che contiene l'ultimo evento, per ciascun campione, con mediana e intervallo interquartile. (b) Quota dei passi, e quindi delle MAC dense, e quota delle operazioni sinaptiche spese dopo l'ultimo evento, sommate sull'intera validazione. Checkpoint con seed 42, valutati in singola precisione.

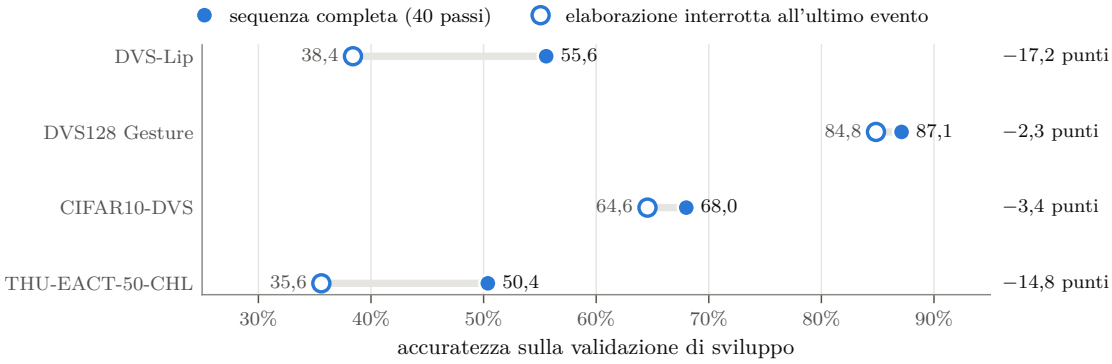


Figura 4.2: Dipendenza dalla coda appresa dal modello addestrato con la finestra fissa. Ogni riga confronta l'accuratezza dello stesso checkpoint quando elabora l'intera finestra e quando l'elaborazione si interrompe al bin che contiene l'ultimo evento. A destra la differenza in punti percentuali. Seed 42, validazione di sviluppo, valutazione in singola precisione.

che misura una dipendenza appresa e non la necessità della coda (sezione 2.4). Interrompere l'elaborazione cambia anche il denominatore della media del readout, e una rete addestrata fin dall'inizio senza coda potrebbe imparare a decidere in modo diverso. È esattamente ciò che verifica la rappresentazione a lunghezza variabile introdotta nella sezione 4.2.

Registrazioni più lunghe della finestra Il problema opposto, quello delle registrazioni che eccedono la finestra, riguarda soltanto THU-EACT-50-CHL, dove la finestra di 8 s tronca due registrazioni della validazione (sezione 2.2). Anche nell'ipotesi più favorevole, in cui la registrazione completa bastasse a correggere entrambe le predizioni, il guadagno non supererebbe mezzo punto di accuratezza. Il troncamento esiste e va dichiarato, ma non può spiegare le differenze discusse più avanti.

4.1.2 Quando il movimento è più rapido del clock

Il terzo effetto è il più sottile, e lo si osserva con chiarezza sulle rotazioni di DVS128 Gesture. Quattro classi del dataset descrivono un braccio che ruota, il destro o il sinistro, in senso orario o antiorario, e si distinguono soltanto per il verso. Con bin da 500 ms e una rotazione che compie un giro in poco più di un secondo, tra il centro di un bin e quello del successivo il braccio percorre circa mezzo giro. A quel punto due posizioni consecutive sono compatibili tanto con mezzo giro in un verso quanto con mezzo giro nell'altro, e il verso diventa ambiguo. È la ragione per cui una ruota ripresa con troppi pochi fotogrammi al secondo sembra girare al contrario, un caso di aliasing temporale che compare quando il campionamento non è almeno due volte più rapido del moto osservato [40].

La figura 4.3 rende il fenomeno concreto su una registrazione della validazione, una rotazione antioraria del braccio sinistro. La posizione del braccio è approssimata dal baricentro degli eventi di ciascun bin, e l'angolo attorno al centro della scena ne ricostruisce la rotazione. Con bin da 20 ms la traccia di riferimento compie quasi sette giri sempre nello stesso verso. Con i bin da 500 ms della finestra fissa, invece, la metà degli intervalli tra bin consecutivi raggiunge o supera il mezzo giro, e la rotazione ricostruita procede nel verso opposto. Le due rappresentazioni alternative della sezione 4.2, che usano passi più brevi, restano quasi sempre sotto la soglia. Il modello addestrato con la finestra fissa classifica proprio questo campione come una rotazione oraria dello stesso braccio.

Il caso è stato scelto a posteriori, e il baricentro è un'approssimazione grossolana della posizione. Inoltre ogni bin accumula conteggi lungo tutta la sua durata, anziché campionare una singola posizione, e l'esempio non dimostra che i tensori delle due classi siano indistinguibili. Il comportamento dei modelli sull'intera validazione va però nella stessa direzione (figura 4.4). Sugli 88 campioni di rotazione il modello

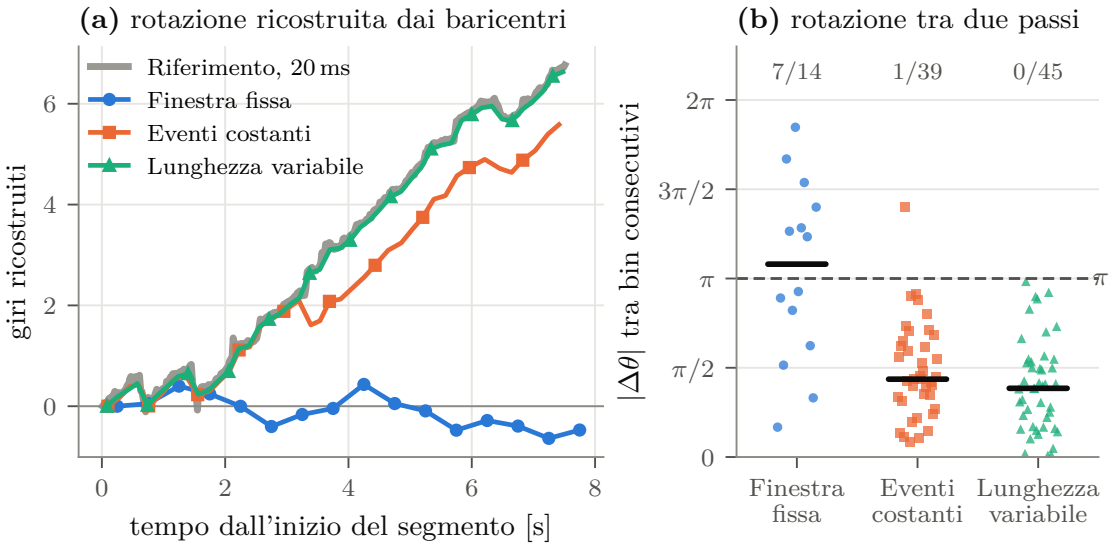


Figura 4.3: Aliasing temporale su una rotazione antioraria del braccio sinistro di DVS128 Gesture (validazione di sviluppo). (a) Giri ricostruiti dall'angolo del baricentro degli eventi di ciascun passo, per una traccia di riferimento a 20 ms e per i tre clock. (b) Variazione angolare della traccia di riferimento tra i centri di passi consecutivi. In alto, il numero di intervalli che raggiungono o superano il mezzo giro. Il baricentro è un'approssimazione della posizione del braccio e il campione è stato scelto a posteriori, sicché la figura illustra il meccanismo senza stimarne la frequenza.

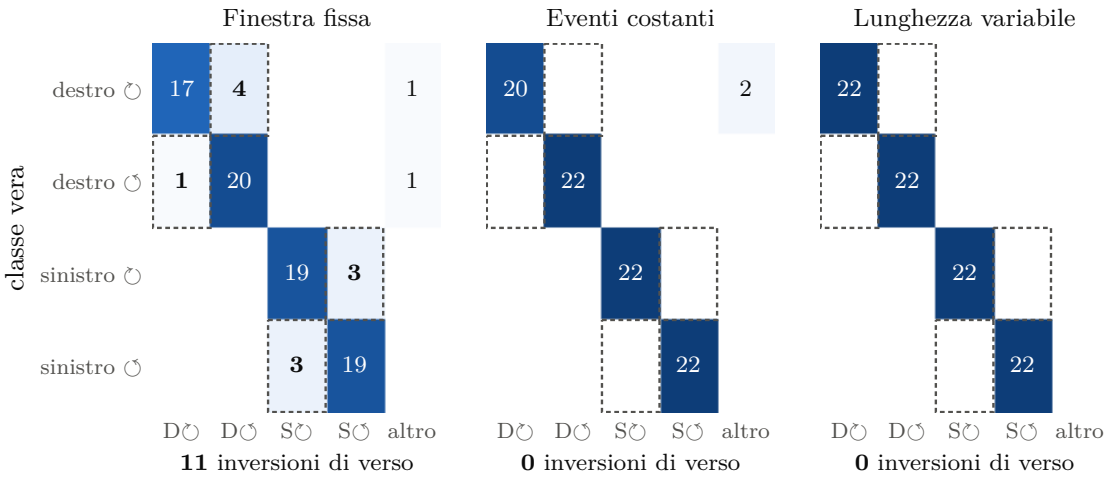


Figura 4.4: Matrici di confusione ristrette alle quattro classi di rotazione di DVS128 Gesture, per il modello di riferimento addestrato con i tre clock (validazione di sviluppo, seed 42). Le righe indicano la classe vera, le colonne la classe predetta, con D e S per il braccio destro e sinistro e le frecce per il verso. Le celle tratteggiate corrispondono alle inversioni di verso a parità di braccio, la colonna *altro* alle predizioni di classi diverse dalle rotazioni.

addestrato con la finestra fissa commette 13 errori, e 11 di questi scambiano il verso a parità di braccio. Con le due rappresentazioni a passo più breve le inversioni scompaiono del tutto. Su questo dataset, circa un terzo di tutti gli errori della finestra fissa è un errore di verso.

4.2 Tre clock a confronto

Le tre criticità hanno origini diverse ma una radice comune. La finestra fissa stabilisce la durata e il numero dei passi senza guardare il campione. Le due alternative esaminate in questa sezione allentano questo vincolo in modi opposti. La prima rinuncia al tempo fisico e taglia la registrazione secondo il numero di eventi. La seconda conserva il tempo fisico ma lascia che il numero dei passi segua la durata della registrazione. La figura 4.5 mostra lo stesso gesto suddiviso secondo i tre schemi, la tabella 4.2 ne confronta le proprietà.

4.2.1 Due alternative alla finestra fissa

Frame a eventi costanti La rappresentazione a eventi costanti (*fixed event-count frames*) divide l'intera registrazione in 40 gruppi consecutivi che contengono lo stesso numero di eventi, a meno di uno, secondo l'equazione (1.8), e in ciascun

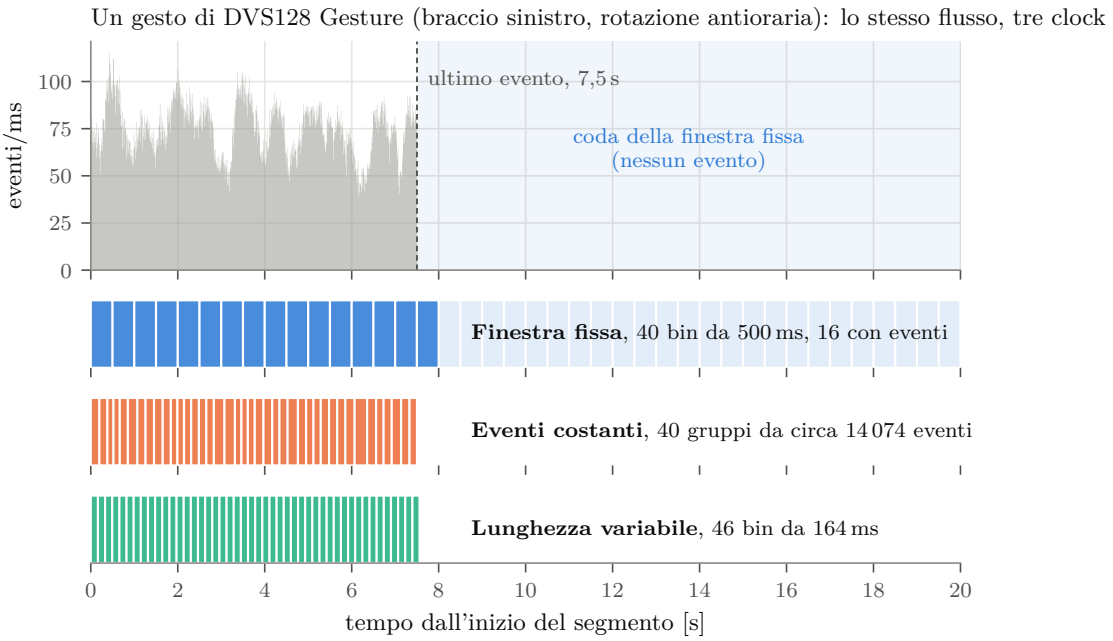


Figura 4.5: La registrazione di DVS128 Gesture della figura 4.3, circa 7,5 s di rotazione, suddivisa secondo i tre clock. In alto, la frequenza degli eventi su intervalli di 10 ms. Sotto, i passi delle tre rappresentazioni, con i passi privi di eventi in tinta chiara. La finestra fissa usa 40 bin da 500 ms, dei quali soltanto 16 contengono eventi. I gruppi a eventi costanti e i bin a lunghezza variabile coprono esattamente la registrazione.

gruppo conta gli eventi ON e OFF di ogni pixel come la finestra fissa. Suddivisioni di questo tipo sono diffuse tanto nella visione a eventi, dove le fette a numero costante di eventi si affiancano a quelle a durata costante [41, 2], quanto nelle librerie per le SNN [8].

Il vantaggio è evidente. Ogni passo riceve la stessa quantità di attività, la coda vuota scompare e una fase molto intensa viene distribuita su più passi. Il prezzo è altrettanto chiaro. La durata fisica di un passo dipende dal campione e, all'interno del campione, dall'intensità dell'attività, sicché un ritardo di 8 passi nel mixer temporale può corrispondere, a seconda del dataset e del tratto di registrazione, a poche decine di millisecondi oppure a interi secondi. Inoltre, per dividere la registrazione in 40 parti uguali occorre conoscere in anticipo il numero totale degli eventi, e la rappresentazione è perciò applicabile soltanto a registrazioni già concluse. Negli esperimenti di questo capitolo il modello e i suoi ritardi restano identici a quelli usati con la finestra fissa.

Durata fissa e lunghezza variabile La rappresentazione a lunghezza variabile (*fixed-duration bins with variable sequence length*) conserva invece il tempo fisico. Tutti i bin di un dataset hanno la stessa durata Δ, ma la sequenza parte dal primo evento e si ferma al bin che contiene l'ultimo, secondo la regola dell'equazione (1.6). Per un campione i con timestamp interi in microsecondi, la lunghezza è

$$T_i = \left\lceil \frac{D_i}{\Delta} \right\rceil, \qquad D_i = t_{\text{last},i} - t_{\text{first},i} + 1, \tag{4.2}$$

dove il microsecondo aggiunto garantisce che anche un evento caduto sul bordo di un bin trovi posto nella sequenza. Il numero dei passi varia così da un campione all'altro, mentre il loro significato fisico resta lo stesso. Non c'è coda vuota, nessun evento viene scartato, e i silenzi interni alla registrazione restano visibili come bin validi privi di eventi.

Resta da scegliere Δ. Per conservare lo stesso budget medio di passi della finestra fissa, la durata dei bin viene calibrata sul solo insieme di addestramento, come il più piccolo intero per cui la lunghezza media non supera 40,

$$\Delta = \min\left\{ d \in \mathbb{N}^+ \;\middle|\; \frac{1}{|\mathcal{D}_{\text{tr}}|} \sum_{i \in \mathcal{D}_{\text{tr}}} \left\lceil \frac{D_i}{d} \right\rceil \leq 40 \right\}, \tag{4.3}$$

dove $\mathcal{D}_{\text{tr}}$ è l'insieme di addestramento. La tabella 4.1 riporta i valori ottenuti. La stessa Δ viene applicata senza modifiche alla validazione, dove la lunghezza media resta vicina a 40 ma i singoli campioni si distribuiscono su intervalli ampi, da 16 a 74 passi su DVS128 Gesture e fino a 285 per la registrazione più lunga di THU-EACT-50-CHL.

Sequenze di lunghezza diversa richiedono, all'interno di un batch, passi di padding. Coerentemente con la distinzione discussa a proposito dell'equazione (1.13),

Tabella 4.1: Calibrazione della rappresentazione a lunghezza variabile. La durata Δ è calibrata sul solo addestramento e applicata invariata alla validazione. I ritardi del mixer temporale sono riscalati per conservare gli orizzonti fisici della finestra fissa, indicati nell'ultima colonna.

Dataset	**Δt fissa**	**Δ variabile**	**Passi in validazione**	**Ritardi**	**Orizzonti [ms]**
DVS-Lip	50 ms	27,42 ms	da 25 a 67	2,4,7,15	55,110,192,411
DVS128 Gesture	500 ms	164,36 ms	da 16 a 74	3,6,12,24	493,986,1972,3945
CIFAR10-DVS	50 ms	32,88 ms	da 37 a 44	2,3,6,12	66,99,197,395
THU-EACT-50-CHL	200 ms	70,00 ms	da 20 a 285	3,6,11,23	210,420,770,1610

questi passi sono esclusi da ogni calcolo. Non contribuiscono alle statistiche della batch normalization, non aggiornano i potenziali di membrana né la memoria del mixer temporale, e non entrano nel readout, che diventa una media sui soli T_i passi validi del campione, nella forma dell'equazione (2.5) con T_i al posto di T. Un bin valido senza eventi, invece, resta un'osservazione e viene elaborato come tale.

Cambiando la durata dei bin cambia anche il significato dei ritardi. Per conservare gli orizzonti fisici del mixer temporale, i ritardi vengono riscalati e arrotondati alla nuova griglia, $d'_k = \mathrm{round}\big(d_k\,\Delta t/\Delta\big)$, dove Δt è la durata dei bin della finestra fissa. Su DVS-Lip, per esempio, i ritardi (1,2,4,8) da 50 ms diventano (2,4,7,15) da 27,4 ms, e il più lungo passa da 400 a 411 ms. La costante τ_d dei neuroni resta invece pari a 2 passi, una scelta su cui si tornerà nella sezione 4.2.3.

4.2.2 Risultati sui quattro dataset

La figura 4.6 riassume il confronto, e la tabella 4.3 ne riporta i valori. Il quadro non è uniforme, ed è proprio la sua disomogeneità a essere istruttiva.[1]

DVS128 Gesture è il caso più netto. Entrambe le alternative superano la finestra fissa di circa otto punti di Macro-F1, 94,8 con gli eventi costanti e 94,2 con la lunghezza variabile contro 86,5, e le repliche con un secondo seed confermano il distacco. Anche la media delle ultime 16 epoche, meno esposta alla scelta del massimo, sale da 81,4 a 93,9 e 93,1. Una parte di questo guadagno si lascia ricondurre direttamente alle rotazioni. La finestra fissa sbaglia 19 campioni in più della lunghezza variabile, e 13 di questi appartengono alle quattro classi di rotazione.

[1]Le esecuzioni di riferimento non condividono tutte lo stesso frazionamento in microbatch, e su DVS-Lip e DVS128 Gesture il confronto tra finestra fissa e lunghezza variabile ne combina due diversi. Dove esistono esecuzioni che differiscono soltanto per il frazionamento, la Macro-F1 cambia di 0,09 punti su DVS-Lip e di 0,14 su CIFAR10-DVS, molto meno degli effetti discussi nel testo. Su THU-EACT-50-CHL tutte le esecuzioni usano lo stesso frazionamento.

Tabella 4.2: Proprietà dei tre clock dell'input.

	Finestra fissa	**Eventi costanti**	**Lunghezza variabile**
Durata di un passo	fissa per dataset	variabile, segue l'attività	fissa per dataset, calibrata sull'addestramento
Numero di passi	40	40	variabile, 40 in media
Parte di registrazione elaborata	quella entro la finestra	intera	intera
Passi dopo l'ultimo evento	sì, la coda vuota	no	no
Significato fisico di un ritardo	costante	variabile	costante
Informazione necessaria in anticipo	nessuna	numero totale di eventi	nessuna

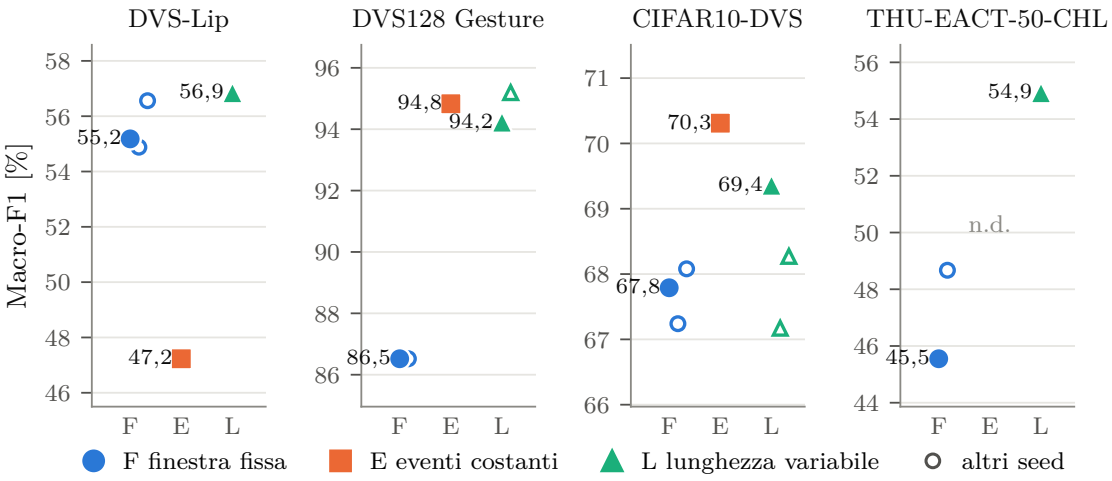


Figura 4.6: Macro-F1 del modello di riferimento con i tre clock, sulla validazione di sviluppo. Il simbolo pieno indica l'esecuzione con seed 42, i simboli vuoti le repliche con altri seed disponibili. Ogni pannello ha una propria scala verticale. Su THU-EACT-50-CHL la rappresentazione a eventi costanti non ha superato le verifiche preliminari.

Tabella 4.3: Accuratezza e Macro-F1 [%] del modello di riferimento con i tre clock, sulla validazione di sviluppo, seed 42. Le repliche disponibili con altri seed danno, in Macro-F1, 54,88 e 56,56 per la finestra fissa su DVS-Lip, 86,52 per la finestra fissa e 95,19 per la lunghezza variabile su DVS128 Gesture, 67,24 e 68,08 per la finestra fissa e 67,17 e 68,27 per la lunghezza variabile su CIFAR10-DVS, 48,67 per la finestra fissa su THU-EACT-50-CHL.

Dataset	**Finestra fissa**	**Eventi costanti**	**Lunghezza variabile**
DVS-Lip	55,53 / 55,18	47,21 / 47,23	56,76 / 56,85
DVS128 Gesture	87,12 / 86,52	95,08 / 94,83	94,32 / 94,23
CIFAR10-DVS	68,00 / 67,79	70,11 / 70,31	69,33 / 69,36
THU-EACT-50-CHL	50,38 / 45,55	non disponibile	61,15 / 54,93

Su THU-EACT-50-CHL la lunghezza variabile porta la Macro-F1 da 45,6 a 54,9 e l'accuratezza da 50,4 a 61,2. Una seconda esecuzione della finestra fissa con un altro seed arriva a 48,7, riducendo il distacco senza annullarlo. La rappresentazione a eventi costanti, invece, non ha superato su questo dataset il test di sovradattamento controllato descritto nella sezione 2.4, e non è stata addestrata in forma completa. È un esito da registrare più che da interpretare. Una possibile ragione, non verificata, è la forte variabilità del numero di eventi, che su THU-EACT-50-CHL spazia su oltre due ordini di grandezza e rende i gruppi di campioni diversi molto disomogenei tra loro.

DVS-Lip separa invece le due alternative. Gli eventi costanti peggiorano nettamente il risultato, da 55,2 a 47,2, mentre la lunghezza variabile arriva a 56,9. Quest'ultimo valore supera di poco tutti e tre i seed della finestra fissa, compresi tra 54,9 e 56,6, e il margine va letto come un piccolo vantaggio, non come un salto. Una lettura plausibile del risultato degli eventi costanti chiama in causa la natura del segnale. Nella lettura labiale contano il ritmo e la durata relativa dei movimenti, e una suddivisione che dilata i tratti quieti e comprime quelli concitati altera proprio queste proporzioni.

CIFAR10-DVS, infine, si comporta come ci si poteva attendere da un dataset con dinamica imposta e durate quasi identiche. Con il seed di riferimento le due alternative guadagnano tra un punto e mezzo e due punti e mezzo, 70,3 con gli eventi costanti e 69,4 con la lunghezza variabile contro 67,8. Con tre seed per ciascuna configurazione, però, le medie della finestra fissa e della lunghezza variabile, 67,7 e 68,3, distano poco più di mezzo punto, meno della variabilità tra seed. Su questo dataset il clock conta poco.

Dove nasce il guadagno? La figura 4.7 divide la validazione in quartili di durata e confronta, quartile per quartile, i due modelli che lavorano in tempo fisico. Su DVS128 Gesture e THU-EACT-50-CHL la lunghezza variabile è migliore in tutti i

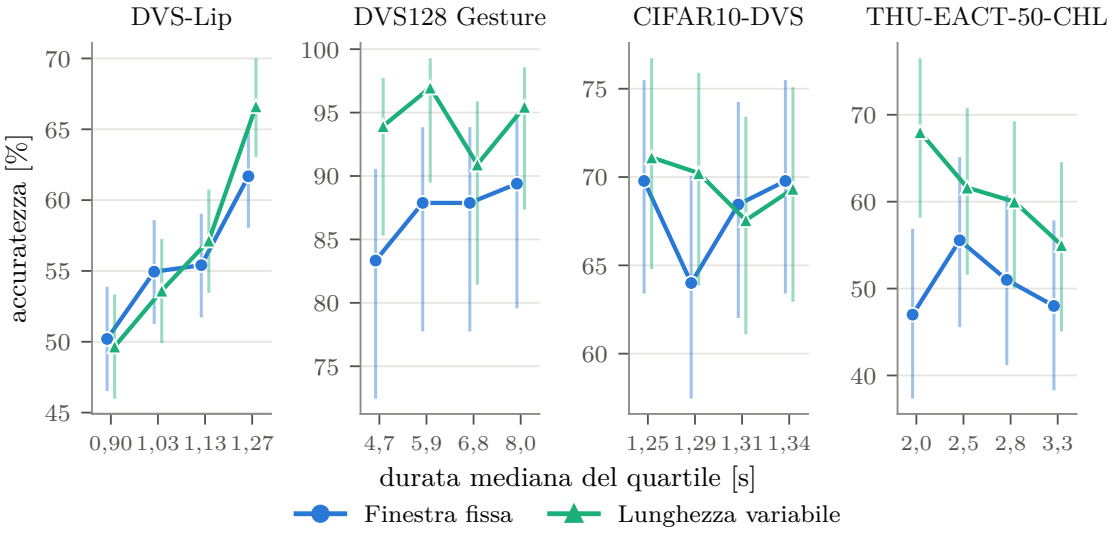


Figura 4.7: Accuratezza del modello di riferimento con la finestra fissa e con la lunghezza variabile, per quartili di durata della registrazione. Le barre verticali indicano l'intervallo di confidenza al 95% di Wilson, ampio dove i quartili contano pochi campioni, come i 66 di DVS128 Gesture. Seed 42, validazione di sviluppo, valutazione in singola precisione.

quartili, e il vantaggio più ampio si concentra sui campioni più brevi, quelli che nella finestra fissa lasciano la coda più lunga. Sul quartile più breve di THU-EACT-50-CHL l'accuratezza passa dal 47% al 68%. Su DVS-Lip il quadro è quasi rovesciato. I campioni più brevi vengono osservati con più passi utili, in mediana 33 contro i 18 della finestra fissa, ma perdono i passi di assestamento della coda, e il loro risultato non cambia. Il guadagno si concentra nel quartile più lungo, dove l'accuratezza sale dal 61,7% al 66,6%. Su CIFAR10-DVS, con durate quasi identiche, le differenze tra quartili rientrano nell'incertezza.

Il confronto appaiato, campione per campione, aggiunge un'ultima sfumatura. Su DVS-Lip la lunghezza variabile corregge 467 errori della finestra fissa, ma ne introduce 431 nuovi. I due modelli hanno un'accuratezza simile e sbagliano in buona parte campioni diversi, un indizio che il clock non sposta soltanto il livello delle prestazioni ma anche il tipo di informazione che la rete sfrutta. Su DVS128 Gesture e THU-EACT-50-CHL lo sbilanciamento è invece netto, con 25 correzioni contro 6 nuovi errori e 72 contro 29.

4.2.3 Assestamento e costante di tempo

Il passaggio dalla finestra fissa alla lunghezza variabile cambia più cose insieme. Elimina la coda, ma riduce anche la durata dei bin, e poiché τ_d resta di 2 passi

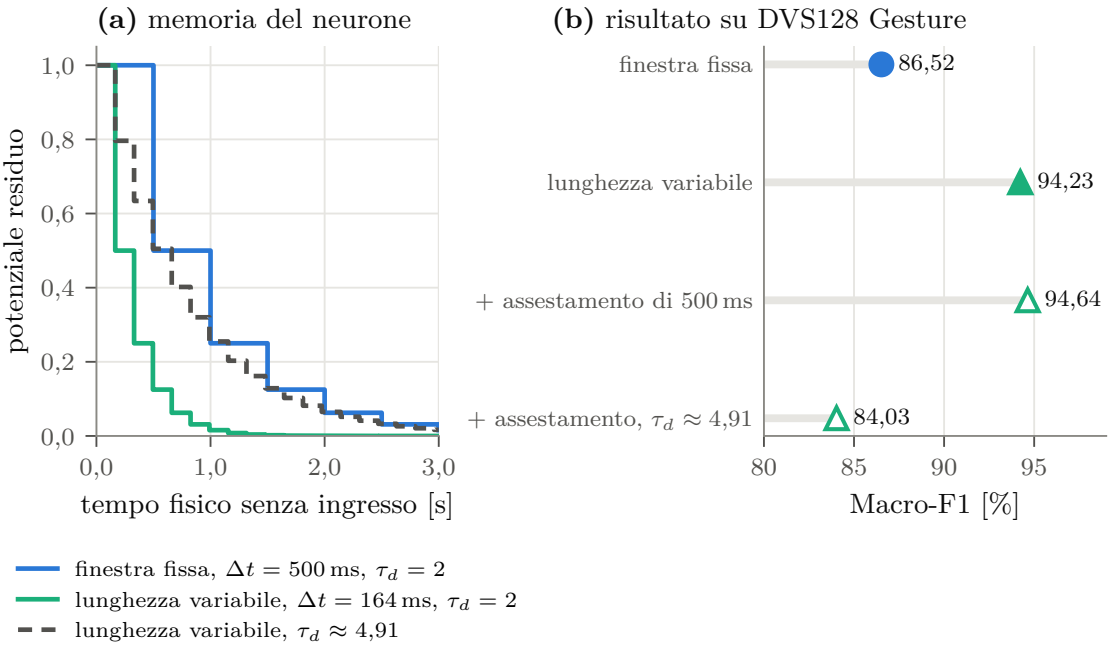


Figura 4.8: Assestamento e costante di tempo su DVS128 Gesture. (a) Frazione del potenziale che sopravvive in assenza di ingresso, in funzione del tempo fisico, per un neurone isolato al di sotto della soglia. Con $\tau_d = 2$ il potenziale si dimezza a ogni passo e, con i bin da 164 ms, svanisce circa tre volte più in fretta che con quelli da 500 ms. Con $\tau_d \approx 4{,}91$ la curva torna a coincidere con quella della finestra fissa ogni 500 ms. (b) Macro-F1 della finestra fissa, della lunghezza variabile e dei due controlli riaddestrati, seed 42.

accorcia la memoria fisica dei neuroni. Su DVS128 Gesture il bin passa da 500 a 164 ms, e il tempo in cui il potenziale si dimezza si riduce nella stessa proporzione (figura 4.8a). Per capire quale di questi cambiamenti conti, due controlli riaddestrati modificano la rappresentazione a lunghezza variabile un elemento alla volta.

Il primo aggiunge dopo l'ultimo evento 500 ms di passi validi senza eventi, restituendo alla rete un breve intervallo di assestamento. La Macro-F1 resta praticamente invariata, 94,6 contro 94,2. Il secondo conserva l'assestamento e porta τ_d a circa 4,91, il valore per cui in 500 ms il potenziale decade esattamente come con la finestra fissa. La memoria fisica dei neuroni torna così quella originale, e la Macro-F1 scende a 84,0, al di sotto della stessa finestra fissa. Le due esecuzioni di controllo condividono tra loro il frazionamento in microbatch, diverso da quello dell'esecuzione canonica, e differiscono soltanto per τ_d.

Il risultato sposta l'attenzione dalla coda alla scala temporale. Su DVS128 Gesture, restituire ai neuroni la memoria fisica della finestra fissa basta ad annullare

il vantaggio della lunghezza variabile, anche con l'intera registrazione a disposizione e passi più fini. La scala della memoria neuronale pesa dunque almeno quanto l'eliminazione della coda. Il dato riguarda un solo dataset e un solo seed e non autorizza una regola generale, ma mostra con chiarezza che il clock dell'input e la dinamica dei neuroni non possono essere progettati separatamente.

4.3 Conseguenze per i confronti successivi

Nessuna delle due alternative prevale ovunque, ma la scelta per i capitoli successivi è netta e cade sulla rappresentazione a lunghezza variabile. Il motivo principale è il clock fisico. Ogni passo ha la stessa durata in tutto il dataset, e ritardi e costanti di tempo conservano un significato preciso, una condizione indispensabile per studiare la memoria della rete. A questo si aggiungono la copertura dell'intera registrazione senza code artificiali e la possibilità di costruire i bin man mano che gli eventi arrivano, senza conoscerne in anticipo il numero totale. Anche sul piano dei risultati la scelta regge il confronto. La lunghezza variabile resta vicina agli eventi costanti su DVS128 Gesture e CIFAR10-DVS, li supera di circa 9,6 punti di Macro-F1 su DVS-Lip e funziona su THU-EACT-50-CHL, dove gli eventi costanti non hanno superato le verifiche preliminari.

Da qui in avanti, il termine di confronto di ogni modifica della memoria è quindi il modello di riferimento addestrato con la rappresentazione a lunghezza variabile, con una Macro-F1 di 56,85 su DVS-Lip, 94,23 su DVS128 Gesture, 69,36 su CIFAR10-DVS e 54,93 su THU-EACT-50-CHL, tutte con seed 42.[2] I ritardi del mixer temporale sono quelli riscalati della tabella 4.1, mentre τ_d resta pari a 2 passi.

Il cambio di clock non rende la rete più economica, ma nemmeno più costosa nel modo che ci si potrebbe aspettare. Poiché Δ è calibrata per mantenere in media 40 passi, il numero medio dei passi, e con esso quello delle MAC dense, resta praticamente invariato. Crescono invece le operazioni sinaptiche, del 30% su DVS128 Gesture e del 28% su THU-EACT-50-CHL, perché i passi che la finestra fissa dedicava alla coda sono ora occupati da eventi. Lo stesso budget di passi viene speso osservando anziché aspettando. Per una futura implementazione hardware compaiono però due costi nuovi. La lunghezza della sequenza non è più limitata a 40 passi e cresce con la durata della registrazione, fino ai 285 passi del campione più lungo di THU-EACT-50-CHL, e con essa crescono latenza ed energia del singolo campione. Inoltre il buffer del mixer temporale, che deve conservare le correnti fino

[2]Su ciascun dataset il riferimento condivide il frazionamento in microbatch con le varianti del capitolo successivo. Su DVS-Lip e CIFAR10-DVS si tratta di un unico microbatch da 32 campioni.

al ritardo massimo, si allunga con i ritardi espressi in passi, da 8 a 15 su DVS-Lip e a 24 su DVS128 Gesture, mentre lo stato dei neuroni resta invariato.

Infine, la rappresentazione a lunghezza variabile non rende la rete invariante al ritmo. Ritardi e costanti di tempo restano espressi in passi, e un gesto eseguito al doppio della velocità occupa la metà dei passi. La robustezza a questo tipo di variazione viene esaminata nell'analisi trasversale dei risultati.

Il capitolo lascia così in eredità un clock in cui un passo ha lo stesso significato fisico dall'inizio alla fine della sequenza e da un campione all'altro. È la condizione che rende sensato chiedere alla memoria della rete di confrontare il presente con il passato a distanze fisiche prestabilite, ed è da qui che parte il capitolo successivo.

Capitolo 5

Memoria temporale relazionale

Con il clock fissato nel capitolo precedente, ogni passo ha una durata fisica nota, e i ritardi del mixer temporale corrispondono a distanze precise nel passato. Questo capitolo si chiede come usare meglio quella memoria. La domanda non riguarda quanto passato conservare, ma che cosa farne quando lo si mette accanto al presente.

Il punto di partenza è il modo in cui il modello di riferimento tratta il passato. Il mixer a ritardi lo richiama e lo somma al presente, ma non li confronta mai direttamente. Il capitolo propone un modulo che separa il problema in tre parti, che cosa del passato conservare, quanto di quel passato esista davvero e come confrontarlo con il presente, e le affronta nell'ordine. Per tutto il capitolo l'ingresso è la rappresentazione a lunghezza variabile, e il termine di confronto è il modello di riferimento addestrato con essa, con una Macro-F1 di 56,85 su DVS-Lip, 94,23 su DVS128 Gesture, 69,36 su CIFAR10-DVS e 54,93 su THU-EACT-50-CHL (sezione 4.3).

5.1 Dal richiamo del passato al confronto tra passato e presente

Nel modello di riferimento il passato entra nella corrente di uno strato attraverso il mixer causale a più ritardi dell'equazione (4.1). A ogni passo il mixer rilegge le correnti di alcuni passi precedenti, le trasforma con una matrice per ciascun ritardo e le somma alla corrente presente. Si tratta, a tutti gli effetti, di un'operazione di richiamo. Il passato viene riportato nel presente, e il neurone che segue integra il risultato insieme a tutto il resto.

La forma della somma ha una conseguenza precisa. Ogni termine dipende da un solo istante, z_t dal presente e $W_k z_{t-d_k}$ dal passato a distanza d_k, e il contributo di ciascuno resta lo stesso qualunque cosa contengano gli altri. Il mixer è additivamente separabile nel tempo. Una relazione tra passato e presente, come il fatto che una certa configurazione delle feature ne segua un'altra, non viene mai calcolata in modo esplicito. Può emergere soltanto più avanti, dalla soglia del neurone, che reagisce quando più contributi si sommano oltre il valore critico, oppure dagli strati successivi. Il mixer mette a disposizione il passato e lascia ad altri il compito di confrontarlo con il presente.

Non è detto che questo sia un limite. Combinazioni lineari seguite da una soglia possono rappresentare, in linea di principio, molte relazioni, e una rete abbastanza grande è in grado di costruirle. In un regime compatto, però, ogni relazione che la rete deve ricavare implicitamente consuma capacità che potrebbe essere spesa altrove. L'idea di questo capitolo è rendere esplicito il confronto, con un termine che dipenda congiuntamente dal passato e dal presente.

L'idea ha un precedente illustre nello studio della percezione del movimento. Il rilevatore di correlazione proposto da Hassenstein e Reichardt per spiegare la risposta al moto di un insetto confronta i segnali di due fotorecettori vicini, moltiplicando il segnale ritardato dell'uno per quello istantaneo dell'altro [42]. Il prodotto, da solo, risponde al moto in entrambe le direzioni. Sottraendo il prodotto speculare, in cui i ruoli dei due recettori sono scambiati, si ottiene una risposta il cui segno indica il verso del movimento,

$$R(t) = s_1(t-\delta)\, s_2(t) - s_2(t-\delta)\, s_1(t), \tag{5.1}$$

dove s_1 e s_2 sono i segnali dei due recettori e δ il ritardo. Nelle sue varianti elaborate lo schema è equivalente ai modelli di energia del movimento, ed è ancora oggi un riferimento per i circuiti biologici che codificano la direzione [43, 44]. Due ingredienti lo distinguono da un semplice richiamo. Il prodotto rende la risposta dipendente dalla coincidenza di passato e presente, la sottrazione del termine speculare la rende sensibile all'ordine. Il capitolo precedente ha mostrato quanto l'ordine possa contare. Sulle rotazioni di DVS128 Gesture, perdere il verso del movimento bastava a confondere classi che per il resto erano identiche (sezione 4.1.2).

Il modulo proposto in questo capitolo trasferisce lo stesso schema dai pixel alle feature latenti (figura 5.1). Il ruolo dei due recettori è affidato a due proiezioni apprese delle feature, quello del segnale ritardato a un riassunto del passato, e il confronto avviene in ogni posizione spaziale tra il presente e più riassunti a distanze diverse. L'analogia motiva il calcolo ma non ne garantisce l'interpretazione. Nulla obbliga i canali della rete a diventare rilevatori di movimento, e le proiezioni possono combinare feature dal significato molto diverso.

Negli Spiking Transformer l'interazione esplicita tra istanti diversi è già stata affrontata, con un'attenzione che correla feature appartenenti a blocchi di passi [19]



Figura 5.1: Dal richiamo al confronto. (a) Il mixer a ritardi del modello di riferimento proietta le correnti di alcuni passi precedenti e le somma alla corrente presente, sicché passato e presente contribuiscono in modo indipendente. (b) Il rilevatore di correlazione di Hassenstein e Reichardt moltiplica il segnale ritardato di un recettore per quello istantaneo dell'altro e sottrae il prodotto speculare, ottenendo una risposta il cui segno dipende dal verso del moto. (c) Il mixer proposto riassume il passato in bande, lo richiama linearmente e lo confronta con il presente attraverso due proiezioni apprese, con un ruolo analogo a quello dei due recettori.

o con la combinazione di informazioni provenienti da più ritardi [18], come discusso nella sezione 1.5. La strada seguita qui è più locale e più economica. Il confronto non passa per una matrice di attenzione, ma per un termine bilineare a basso rango calcolato in ogni posizione, e si colloca nello stesso punto della rete in cui oggi opera il mixer a ritardi.

Da queste osservazioni, e dai capitoli che precedono, discendono alcuni requisiti. Il modulo deve essere causale e leggere soltanto il passato già osservato. Deve conservare la scala fisica introdotta dalla rappresentazione a lunghezza variabile, e quindi riassumere il passato a distanze fisiche prestabilite. Deve sapere quanta storia esista davvero, perché all'inizio di ogni sequenza il passato è incompleto. Deve mantenere il percorso lineare del mixer a ritardi, in modo che il confronto si aggiunga al richiamo anziché sostituirlo. Deve infine costare poco e partire, all'inizializzazione, dalla stessa funzione del modello di riferimento, così che ogni differenza sia frutto dell'addestramento.

I requisiti si traducono nella forma generale

$$u_t = z_t + \sum_{k=1}^{K} b_{k,t}\Big[W_k\, m_{k,t} + U_k\, \rho\Big(m_{k,t}, z_t\Big)\Big], \tag{5.2}$$

dove $z_t \in \mathbb{R}^C$ è la corrente dello strato al passo t in una posizione spaziale, prima del neurone. Il vettore $m_{k,t}$ riassume il passato alla k-esima scala, il coefficiente $b_{k,t}$ tiene conto della storia disponibile, $W_k m_{k,t}$ è il richiamo lineare, erede diretto del mixer a ritardi, e $\rho(m_{k,t}, z_t)$ è una relazione tra passato e presente, riportata nello spazio delle feature dalla matrice U_k. Il modulo che ne risulta viene indicato nel seguito come *interval-relational mixer* (IRM). Il nome descrive una combinazione di ingredienti noti e non rivendica un nuovo tipo di operatore.

La forma separa tre domande che conviene tenere distinte. Che cosa del passato viene conservato, nel riassunto $m_{k,t}$. Quanto di quel passato esiste davvero, nel coefficiente $b_{k,t}$. Come il passato viene confrontato con il presente, nella relazione ρ. Ponendo $\rho = 0$ si ottiene una memoria puramente lineare, che è a sua volta un controllo informativo, mentre cambiando la relazione a parità di tutto il resto si possono confrontare forme di interazione con proprietà diverse. La sezione 5.2 affronta le prime due domande, le sezioni successive la terza.

Per attribuire l'effetto con chiarezza, l'IRM sostituisce il mixer a ritardi in un solo punto della rete, nel secondo patch embedding, sulla corrente della convoluzione con passo 2, dopo la batch normalization e prima del neurone. È una scelta di progetto. A quel livello la rappresentazione ha 128 canali a un sedicesimo della risoluzione d'ingresso, le feature sono più astratte di quelle del primo stadio e le posizioni sono poche, 64 per un ingresso di 128×128 pixel, sicché il costo del confronto resta contenuto. Il primo patch embedding conserva il mixer a ritardi, mentre SSA, MLP e readout restano invariati.

5.2 Memoria causale a intervalli e disponibilità della storia

5.2.1 Dai ritardi puntuali alle bande

Il mixer a ritardi legge il passato in pochi punti. Con la rappresentazione a lunghezza variabile, su DVS-Lip i ritardi (2,4,7,15) campionano 4 dei 15 passi che separano il presente dal ritardo massimo, e su DVS128 Gesture 4 su 24 (figura 5.2). Ciò che accade nei passi intermedi raggiunge il mixer soltanto attraverso le altre vie della rete, in particolare la memoria dei neuroni. Una lettura così rada ha un difetto evidente. Un movimento che avviene un passo prima o un passo dopo cade fuori dal ritardo e sparisce dalla lettura, anche se la sua distanza fisica dal presente è quasi la stessa.

La memoria a intervalli sostituisce i punti con bande contigue di ritardi. Conservando come bordi i ritardi del modello di riferimento, $d_0 = 0 < d_1 < \cdots < d_K$, la k-esima banda raccoglie i ritardi

$$I_k = \{\ell \in \mathbb{N} \mid d_{k-1} < \ell \leq d_k\}, \qquad k = 1, \ldots, K. \tag{5.3}$$

Le bande coprono senza sovrapposizioni tutti i ritardi da 1 a d_K, il presente ne resta escluso e continua ad arrivare dal percorso diretto, e l'orizzonte complessivo coincide con quello del mixer a ritardi. Su DVS-Lip le bande sono [1,2], [3,4], [5,7] e [8,15], da circa 27 a 411 ms nel passato. Su THU-EACT-50-CHL diventano [1,3], [4,6], [7,11] e [12,23], e arrivano a 1,6 s.

Le bande si allargano allontanandosi dal presente. Il passato recente è descritto con la risoluzione di uno o due passi, quello più remoto con medie su otto o dodici passi. È una forma rudimentale di compressione del tempo, in cui la precisione con cui un evento viene collocato diminuisce con la sua distanza, vicina nello spirito alle rappresentazioni a scala logaritmica proposte per la memoria temporale [45]. Un'alternativa sarebbe lasciare che la rete apprenda la posizione dei ritardi, come avviene in alcune SNN dotate di ritardi sinaptici apprendibili [46]. Qui si è preferito conservare i bordi del modello di riferimento, così che memoria a punti e memoria a bande coprano esattamente lo stesso orizzonte.

Medie sulla storia disponibile All'interno di ciascuna banda il passato viene riassunto dalla media delle correnti. All'inizio della sequenza, però, una banda può essere vuota o soltanto in parte riempita, perché i passi corrispondenti precederebbero il primo evento. Indicando con $J_{k,t} = \{\ell \in I_k \mid \ell \leq t\}$ i ritardi della banda effettivamente osservati al passo t, contato a partire da zero, e con $n_{k,t} = |J_{k,t}|$ il

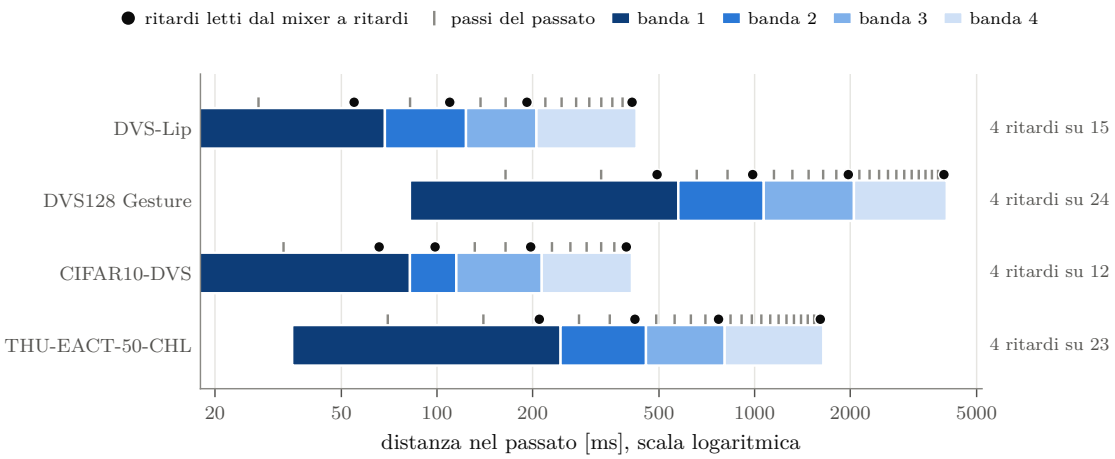


Figura 5.2: Ritardi del mixer a ritardi e bande della memoria a intervalli, in tempo fisico, con la rappresentazione a lunghezza variabile. I cerchi indicano i quattro ritardi letti dal mixer, i trattini grigi tutti i passi del passato fino al ritardo massimo. Le bande, delimitate dagli stessi ritardi, coprono senza lacune l'intero orizzonte e si allargano allontanandosi dal presente. A destra, quanti dei passi del passato vengono letti dal mixer a ritardi.

loro numero, il riassunto è

$$m_{k,t} = \begin{cases} \dfrac{1}{n_{k,t}} \displaystyle\sum_{\ell \in J_{k,t}} z_{t-\ell}, & n_{k,t} > 0, \\ 0, & n_{k,t} = 0. \end{cases} \tag{5.4}$$

La media divide per la storia realmente disponibile e non per la larghezza nominale della banda. Un riassunto parziale non viene così diluito da zeri che non corrispondono ad alcuna osservazione.

La media produce due effetti. Attenua il rumore e rende la memoria tollerante a piccoli spostamenti temporali, perché un movimento anticipato o ritardato di un passo resta quasi sempre nella stessa banda. Il prezzo è la perdita dell'ordine all'interno della banda, che la media rende invisibile. Le bande strette vicino al presente e il percorso diretto limitano il danno dove l'ordine fine conta di più, ma non lo annullano.

5.2.2 Quanta storia è davvero disponibile

Nel mixer a ritardi il passato mancante è rappresentato da zeri nel buffer. Poiché le proiezioni non hanno bias, uno zero produce una corrente nulla, e la scelta è innocua dal punto di vista numerico. Lascia però indistinguibili due situazioni molto diverse, una feature che vale davvero zero e un passato che non esiste ancora.

Il fenomeno non è marginale (figura 5.3). Con le lunghezze della rappresentazione a lunghezza variabile, la banda più lontana dispone di una storia completa nel 62% dei passi su DVS-Lip e nel 70% su CIFAR10-DVS, ma soltanto nel 43% su THU-EACT-50-CHL e nel 39% su DVS128 Gesture, dove in un terzo dei passi la trova del tutto vuota. Quando il ritardo massimo supera la metà della lunghezza tipica della sequenza, come accade su questi due dataset, la memoria più lontana lavora più spesso a regime ridotto che a regime pieno.

Per rendere esplicita questa informazione, ogni banda riceve la propria frazione di storia disponibile, $q_{k,t} = n_{k,t}/|I_k|$, che modula il contributo della banda attraverso il coefficiente

$$b_{k,t} = \mathbf{1}\big[n_{k,t} > 0\big]\; 2\,\sigma\big(\alpha_k\, q_{k,t} + \beta_k\big), \tag{5.5}$$

dove σ è la sigmoide e α_k, β_k sono due parametri apprendibili per banda. All'inizializzazione $\alpha_k = \beta_k = 0$, e il coefficiente vale 1 per ogni banda che contenga almeno un passo. Durante l'addestramento la rete può imparare ad attenuare o ad amplificare un riassunto in funzione di quanto è completo, al costo di $2K = 8$ parametri.

Il coefficiente dipende soltanto dal tempo trascorso dall'inizio della sequenza, non dalle feature, e non va confuso con il gate di contenuto della variante condizionale del mixer a ritardi discussa nel capitolo 3. Rende tuttavia accessibile alla rete l'età

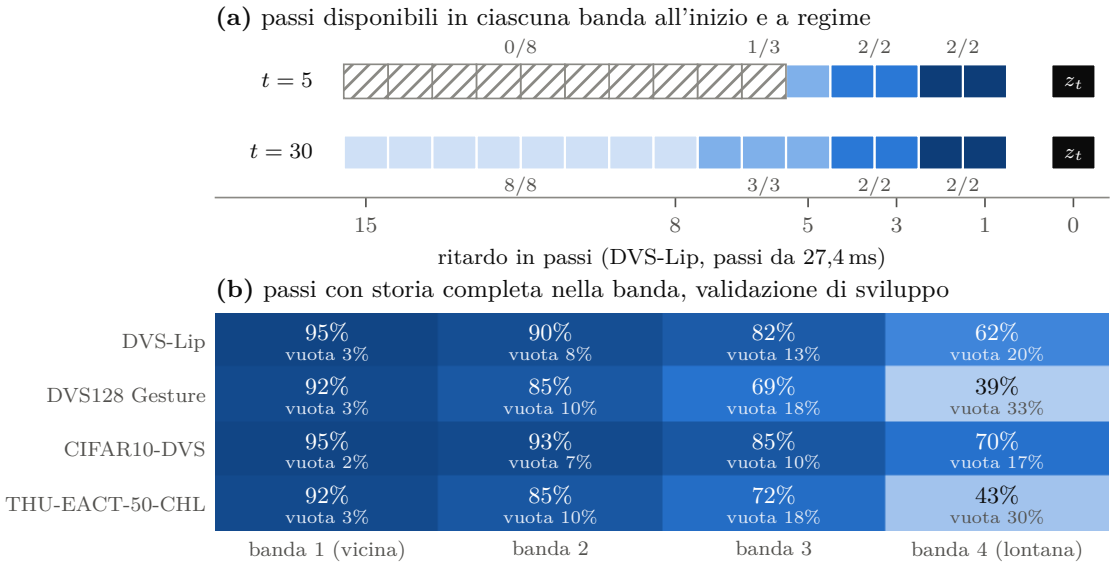


Figura 5.3: Storia disponibile nella memoria a intervalli. (a) Le bande di DVS-Lip al passo $t = 5$, vicino all'inizio della sequenza, e al passo $t = 30$, a regime. I passi tratteggiati precedono il primo evento e non esistono. Sopra e sotto ciascuna banda, il numero di passi osservati rispetto alla sua larghezza. (b) Frazione dei passi della validazione di sviluppo in cui ciascuna banda dispone di una storia completa e, in piccolo, frazione in cui è del tutto vuota.

della registrazione, cioè quanti passi siano trascorsi dal primo evento, ma non la sua durata complessiva, che resta ignota fino all'ultimo passo.

Con questo meccanismo la memoria tiene separate tre condizioni che sul piano numerico potrebbero confondersi, le stesse discusse a proposito dell'equazione (1.13). Il passato che precede il primo evento non esiste e non entra nel conteggio. Un bin valido privo di eventi è un'osservazione di silenzio, partecipa alla media e contribuisce a $n_{k,t}$. Il padding introdotto dal batching, infine, non entra in alcun calcolo e non fa avanzare il conteggio.

5.2.3 Implementazione e costo

Il calcolo delle medie non richiede di sommare ogni volta tutti i passi di una banda. La memoria conserva le correnti degli ultimi d_K passi, esattamente come il buffer del mixer a ritardi, e un contatore della storia disponibile per ciascun campione. Durante l'addestramento le somme di ogni banda si ottengono come differenze di somme cumulative lungo il tempo, sicché il costo per passo non dipende dalla larghezza delle bande. Rispetto al mixer a ritardi restano le stesse K proiezioni $C \times C$ per posizione e passo, applicate ai riassunti anziché ai singoli punti. A queste si aggiungono, per ogni canale, una manciata di operazioni elementari per banda, tra sottrazioni, divisioni e coefficienti, contro le KC moltiplicazioni e accumuli che ogni canale richiede alle proiezioni. Con $C = 128$ il sovraccarico aritmetico resta di oltre un ordine di grandezza inferiore al costo delle proiezioni.

Lo stato da conservare tra un passo e il successivo coincide con il buffer del mixer a ritardi nello stesso punto, $d_K \times C$ valori per posizione, più un contatore per campione. Su DVS-Lip, con $d_K = 15$ e 64 posizioni, sono 122 880 valori per campione. Le formule sono verificate nel codice rispetto a una loro definizione letterale, insieme alla causalità del modulo e all'equivalenza tra l'elaborazione dell'intera sequenza, quella a blocchi e quella passo per passo. Come per il resto della rete, la causalità riguarda il modulo, perché durante l'addestramento la batch normalization aggrega passi diversi (sezione 2.4).

All'inizializzazione le matrici W_k partono dagli stessi valori nulli del mixer a ritardi e i coefficienti di disponibilità valgono 1, sicché il modulo restituisce esattamente la corrente presente. Poiché anche il mixer a ritardi parte da matrici nulle, e i pesi condivisi vengono generati con lo stesso seed in un flusso pseudo-casuale separato da quello del nuovo modulo, la rete con la memoria a intervalli parte esattamente dalla stessa funzione del modello di riferimento.

La memoria a intervalli, presa da sola, resta un richiamo. Offre un riassunto del passato più completo e più robusto dei punti isolati, e sa quanto di quel passato esista davvero, ma lo restituisce al presente come una somma. Anticipando un risultato discusso più avanti, questa sola modifica non migliora il modello di

riferimento, né su DVS-Lip, dove la Macro-F1 passa da 56,85 a 54,67, né su THU-EACT-50-CHL, dove passa da 54,93 a 54,44, e la media delle ultime 16 epoche conferma lo stesso ordinamento. Riassumere meglio il passato non basta. Il passo successivo consiste nel confrontarlo con il presente.

Appendice A

```
    import os
    os.system("echo 1")
```

$\mathcal{O}(n \log n)$
`numpy`

Bibliografia

[1] Patrick Lichtsteiner, Christoph Posch e Tobi Delbruck. «A 128 × 128 120 dB 15 μs Latency Asynchronous Temporal Contrast Vision Sensor». In: *IEEE Journal of Solid-State Circuits* 43.2 (2008), pp. 566–576. DOI: 10.1109/JSSC.2007.914337 (cit. a p. 3).

[2] Guillermo Gallego et al. «Event-Based Vision: A Survey». In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.1 (2022), pp. 154–180. DOI: 10.1109/TPAMI.2020.3008413 (cit. alle pp. 3, 5, 40, 60).

[3] Alex Zihao Zhu, Liangzhe Yuan, Kenneth Chaney e Kostas Daniilidis. «Unsupervised Event-Based Learning of Optical Flow, Depth, and Egomotion». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 989–997. URL: https://openaccess.thecvf.com/content_CVPR_2019/html/Zhu_Unsupervised_Event-Based_Learning_of_Optical_Flow_Depth_and_Egomotion_CVPR_2019_paper.html (cit. alle pp. 8, 9).

[4] Amos Sironi, Manuele Brambilla, Nicolas Bourdis, Xavier Lagorce e Ryad Benosman. «HATS: Histograms of Averaged Time Surfaces for Robust Event-Based Object Classification». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 1731–1740. URL: https://openaccess.thecvf.com/content_cvpr_2018/html/Sironi_HATS_Histograms_of_CVPR_2018_paper.html (cit. a p. 9).

[5] Emre O. Neftci, Hesham Mostafa e Friedemann Zenke. «Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spiking Neural Networks». In: *IEEE Signal Processing Magazine* 36.6 (2019), pp. 51–63. DOI: 10.1109/MSP.2019.2931595 (cit. alle pp. 13, 17).

[6] Bing Han, Gopalakrishnan Srinivasan e Kaushik Roy. «RMP-SNN: Residual Membrane Potential Neuron for Enabling Deeper High-Accuracy and Low-Latency Spiking Neural Network». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 13558–13567 (cit. a p. 14).

[7] Wulfram Gerstner, Werner M. Kistler, Richard Naud e Liam Paninski. *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014. DOI: 10.1017/CBO9781107447615. URL: https://neuronaldynamics.epfl.ch/online/Ch1.S3.html (cit. a p. 14).

[8] Wei Fang et al. «SpikingJelly: An Open-Source Machine Learning Infrastructure Platform for Spike-Based Intelligence». In: *Science Advances* 9.40 (2023), eadi1480 (cit. alle pp. 15, 60).

[9] Yujie Wu, Lei Deng, Guoqi Li, Jun Zhu e Luping Shi. «Spatio-Temporal Backpropagation for Training High-Performance Spiking Neural Networks». In: *Frontiers in Neuroscience* 12 (2018), p. 331. DOI: 10.3389/fnins.2018.00331 (cit. a p. 17).

[10] Razvan Pascanu, Tomas Mikolov e Yoshua Bengio. «On the Difficulty of Training Recurrent Neural Networks». In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. 2013, pp. 1310–1318 (cit. alle pp. 17, 46).

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser e Illia Polosukhin. «Attention Is All You Need». In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017 (cit. a p. 20).

[12] Alexey Dosovitskiy et al. «An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale». In: *International Conference on Learning Representations (ICLR)*. 2021 (cit. a p. 20).

[13] Man Yao, Jiakui Hu, Zhaokun Zhou, Li Yuan, Yonghong Tian, Bo Xu e Guoqi Li. «Spike-driven Transformer». In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36. 2023 (cit. alle pp. 22, 23, 25).

[14] Zhaokun Zhou, Yuesheng Zhu, Chenlin He, Yaowei Wang, Shuicheng Yan, Yonghong Tian e Li Yuan. «Spikformer: When Spiking Neural Network Meets Transformer». In: *International Conference on Learning Representations (ICLR)*. 2023 (cit. a p. 22).

[15] Chenlin Zhou, Liutao Yu, Zhaokun Zhou, Han Zhang, Jingyuan Wang, Huihui Zhou, Zhengyang Ma e Yonghong Tian. «Spikingformer: A Key Foundation Model for Spiking Neural Networks». In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 3. 2026, pp. 2236–2244. DOI: 10.1609/aaai.v40i3.37207 (cit. a p. 25).

[16] Chenlin Zhou et al. «QKFormer: Hierarchical Spiking Transformer using Q-K Attention». In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 37. 2024. DOI: 10.52202/079017-0416 (cit. alle pp. 26, 28–31, 35).

[17] Chenlin Zhou et al. *QKFormer: repository ufficiale e nota sull'implementazione della SSA*. URL: https://github.com/zhouchenlin2096/QKFormer#notice-implementation-issue-in-ssa (visitato il giorno 02/11/2026) (cit. alle pp. 30, 45).

[18] K. Shi, H. Liu, Z. Song, Y. Liu, J. Zhang, S. Wang, J. Wu, M. Zhang e Y. Yang. «Temporal Interaction in Spiking Transformers with Multi-Delay Mixer». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2026 (cit. alle pp. 31, 33, 71).

[19] Dongsung Lee, Yuhang Li, Youngeun Kim, Siyuan Xiao e Priyadarshini Panda. «Spiking Transformer with Spatial-Temporal Attention». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025, pp. 13948–13958 (cit. alle pp. 32, 33, 69).

[20] Mohammad Karamimanesh, Ebrahim Abiri, MohammadReza Shahsavari, Kaveh Hassanli, André van Schaik e Jason Eshraghian. «Spiking Neural Networks on FPGA: A Survey of Methodologies and Recent Advancements». In: *Neural Networks* 186 (2025), p. 107256. DOI: 10.1016/j.neunet.2025.107256 (cit. a p. 36).

[21] Z. Li, W. Mao, S. Zhang, Q. Dong e Z. Wang. «An Efficient Sparse Hardware Accelerator for Spike-Driven Transformer». In: *IEEE Asia-Pacific Conference on Applied Electromagnetics (APACE)*. 2024. DOI: 10.1109/APACE62360.2024.10877394 (cit. a p. 36).

[22] Chao Du, Qifan Wen, Zhiheng Wei e Huazhi Zhang. «Energy Efficient Spike Transformer Accelerator at the Edge». In: *Intelligent Marine Technology and Systems* 2 (2024), p. 24. DOI: 10.1007/s44295-024-00040-5 (cit. a p. 36).

[23] Gengshen Tan, Yang Wang, Huiguang Han, Yiyong Cao, Feng Wu e Zheng-Jun Zha. «Multi-Grained Spatio-Temporal Features Perceived Network for Event-Based Lip-Reading». In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 20094–20103 (cit. a p. 38).

[24] Arnon Amir, Brian Taba, David Berg et al. «A Low Power, Fully Event-Based Gesture Recognition System». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 7243–7252 (cit. a p. 38).

[25] Hongmin Li, Hanchao Liu, Xiangyang Ji, Guoqi Li e Luping Shi. «CIFAR10-DVS: An Event-Stream Dataset for Object Classification». In: *Frontiers in Neuroscience* 11 (2017), p. 309 (cit. a p. 38).

[26] Yue Gao, Jiaxuan Lu, Siqi Li, Nan Ma, Shaoyi Du, Yipeng Li e Qionghai Dai. «Action Recognition and Benchmark Using Event Cameras». In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.12 (2023) (cit. a p. 38).

[27] Ana I. Maqueda, Antonio Loquercio, Guillermo Gallego, Narciso García e Davide Scaramuzza. «Event-Based Vision Meets Deep Learning on Steering Prediction for Self-Driving Cars». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 5419–5427 (cit. a p. 40).

[28] Ilya Loshchilov e Frank Hutter. «Decoupled Weight Decay Regularization». In: *International Conference on Learning Representations (ICLR)*. 2019 (cit. a p. 46).

[29] Ilya Loshchilov e Frank Hutter. «SGDR: Stochastic Gradient Descent with Warm Restarts». In: *International Conference on Learning Representations (ICLR)*. 2017 (cit. a p. 46).

[30] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens e Zbigniew Wojna. «Rethinking the Inception Architecture for Computer Vision». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 2818–2826 (cit. a p. 46).

[31] Paulius Micikevicius et al. «Mixed Precision Training». In: *International Conference on Learning Representations (ICLR)*. 2018 (cit. a p. 46).

[32] Sergey Ioffe e Christian Szegedy. «Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift». In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015, pp. 448–456 (cit. a p. 46).

[33] Hanle Zheng, Yujie Wu, Lei Deng, Yifan Hu e Guoqi Li. «Going Deeper With Directly-Trained Larger Spiking Neural Networks». In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 12. 2021, pp. 11062–11070 (cit. a p. 46).

[34] Marina Sokolova e Guy Lapalme. «A Systematic Analysis of Performance Measures for Classification Tasks». In: *Information Processing & Management* 45.4 (2009), pp. 427–437 (cit. a p. 48).

[35] Quinn McNemar. «Note on the Sampling Error of the Difference between Correlated Proportions or Percentages». In: *Psychometrika* 12.2 (1947), pp. 153–157 (cit. a p. 48).

[36] Thomas G. Dietterich. «Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms». In: *Neural Computation* 10.7 (1998), pp. 1895–1923 (cit. a p. 48).

[37] Bradley Efron e Robert J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993 (cit. a p. 48).

[38] Xavier Bouthillier et al. «Accounting for Variance in Machine Learning Benchmarks». In: *Proceedings of Machine Learning and Systems (MLSys)*. Vol. 3. 2021 (cit. a p. 49).

[39] Mark Horowitz. «1.1 Computing's Energy Problem (and What We Can Do about It)». In: *IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. 2014, pp. 10–14. DOI: 10.1109/ISSCC.2014.6757323 (cit. a p. 50).

[40] Claude E. Shannon. «Communication in the Presence of Noise». In: *Proceedings of the IRE* 37.1 (1949), pp. 10–21 (cit. a p. 56).

[41] Min Liu e Tobi Delbruck. «Adaptive Time-Slice Block-Matching Optical Flow Algorithm for Dynamic Vision Sensors». In: *Proceedings of the British Machine Vision Conference (BMVC)*. 2018 (cit. a p. 60).

[42] Bernhard Hassenstein e Werner Reichardt. «Systemtheoretische Analyse der Zeit-, Reihenfolgen- und Vorzeichenauswertung bei der Bewegungsperzeption des Rüsselkäfers Chlorophanus». In: *Zeitschrift für Naturforschung B* 11.9–10 (1956), pp. 513–524 (cit. a p. 69).

[43] Jan P. H. van Santen e George Sperling. «Elaborated Reichardt Detectors». In: *Journal of the Optical Society of America A* 2.2 (1985), pp. 300–321 (cit. a p. 69).

[44] Alexander Borst e Thomas Euler. «Seeing Things in Motion: Models, Circuits, and Mechanisms». In: *Neuron* 71.6 (2011), pp. 974–994 (cit. a p. 69).

[45] Karthik H. Shankar e Marc W. Howard. «A Scale-Invariant Internal Representation of Time». In: *Neural Computation* 24.1 (2012), pp. 134–193 (cit. a p. 72).

[46] Ilyass Hammouamri, Ismail Khalfaoui-Hassani e Timothée Masquelier. «Learning Delays in Spiking Neural Networks using Dilated Convolutions with Learnable Spacings». In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. a p. 72).